Tesi Triennale

Corso di Laurea Triennale in Scienze della Comunicazione, Dipartimento di Filosofia Comunicazione Spettacolo.

Stance detection sui commenti YouTube come predittore del referendum costituzionale italiano del 22-23 marzo 2026

Abstract

La ricerca indaga se la stance espressa nei commenti YouTube possa fungere da predittore accurato di un esito referendario, prendendo come caso di studio il referendum costituzionale italiano del 22-23 marzo 2026 sulla separazione delle carriere in magistratura (risultato: 53,74% No, 46,26% Sì, affluenza 58,7%).

Sono stati raccolti 196.050 commenti su 1.564 video nell'arco di sette mesi tramite la YouTube Data API v3. Per l'estrazione della stance è stato fine-tunato il modello BERT italiano dbmdz/bert-base-italian-xxl-uncased, con un'innovazione architetturale che usa la coppia padre-risposta come input contestuale, raggiungendo F1 macro 0,762 e accuracy 0,784. L'aggregato deduplicato secondo la regola «un autore, un voto» stima il Sì al 45,9%, con uno scarto di soli 0,40 punti percentuali rispetto al risultato reale; l'aggregato grezzo lo sovrastima di 2,68 pp, fornendo una misura quantitativa diretta del bias della minoranza rumorosa.

Nella serie temporale della stance sono documentati sette turning points, tutti riconducibili a eventi mediatici, video divulgativi virali (Barbero, Travaglio), endorsement politici, passaggi parlamentari, giornalismo investigativo; la segmentazione automatica li recupera con recall del 71% e precisione del 62%. I test di causalità di Granger, condotti previa verifica ADF di stazionarietà su serie giornaliere grezze, restituiscono un'asimmetria netta: la stampa tradizionale Granger-causa la stance referendaria a lag 3 ($p = 0,037$) e 7 giorni ($p = 0,0029$); il consenso al governo Granger-causa la stance referendaria a lag 3 ($p = 0,038$); le direzioni inverse non sono significative. YouTube si configura quindi come termometro dell'opinione pubblica, non come termostato.

I risultati confermano le ipotesi H1 (errore predittivo sotto i 2 punti percentuali) e H3 (asimmetria causale netta), mentre H2 sui turning points, pur qualitativamente solida, non raggiunge la soglia formale di precisione del 70%. Nel complesso il lavoro costituisce una risposta empirica alla critica di Gayo-Avello (2013): una pipeline metodologicamente disciplinata, deduplicazione, contesto conversazionale ed econometria su serie grezze, è in grado di estrarre dai dati social un segnale di nowcasting accurato, particolarmente prezioso durante il blackout demoscopico previsto dalla normativa italiana.

Parole chiave: stance detection, BERT, NLP italiano, referendum costituzionale, agenda-setting, causalità di Granger, nowcasting elettorale, deduplicazione utente, minoranza

rumorosa, opinione pubblica digitale.

Abstract (English)

This study investigates whether the stance expressed in YouTube comments can serve as an accurate predictor of a referendum outcome, taking as a case study the Italian constitutional referendum of 22-23 March 2026 on the separation of judicial careers (result: 53.74% No, 46.26% Yes, turnout 58.7%).

A total of 196,050 comments across 1,564 videos were collected over seven months through the YouTube Data API v3. For stance extraction, the Italian BERT model dbmdz/bert-base-italian-xxl-uncased was fine-tuned, with an architectural innovation that uses the parent-reply pair as contextual input, achieving a macro F1 of 0.762 and an accuracy of 0.784. The deduplicated aggregate, following the "one author, one vote" rule, estimates Yes at 45.9%, a deviation of just 0.40 percentage points from the actual result; the raw aggregate overestimates it by 2.68 pp, providing a direct quantitative measure of the vocal-minority bias.

Seven turning points are documented in the stance time series, all traceable to media events: viral explainer videos (Barbero, Travaglio), political endorsements, parliamentary milestones, and investigative journalism; automatic segmentation recovers them with 71% recall and 62% precision. Granger causality tests, run after ADF stationarity checks on raw daily series, reveal a clean asymmetry: the traditional press Granger-causes referendum stance at lag 3 ($p = 0.037$) and lag 7 days ($p = 0.0029$); government approval Granger-causes referendum stance at lag 3 ($p = 0.038$); the reverse directions are not significant. YouTube thus behaves as a thermometer of public opinion, not a thermostat.

The results confirm hypotheses H1 (predictive error below 2 percentage points) and H3 (clean causal asymmetry), while H2 on turning points, though qualitatively solid, does not reach the formal 70% precision threshold. Overall, the work offers an empirical response to Gayo-Avello's (2013) critique: a methodologically disciplined pipeline, deduplication, conversational context, and econometrics on raw series, can extract an accurate nowcasting signal from social data, particularly valuable during the polling blackout mandated by Italian law.

Keywords: stance detection, BERT, Italian NLP, constitutional referendum, agenda-setting, Granger causality, electoral nowcasting, user deduplication, vocal minority, digital public opinion.

Indice

- Capitolo 1, Introduzione
- Capitolo 2, Quadro teorico e letteratura
- Capitolo 3, Metodologia
- Capitolo 4, Accuratezza predittiva (RQ1)

- Capitolo 5, Turning points e mappatura eventi (RQ2)
- Capitolo 6, Causalità di Granger (RQ3)
- Capitolo 7, Discussione, limiti e sviluppi futuri

Capitolo 1: Introduzione

1.1 Contesto e motivazione

Il referendum costituzionale del 22-23 marzo 2026 sulla separazione delle carriere in magistratura ha rappresentato uno degli appuntamenti referendari più seguiti dell'ultimo decennio italiano. Nei mesi precedenti il voto si è sviluppato un intenso dibattito online, in particolare su YouTube, che ha coinvolto milioni di cittadini.

Il costo e i tempi dei sondaggi tradizionali, uniti al blackout demoscopico obbligatorio degli ultimi 15 giorni pre-voto, creano una finestra informativa dove i social media diventano l'unica fonte osservabile di opinione pubblica.

1.2 Domande di ricerca

- **RQ1** (Predittiva): I commenti YouTube, opportunamente deduplicati, predicono l'esito del referendum con un errore inferiore a 2 punti percentuali?
- **RQ2** (Turning points): I turning points nell'andamento della stance referendaria su YouTube sono concentrati attorno a eventi politici e mediatici identificabili?
- **RQ3** (Granger): Esiste una relazione causale asimmetrica, nel senso di Granger, tra la copertura della stampa tradizionale, il consenso al governo Meloni e la stance referendaria su YouTube?

1.3 Ipotesi falsificabili

Ipotesi	Enunciato	Esito
H1	L'aggregato deduplicato (un utente = un voto) si attesta entro 2pp dal risultato reale	✅ Confermata (0,40 pp)
H2	Almeno il 70% dei turning points individuati automaticamente corrisponde a un evento documentabile (video virale, dichiarazione pubblica, copertura stampa)	⚠️ Non confermata (precisione 62%; recall 71%)
H3	Il test di Granger mostra causalità Stampa→Referendum e Governo→Referendum, ma non il contrario	✅ Confermata (asimmetria pulita)

1.4 Cinque contributi originali

1. **Primo studio** di stance detection su un referendum italiano a partire da commenti YouTube nativi
2. **Metodologia di deduplicazione** per correggere il bias della "minoranza rumorosa" nei social media politici
3. **Nowcasting nel blackout demoscopico**: sfruttare YouTube come fonte legale di opinion polling durante i 15 giorni in cui i sondaggi sono vietati

4. **Analisi di causalità Granger a 3 serie** (YouTube referendum, stampa, consenso governo) con pre-test ADF rigoroso
5. **Dataset pubblico** di 196.050 commenti in italiano su temi politici, annotati automaticamente con stance pro/contro

1.5 Mappa dei capitoli

Capitolo	Contenuto
2	Quadro teorico: agenda-setting, NLP per stance detection, social media come predittori
3	Metodologia: scraping, BERT fine-tuning, deduplicazione, Granger
4	Accuratezza predittiva (RQ1): metriche modello, previsione 0,40 pp, anatomia del bias
5	Turning points e mappatura eventi (RQ2): 7 turning points, video virali, endorsement
6	Causalità di Granger (RQ3): ADF, asimmetria stampa→ref e governo→ref
7	Discussione, limiti e sviluppi futuri

Capitolo 2: Quadro teorico e letteratura

Questo capitolo colloca la ricerca nel suo contesto teorico e ne ricostruisce le radici nella letteratura, lungo tre direttrici che convergono nel disegno empirico dei capitoli successivi: la teoria dell'agenda-setting e del framing, che spiega il rapporto fra copertura mediatica e opinione pubblica; la distinzione, propria del trattamento automatico del linguaggio, fra *sentiment analysis* e *stance detection*; e il dibattito, ormai ventennale, sull'effettiva capacità dei social media di predire i risultati elettorali. La rassegna non è fine a sé stessa: ciascuno dei tre filoni motiva una scelta metodologica precisa, dalla costruzione della serie stampa all'architettura del classificatore, fino alla correzione del bias adottata per la previsione.

2.1 Agenda-setting e framing

Il punto di partenza teorico è la teoria dell'agenda-setting, formulata da McCombs e Shaw (1972) a partire dallo studio della campagna presidenziale americana del 1968. La tesi centrale è che i mezzi di informazione non riescono a dire al pubblico *cosa* pensare, ma sono notevolmente efficaci nel dirgli *su cosa* pensare: selezionando e gerarchizzando i temi, la stampa trasferisce all'agenda pubblica la propria agenda. A questo primo livello, che riguarda la salienza dei temi, la ricerca successiva ha affiancato un secondo livello, il *framing*, che riguarda invece il modo in cui un tema viene presentato. La scelta degli attributi messi in evidenza, del lessico e della cornice interpretativa orienta non solo l'attenzione ma anche la valutazione del pubblico: definire la separazione delle carriere come una garanzia di terzietà del giudice oppure come un assoggettamento della magistratura al potere politico significa attivare, a parità di tema, reazioni opposte.

Nel contesto italiano questi meccanismi operano su un sistema mediatico caratterizzato da una marcata concentrazione proprietaria e da una polarizzazione editoriale storicamente leggibile: la collocazione di ciascuna testata sull'asse politico è nota e relativamente stabile nel tempo, al punto da poter essere usata, in questa tesi, come segnale per la classificazione automatica della polarità degli articoli (§3.1.2). Su questo sfondo si innesta però una discontinuità: nell'era delle piattaforme digitali YouTube si configura come un'arena parallela a quella dei media tradizionali, dove la gerarchia dei temi e dei toni può divergere da quella imposta dall'agenda mainstream. È precisamente questa divergenza che la tesi misura empiricamente. La copertura della stampa, schierata fra il 65% e il 75% in direzione pro-Sì, non ha trascinato con sé il pubblico di YouTube, che si è attestato in una fascia compresa fra il 45% e il 55%: un *gap* di venti-venticinque punti percentuali che suggerisce una frammentazione dell'agenda-setting classico nell'ecosistema dei social. Il Capitolo 6 mostrerà tuttavia che, sul piano della *dinamica* temporale, una traccia dell'effetto di agenda sopravvive: è la stampa a Granger-causare la stance su YouTube, e non viceversa.

2.2 Dalla sentiment analysis alla stance detection

Il secondo filone teorico riguarda il compito di trattamento automatico del linguaggio scelto per misurare la posizione dei commentatori. È cruciale distinguere due compiti che vengono

spesso confusi. La *sentiment analysis* misura la polarità emotiva di un testo, classificandolo come positivo, negativo o neutro: coglie il tono, ma è cieca rispetto al bersaglio dell'opinione. La *stance detection*, al contrario, misura la posizione del parlante *rispetto a un target esplicito*, ad esempio a favore o contro una determinata proposta. La differenza non è terminologica ma operativa: un commento entusiasta nei confronti di un magistrato critico verso la riforma ha sentiment positivo ma stance contraria alla riforma stessa, e solo la stance detection coglie questa distinzione. Proprio per questo il compito adottato nella tesi è la stance detection rispetto al referendum, non la sentiment analysis.

Il riferimento canonico per il compito è il Task 6 di SemEval-2016, che ha fornito il primo benchmark condiviso per la stance detection su testi di social media e ne ha codificato l'impianto valutativo. Sul versante dei modelli, lo stato dell'arte è segnato dall'introduzione di BERT (Devlin et al., 2019), un transformer pre-addestrato che apprende rappresentazioni contestuali del linguaggio e che, con un fine-tuning supervisionato relativamente leggero, raggiunge prestazioni elevate su un'ampia gamma di compiti. Per l'italiano sono disponibili varianti dedicate, fra cui `dbmdz/bert-base-italian-xxl-uncased`, scelto in questa tesi per la sua copertura dell'italiano nativo e colloquiale. Da questa letteratura discende anche la principale scelta architetturale del lavoro, motivata in dettaglio nel Capitolo 3: dare in input al modello non il commento isolato ma la coppia (commento padre, risposta), così da catturare il contesto conversazionale senza il quale molte risposte brevi resterebbero di stance indeterminata.

2.3 I social media come predittori elettorali

Il terzo filone è il dibattito sull'uso dei social media come predittori del voto, che attraversa l'intera letteratura di *computational social science*. Sul fronte ottimista, il lavoro di Tumasjan et al. (2010) sostenne che il semplice volume di menzioni dei partiti su Twitter avesse «saputo prevedere» l'esito delle elezioni federali tedesche, aprendo una stagione di studi che proponeva il conteggio delle menzioni come alternativa rapida ed economica al sondaggio tradizionale. La replica critica più sistematica è quella di Gayo-Avello (2013), che ha confutato punto per punto questa pretesa: i social media, sostiene, non predicono le elezioni ma tendono a replicare i bias di chi li analizza. I problemi sono strutturali e ricorrenti: la non rappresentatività della popolazione online rispetto a quella votante, la presenza di account automatizzati, l'arbitrarietà nella definizione delle finestre temporali e, soprattutto, il peso sproporzionato di una *minoranza rumorosa* di utenti iperattivi che gonfia il segnale apparente di una delle parti.

Questa tesi prende sul serio la critica di Gayo-Avello e vi risponde sul piano metodologico anziché ignorarla. La risposta principale è la deduplicazione «una testa, un voto» descritta nel Capitolo 3, che riconduce ogni utente a una sola opinione per giornata e neutralizza così l'effetto della minoranza rumorosa; ad essa si affianca la misura esplicita e onesta dell'errore di previsione contro il risultato reale delle urne, anziché la sola constatazione *a posteriori* di una presunta capacità predittiva. La scelta di YouTube come piattaforma, infine, non è casuale rispetto a questo dibattito: nel contesto italiano YouTube ha un'alta penetrazione

nel dibattito politico, una presenza di bot inferiore a quella di Twitter/X e, soprattutto, commenti mediamente più lunghi e argomentati, dunque più informativi per un compito di stance detection.

2.4 Posizionamento della ricerca

Alla luce dei tre filoni, la ricerca si colloca alla loro intersezione. Rispetto alla *computational political science* contribuisce un'applicazione di stance detection al discorso politico in lingua italiana; rispetto al *nowcasting* elettorale propone l'uso di dati non strutturati e in tempo reale durante la finestra di blackout demoscopico, quando i sondaggi tradizionali sono vietati dalla legge; rispetto allo studio della *causalità nei media digitali* applica i test di Granger a tre serie temporali di *opinion data* per ricostruire la direzione delle influenze fra stampa, consenso al governo e stance referendaria. Il gap colmato è puntuale e verificabile: nessun lavoro precedente ha applicato la stance detection a un referendum italiano a partire da commenti YouTube nativi, validando la previsione contro il risultato effettivo delle urne. È questo confronto con il dato reale, più ancora delle singole tecniche impiegate, a costituire la cifra del lavoro.

Capitolo 3: Metodologia

Questo capitolo descrive l'intera pipeline empirica della tesi: dalla raccolta dei dati grezzi (scraping su YouTube e ingestione della rassegna stampa) fino ai test di causalità di Granger sulle serie temporali aggregate. La metodologia è organizzata in cinque blocchi: (i) raccolta dati, (ii) annotazione e training di un classificatore di *stance*, (iii) inferenza su larga scala, (iv) aggregazione e deduplicazione, (v) analisi di serie storiche.

L'obiettivo del capitolo è duplice. Da un lato, rendere l'intera pipeline **riproducibile**, ogni scelta è motivata e i parametri operativi sono esplicitati. Dall'altro, rendere visibili le **decisioni metodologiche non banali** che distinguono questo lavoro dalla letteratura precedente: l'uso del contesto padre-risposta come input al classificatore, la deduplicazione "una testa, un voto" come correzione del bias della minoranza rumorosa, e l'uso delle serie grezze (non lisciate) nei test di Granger.

3.1 Raccolta dei dati

3.1.1 YouTube: corpus referendum e corpus governo

I dati YouTube provengono da uno **scraping quota-aware** condotto tramite YouTube Data API v3. La pipeline di raccolta segue quattro passi, identici per i due corpora referendum e governo:

1. **Ricerca video per keyword.** Per il corpus referendum sono state usate query come *"referendum magistratura"*, *"separazione carriere"*, *"riforma Nordio"*, integrate da varianti polarizzate (*"sì al referendum"*, *"no al referendum"*) e da nomi di protagonisti del dibattito (Nordio, Travaglio, Barbero, Gratteri, Di Pietro). Per il corpus governo le query riguardano le keyword di consenso politico al governo Meloni nel periodo 2025-2026.
2. **Raccolta metadata video.** Per ogni video selezionato sono raccolti `video_id`, titolo, canale, data di pubblicazione, view count, like count, numero totale di commenti.
3. **Raccolta dei commenti.** Per ogni video sono scaricati testo, autore (`author_id`), data, like, e, quando presente, l'identificativo del commento padre, che permette di ricostruire la struttura ad albero della discussione.
4. **Deduplicazione tecnica e pulizia.** I duplicati esatti (stesso `comment_id`) vengono rimossi; si scarta il rumore (commenti vuoti, URL puri, spam riconoscibile). Questa è una deduplicazione *tecnica*, distinta dalla deduplicazione *concettuale* "una testa, un voto" descritta in §3.4.

Lo scraper è **quota-aware**: monitora il consumo della quota giornaliera dell'API e sospende automaticamente la raccolta quando si avvicina alla soglia, riprendendo il giorno successivo. Questo accorgimento ha permesso di sostenere una raccolta continuativa di sette mesi senza interruzioni manuali.

Esiti della raccolta:

Corpus	Commenti raccolti	Video unici	Periodo di analisi
YouTube Referendum	196.050	1.564	30 ott 2025, 23 mar 2026
YouTube Governo Meloni	136.624	1.147	25 dic 2025, 23 mar 2026

Dei 196.050 commenti del corpus referendum, raccolti anche da video pubblicati prima dell'avvio della campagna, **175.289 ricadono nella finestra di analisi 30 ottobre 2025, 23 marzo 2026** (approvazione della riforma al Senato fino al giorno del voto); su questo sottoinsieme, e in particolare sui 107.572 commenti con stance interpretabile (POS/NEG), è calcolato l'aggregato predittivo del Capitolo 4. Il corpus governo (136.624 commenti, 55.691 autori unici), raccolto nei circa novanta giorni precedenti il voto, serve invece da terza serie per l'analisi di causalità del Capitolo 6.

3.1.2 La serie della stampa nazionale

La serie *stampa* è costruita a partire da un corpus di articoli pubblicati dalle principali testate nazionali italiane nel periodo della campagna referendaria. L'obiettivo non è un censimento esaustivo della produzione giornalistica, bensì la costruzione di un indicatore giornaliero della *direzione* della copertura mediatica, da contrapporre alla stance espressa dal pubblico su YouTube. L'ampiezza del ventaglio di testate considerate, che abbraccia l'intero spettro dell'allineamento editoriale italiano, riduce il rischio che la serie rifletta la linea di una singola fonte anziché il clima mediatico complessivo, attenuando così il selection bias che affliggerebbe un'aggregazione manuale per testata.

La pipeline di costruzione si compone di quattro passi. Il primo è la **raccolta degli articoli** pertinenti al referendum, selezionati in base a parole chiave sul tema della separazione delle carriere, dell'ordinamento giudiziario e della riforma Nordio. Il secondo è la **classificazione della polarità** di ciascun articolo (pro-Sì, pro-No, neutro), affidata interamente a un modello linguistico di grandi dimensioni (Claude Sonnet 4.6) tramite un prompt dedicato. Dato il volume contenuto del corpus stampa, etichettare direttamente con l'LLM è risultato più efficiente che addestrare un classificatore supervisionato ad hoc, come invece è stato fatto per i corpora YouTube, molto più grandi. Il terzo è l'**aggregazione per testata e per giorno**, con etichettatura di ciascun articolo. Il quarto è il **calcolo della serie temporale** giornaliera della *coverage* pro-Sì e pro-No.

Il corpus risultante è di **5.152 articoli**, raccolti da 85 testate su 91 giorni. Di questi, **3.424 esprimono una polarità netta** (pro-Sì o pro-No) e costituiscono la base per l'indicatore di *coverage*; i restanti sono neutri o di posizione dubbia, ed esclusi dal calcolo della quota pro-Sì. La polarizzazione editoriale, identificata empiricamente, è coerente con la letteratura sui media italiani:

- *Pro-Sì* (favorevoli alla riforma): Libero, Il Giornale, Il Tempo, Il Riformista, Il Dubbio.
- *Contro* (contrari alla riforma): Il Fatto Quotidiano, Domani, Manifesto, L'Unità, La Stampa.

Aggregando i toni a livello di sistema, la stampa pesa fra il **65% e il 75%** in direzione pro-Sì, in media il 68,5% (2.347 articoli pro-Sì su 3.424 polarizzati), un *gap* di 20-25 punti percentuali rispetto a YouTube, dato che ritorna come elemento centrale nella discussione (Cap. 5).

3.2 Annotazione e training del classificatore di stance

3.2.1 Stance detection vs sentiment analysis

La scelta del compito è cruciale. Un classificatore di **sentiment analysis** misura la polarità *emotiva* di un testo (positivo / negativo / neutro): coglie il tono, ma è cieco rispetto al *target* dell'opinione. Un classificatore di **stance detection** misura invece la posizione del parlante *rispetto a un target esplicito*, in questo caso, la riforma referendaria.

La distinzione è operativa, non solo terminologica. Un commento come *"finalmente qualcuno che lo dice"* ha sentiment positivo, ma la sua *stance* sulla riforma è indeterminata senza il contesto del commento padre. Per questo l'architettura adottata in §3.2.4 prende in input la coppia (padre, risposta).

3.2.2 Modello base: BERT italiano

Il modello base è `dbmdz/bert-base-italian-xxl-uncased`, un BERT pre-addestrato su un grande corpus in lingua italiana. Le ragioni della scelta:

- **Pre-training su italiano nativo**, con buone performance sull'italiano colloquiale tipico dei commenti web.
- **Variante `uncased`**: trattando maiuscole e minuscole allo stesso modo, è robusta a uno stile di scrittura informale e disordinato, caratteristica strutturale dei commenti YouTube.
- **Disponibilità open-source** su HuggingFace e dimensione `xxl` compatibile con GPU consumer (Kaggle T4/P100).

Sono stati considerati anche modelli multilingue (mBERT, XLM-R), scartati perché su task italiani di social media il BERT italiano dedicato dà sistematicamente performance superiori, a parità di hardware.

3.2.3 Costruzione del training set: annotazione assistita da LLM e active learning

L'annotazione manuale di un training set sufficientemente ampio è il collo di bottiglia tipico dei progetti di NLP applicato. La strategia adottata in questa tesi è **ibrida** e riguarda la costruzione del *training set* (il *seed*) su cui viene poi addestrato il classificatore. Anziché etichettare a mano migliaia di esempi, il seed è stato annotato quasi per intero da un **modello linguistico di grandi dimensioni** (Claude Sonnet 4.6), impiegato come annotatore automatico, mentre un workflow di **active learning** con interfaccia Gradio è servito come controllo di qualità mirato sui casi più incerti. Il classificatore BERT, addestrato su questo seed, ha poi esteso le etichette all'intero corpus tramite l'inferenza

massiva descritta nel §3.3. In termini di volume, il seed annotato consta di **21.611 commenti** per il corpus referendum e di **6.649 commenti** per il corpus del consenso al governo.

Per l'annotazione automatica sono stati progettati **due prompt distinti**, uno per ciascun dominio: un prompt calibrato sul **dataset referendum**, che etichetta la stance come pro-Sì, pro-No o neutro rispetto alla separazione delle carriere, e un prompt calibrato sul **dataset del consenso al governo**, che etichetta la posizione come favorevole, contraria o neutra rispetto al governo Meloni. La separazione dei due prompt non è un dettaglio implementativo ma una necessità concettuale: il *target* della stance è diverso nei due corpora, e lo stesso commento può essere favorevole al governo e ostile alla riforma (o viceversa). Un prompt unico confonderebbe i due assi e produrrebbe etichette ambigue. Ciascun prompt esplicita la definizione operativa delle tre classi, fornisce esempi rappresentativi e impone un'etichetta secca, riducendo la deriva interpretativa tipica dell'annotazione su larga scala.

Sul prodotto dell'annotazione automatica si è innestato il controllo di qualità via active learning. Un primo modello, addestrato sulle etichette generate dall'LLM, calcola per ciascun esempio del seed una misura di **incertezza** (*least confidence*, basata sulla probabilità della classe top-1); i campioni su cui il modello esita maggiormente, che sono spesso anche quelli su cui l'annotazione automatica è meno affidabile, vengono presentati per la revisione manuale tramite l'interfaccia Gradio, e il modello viene ri-addestrato sul training set corretto. Il razionale è informativo: i campioni *facili* (alta confidence) aggiungono poco segnale, mentre quelli *difficili* riducono la varianza del modello e permettono di intercettare gli errori sistematici dell'annotatore automatico. Empiricamente, questa combinazione ha permesso di raggiungere F1 macro 0,762 contenendo in modo drastico il costo dell'annotazione rispetto a un'etichettatura interamente manuale.

Le tre classi target sono: **Sì** (a favore della riforma), **No** (contrario), **Neutro** (commento non interpretabile in termini di stance, oppure off-topic).

3.2.4 Architettura: contesto padre: risposta

La principale innovazione architetturale è l'uso del **contesto** come input. Anziché classificare il commento isolato, il modello riceve la coppia (commento padre, risposta) tokenizzata nel formato standard di BERT per coppie di frasi:

```
[CLS] testo_padre [SEP] testo_risposta [SEP]
```

Questa scelta nasce dall'osservazione qualitativa che molti commenti YouTube sono **conversazionali**: il loro contenuto informativo dipende dal turno precedente. Esempio:

- Padre: *"Questa riforma è un disastro, separare i pubblici ministeri dai giudici significa subordinare la magistratura al potere politico."*
- Risposta: *"Esatto."*

Letta isolata, *"Esatto."* è classificata neutra. Letta in contesto, è chiaramente di stance **No**. Senza il padre, il segnale è completamente perso.

Per i commenti *root* (senza padre, postati direttamente sotto il video) è stato usato come "padre" il titolo del video, che funge da contesto minimo coerente.

3.2.5 Iperparametri di fine-tuning

Il fine-tuning è stato condotto su Kaggle (GPU T4/P100) con PyTorch e HuggingFace Transformers. I principali parametri:

- **Batch size**: 16 (limite di memoria GPU).
- **Learning rate**: nell'ordine di 2e-5, valore standard per fine-tuning BERT.
- **Epoche**: monitoraggio su validation set, con early stopping.
- **Optimizer**: AdamW.
- **Loss**: cross-entropy multiclasse, con eventuale class weighting per gestire lo sbilanciamento delle classi.
- **Train/val/test split**: stratificato per classe.

3.2.6 Metriche di valutazione

La metrica principale è **F1 macro**, scelta perché: - le classi (Sì, No, Neutro) non sono perfettamente bilanciate; - l'accuracy globale può essere ingannevole se una classe domina; - il *macro averaging* attribuisce uguale peso a ogni classe, riflettendo l'interesse a non sacrificare nessuna delle tre.

Sul test set il classificatore referendario ottiene **F1 macro = 0,762**, **Accuracy = 0,784**, in linea con lo stato dell'arte della letteratura su stance detection in italiano (cfr. SemEval-2016 Task 6 per inquadramento generale). Il classificatore analogo addestrato sul corpus del consenso al governo raggiunge invece una **F1 macro = 0,628**: la prestazione più bassa riflette un compito intrinsecamente più difficile (la stance verso il governo è multi-tematica, dall'economia all'immigrazione, mentre quella referendaria è mono-tematica) e un seed di annotazione più piccolo. Questa differenza va tenuta presente nell'interpretazione della serie governo usata nel Capitolo 6.

3.3 Inferenza su larga scala

Una volta congelato il modello, l'inferenza è stata estesa a tutti i 196.050 commenti del corpus referendum (e ai 136.624 del corpus governo). Ogni commento riceve una predizione `{Sì, No, Neutro}` con la rispettiva probabilità.

La pipeline di inferenza è eseguita in batch GPU per efficienza. L'output è un file CSV arricchito (`Dataset Tesi Aggiornato.csv`) in cui ogni riga contiene il commento originale più la colonna `stance` predetta.

A questo punto il dataset è pronto per le fasi di aggregazione.

3.4 Aggregazione e deduplicazione

3.4.1 Aggregato grezzo: misura del volume di dibattito

L'aggregazione più semplice consiste nel calcolare, per ciascun giorno *t*, la quota di commenti con stance Sì sul totale dei commenti non-neutri:

$$Sì_{\text{grezzo}}(t) = \frac{N_{\text{Sì}}(t)}{N_{\text{Sì}}(t) + N_{\text{No}}(t)}$$

Il problema di questa misura è interpretativo: essa risponde alla domanda *"quanti commenti pro-Sì sono stati postati?"*, non *"quante persone sono pro-Sì?"*. È, tecnicamente, una misura di **engagement**, non di **opinione**.

3.4.2 Il bias della minoranza rumorosa

Nei social media politici è documentato un fenomeno strutturale: una minoranza di utenti molto motivati posta commenti multipli, gonfiando in modo sproporzionato il proprio peso nei dati grezzi. Nel caso del referendum 2026, l'analisi della distribuzione dei commenti per autore mostra una **forte coda lunga**: pochi utenti contribuiscono con decine o centinaia di commenti, e questi utenti sono disproporzionatamente collocati sul fronte Sì.

L'aggregato grezzo dà infatti **48,9%** di Sì, contro un risultato reale (Cap. 4) del 46,26%: uno scarto di +2,68 pp che riflette l'over-rappresentazione dei "rumorosi" pro-riforma.

3.4.3 La regola "una testa, un voto"

La correzione adottata è semplice e teoricamente fondata: per la previsione (H1), per ogni autore unico si tiene **solo l'ultimo commento dell'intero periodo**. La logica: - *un autore = un'opinione*, indipendentemente da quante volte la esprima; - *l'ultimo commento* è il più rappresentativo della posizione finale dell'utente, perché costruito sul thread già letto e più vicino al voto.

Operativamente, in pseudo-codice:

```
df_dedup = (df
    .sort_values("date")
    .drop_duplicates(subset=["author_id"], keep="last")
    .reset_index(drop=True))
```

Il dataset deduplicato è quindi una *vera* misura di distribuzione delle opinioni: ogni unità di osservazione è una persona. Per le serie temporali giornaliere usate nei Capitoli 5 e 6 la deduplicazione è invece applicata **per autore e per giorno**, scelta necessaria a costruire una serie a frequenza giornaliera senza che gli utenti più attivi in una singola giornata la sovra-pesino.

3.4.4 Misure ottenute

L'effetto della correzione è diretto e leggibile:

Aggregazione	Sì %	Errore vs reale (46,26%)
Tutti i commenti (grezzo)	48,9%	+2,68 pp
Utenti unici (deduplicato)	**45,9%**	**0,40 pp**

La deduplicazione **riduce l'errore di oltre sei volte** rispetto al grezzo (da +2,68 a 0,40 pp). Questa è la corona del contributo metodologico della tesi e la risposta diretta alla critica di Gayo-Avello (2013) all'uso ingenuo dei social per la previsione elettorale.

3.4.5 Periodo di analisi e valore di nowcasting

Il calcolo dell'aggregato predittivo è condotto sull'**intero periodo della campagna referendaria, dal 30 ottobre 2025 al 23 marzo 2026**. La data di inizio coincide con l'approvazione definitiva della riforma al Senato, che apre formalmente la campagna; la data di fine è l'ultimo giorno utile per il voto. È la finestra entro cui il dibattito su YouTube è denso e continuo: al di fuori di essa i commenti sono sporadici e poco informativi.

Lavorare sull'intero periodo, anziché su una finestra ristretta, ha anche un valore di *nowcasting*. La legge italiana vieta la pubblicazione di sondaggi negli ultimi 15 giorni prima del voto, e proprio in quella fase di blackout demoscopico YouTube resta l'unica fonte di *opinion data* legalmente osservabile. La stima referendaria mantiene quindi un valore aggiunto rispetto ai sondaggi tradizionali anche quando questi, per legge, tacciono.

3.5 Analisi di serie temporali e Granger causality

3.5.1 Tre serie giornaliere

A partire dai dataset annotati si costruiscono tre serie temporali giornaliere:

1. **Referendum**: percentuale giornaliera di stance No su YouTube Referendum (su utenti unici giornalieri).
2. **Stampa**: indicatore di copertura pro-No vs pro-Sì nella rassegna stampa giornaliera.
3. **Governo**: stance/sentiment pro-governo nel corpus YouTube Meloni (sull'analogo dataset deduplicato).

Le tre serie hanno frequenza giornaliera e sono allineate sullo stesso calendario operativo.

3.5.2 Pre-test: stazionarietà (Augmented Dickey-Fuller)

Il test di Granger richiede serie **stazionarie** (media e varianza costanti nel tempo): senza questa condizione i p-value sono spurii. La verifica è fatta con il test di Augmented Dickey-Fuller (ADF):

- H_0: la serie ha una radice unitaria → non stazionaria.
- Se il p-value ADF è $< 0{,}05$, si rifiuta H_0 e si dichiara la serie stazionaria.

Esito sulle tre serie: **tutte e tre risultano già stazionarie** alle serie grezze giornaliere; non sono state necessarie trasformazioni (differenziazione, log-trasformazioni).

3.5.3 Scelta cruciale: serie grezze, non MA7

In sede di analisi esplorativa si è considerato di applicare una **media mobile a 7 giorni (MA7)** per "ammorbidire" le serie e renderle visivamente più leggibili. Per il *test di Granger*, però, questa trasformazione è da evitare: la media mobile **introduce autocorrelazione artificiale** nei residui, gonfiando i parametri AR e rendendo il test non valido.

La scelta finale è quindi: **serie grezze giornaliere** per i test di Granger e ADF; MA7 solo nei grafici esplorativi del Cap. 4 (con menzione esplicita di questa distinzione).

3.5.4 Specifica del test di Granger

Il test di Granger valuta se l'inclusione dei *lag* di una serie *X* nel modello AR di un'altra serie *Y* migliora significativamente la capacità predittiva, rispetto al modello che usa solo i lag di *Y*. In forma compatta:

$$Y_t = \alpha + \sum_{i=1}^{p} \beta_i Y_{t-i} + \sum_{j=1}^{p} \gamma_j X_{t-j} + \varepsilon_t$$

Il test F congiunto su $\gamma_1, \dots, \gamma_p = 0$ corrisponde all'ipotesi nulla "X non Granger-causa Y". Se p-value < 0,05, X **Granger-causa** Y al lag *p*.

Sono stati testati lag 3 e lag 7 (3 giorni e 1 settimana, intervalli sensati per il ciclo di reazione mediatica e online). Sono state testate tutte e **6 le coppie ordinate** (ogni direzione separatamente):

- Stampa → Referendum, Referendum → Stampa
- Governo → Referendum, Referendum → Governo
- Stampa → Governo, Governo → Stampa

3.5.5 Identificazione dei turning points

Parallelamente al test di Granger, si identificano i **turning points** della serie referendum: i giorni in cui la stance cambia direzione in modo statisticamente rilevante. La procedura combina: - **trasformata arcoseno-radice** della quota giornaliera di Sì, **pesata per il volume** di commenti del giorno; - **segmentazione a media costante** (optimal partitioning con penalità BIC), che individua i punti di rottura fra segmenti a livello diverso; - **annotazione manuale** degli eventi politici e mediatici prossimi al turning point (tramite cross-referencing con le date di pubblicazione dei video più virali, dichiarazioni pubbliche, eventi parlamentari).

Ai fini di H2 (almeno il 70% dei turning points associabile a un evento documentabile), il criterio di documentabilità è esplicitato: video con almeno X view, dichiarazione pubblica con copertura stampa, evento parlamentare. La validazione qualitativa è discussa nel Cap. 4.

3.6 Stack tecnologico e riproducibilità

L'intera pipeline è stata sviluppata in Python su **Kaggle** (sfruttando le GPU T4/P100 gratuite disponibili nei notebook). Lo stack tecnologico:

- **Framework ML**: PyTorch + HuggingFace Transformers (per BERT fine-tuning e inferenza).
- **Data manipulation**: pandas, numpy.
- **Visualizzazione**: plotly (grafici interattivi multilayer).
- **NLP**: tokenizers (HuggingFace).
- **Econometria**: statsmodels (ADF, Granger causality test).
- **Active learning UI**: Gradio.

3.7 Sintesi del capitolo

La metodologia si organizza come una pipeline lineare con tre snodi metodologici qualificanti. Il **primo** è l'uso del contesto padre-risposta nell'input al classificatore, che permette di catturare la natura conversazionale dei commenti YouTube. Il **secondo** è la deduplicazione "una testa, un voto", che corregge il bias della minoranza rumorosa e trasforma il dato da misura di engagement a misura di opinione. Il **terzo** è il rigore econometrico (ADF, serie grezze, lag espliciti) nei test di Granger, che rende le conclusioni causali difendibili dal punto di vista metodologico.

I risultati ottenuti applicando questa pipeline sono presentati nel Capitolo 4.

Implementazione del §3.1: Raccolta dati

> Le celle che seguono implementano la pipeline di **raccolta dati** descritta nel §3.1 (scraping YouTube quota-aware per i corpus referendum e governo, ingestione del corpus della stampa nazionale, ricostruzione del contesto padre-figlio).

Notebook 1: Raccolta Dati

Tesi Triennale: Referendum Costituzionale Giustizia 2026, Separazione delle Carriere

Questo notebook contiene **tutta la pipeline di raccolta dati** per la tesi.

Sezione	Cosa fa	Output
§1 Schema annotazione	Definisce le etichette STANCE (POS/NEG/NEU)	,
§2 YouTube Referendum	Scraping incrementale commenti su video pertinenti	`referendum_giustizia_full_debate.csv`
§3 YouTube Governo	Scraping con tracking quota API (10k/giorno)	`governo_meloni_consenso.csv`
§4 Stampa	Caricamento dataset articoli (raccolto esternamente)	`dataset_articoli_referendum.csv`
§5 Contesto padre-figlio	Aggiunge `Testo_Padre` alle risposte per il modello BERT	`referendum_con_padre.csv`

API: YouTube Data API v3, quota giornaliera 10.000 unità (reset 09:00 ora italiana) **Modalità**: incrementale, ad ogni esecuzione scarica solo i commenti **nuovi** (delta) **Deduplicazione**: chiave (`video_id, autore, data`) per evitare doppioni tra esecuzioni

§1: Schema di Annotazione: STANCE

La variabile **STANCE** classifica la posizione del commentatore rispetto al referendum sulla separazione delle carriere in magistratura.

Etichette aggregate (usate per il training BERT)

Codice	Etichetta	Descrizione
POS	Favorevole	Supporto esplicito o implicito alla riforma
NEG	Contrario	Opposizione (timore indebolimento azione penale, influenza politica)
NEU	Neutro/Disilluso	Nessuna posizione chiara, sfiducia istituzionale generica, off-topic

§2: Raccolta YouTube: Referendum Giustizia

Pipeline a 3 step: 1. **Stato persistente**, carica il CSV esistente per abilitare modalità delta (scarica solo commenti dopo l'ultimo salvato). 2. **Raccolta video**, query mirate sul referendum, filtro per pertinenza (lessico tematico) e rumore (noise keywords). 3. **Estrazione commenti**, per ogni video chiama `commentThreads.list`; per video già noti usa `publishedAfter` per scaricare solo il delta.

Query coprono il referendum sulla separazione delle carriere, includendo anche varianti per Shorts (formato breve). **Filtro pertinenza**: il titolo o la descrizione devono contenere

almeno una keyword del lessico (referendum, giustizia, magistrat, csm, ...) e nessuna noise (fiction, calcio, makeup, ...).

§2.1: Setup, configurazione, filtri

```
import pandas as pd
from googleapiclient.discovery import build
from kaggle_secrets import UserSecretsClient
import time, re, os
from datetime import datetime, timezone

# ── CONFIGURAZIONE ──────────────────────────────────────────────────
# La API key è salvata come secret di Kaggle per non esporla nel notebook
user_secrets = UserSecretsClient()
API_KEY  = user_secrets.get_secret("YOUTUBE_API")
youtube  = build("youtube", "v3", developerKey=API_KEY)

# Path: input read-only (Kaggle dataset versione precedente) e working (dove salviamo l'aggiornato)
INPUT_FILE  = "/kaggle/input/datasets/francescochiarolanza/commenti-youtube-referendum-
giustizia/Dataset Tesi Aggiornato.csv"
OUTPUT_FILE = "/kaggle/working/Dataset Tesi Aggiornato.csv"

# Query: combinazione di parole chiave specifiche del referendum + varianti shorts
# (la versione "shorts" intercetta i video brevi che hanno linguaggio diverso)
QUERIES = [
    "referendum giustizia 2026",
    "separazione carriere magistrati",
    "riforma giustizia",
    "commenti referendum giustizia",
    "referendum giustizia shorts",
    "separazione carriere shorts",
    "riforma giustizia 2026 shorts",
]

# Lessico di pertinenza: il video è incluso se titolo/descrizione contengono almeno una di queste
RELEVANCE_KEYWORDS = [
    "referendum", "giustizia", "magistrat", "separazione carriere",
    "riforma", "csm", "toghe", "procura", "giudic", "pm ",
    "corte", "costituzion", "voto", "quesito",
]

# Noise keywords: il video è escluso se contiene una di queste (anche se ha pertinenza)
# Serve a filtrare contenuti omonimi o totalmente off-topic che usano gli stessi termini
NOISE_KEYWORDS = [
    "fiction", "film", "serie tv", "musica", "canzone", "ricetta",
    "tutorial trucco", "makeup", "calcio", "sport",
]

# Limiti per esecuzione (per non sforare quota API in una sola run)
MAX_PAGES_PER_QUERY = 10   # pagine search.list per ogni query (50 video/pagina)
MAX_COMMENT_PAGES   = 20   # pagine commentThreads.list per ogni video (100 commenti/pagina)


def is_relevant(title, description=""):
    """True se il video parla di referendum giustizia. Controlla keyword di pertinenza E assenza di
noise."""
    text = (title + " " + description).lower()
    if any(noise in text for noise in NOISE_KEYWORDS):
        return False
    return any(kw in text for kw in RELEVANCE_KEYWORDS)


def is_short(duration_iso):
```

```
    """True se la durata ISO 8601 (es. 'PT45S' o 'PT1M30S') è ≤ 60 secondi → YouTube Short."""
    m = re.match(r"PT(?:(\d+)H)?(?:(\d+)M)?(?:(\d+)S)?", duration_iso)
    if not m:
        return False
    h, mn, s = int(m.group(1) or 0), int(m.group(2) or 0), int(m.group(3) or 0)
    return h * 3600 + mn * 60 + s <= 60
```

§2.2: Stato persistente e modalità delta

`load_existing_state` carica il CSV esistente per: - ricavare l'elenco di **video già visti** (`known_ids`) - la **data dell'ultimo commento salvato** per ogni video (`last_dates`) → usata in `commentThreads.list(publishedAfter=...)` per scaricare solo il delta - una **chiave di deduplicazione** (`video_id, autore, data`) per evitare doppioni residui

Priorità: `/kaggle/working` (versione aggiornata di questa run) > `/kaggle/input` (read-only).

```
def load_existing_state(input_path, output_path):
    """Carica stato del CSV esistente per la modalità delta."""
    filepath = output_path if os.path.exists(output_path) else (
               input_path  if os.path.exists(input_path)  else None)
    if filepath is None:
        print("Partenza da zero (nessun CSV esistente).")
        return pd.DataFrame(), set(), {}, set()

    print(f"Carico stato esistente da: {filepath}")
    df = pd.read_csv(filepath, encoding="utf-8-sig", dtype=str, sep=";",
                     on_bad_lines="skip", engine="python")
    df["Data_Commento"] = pd.to_datetime(df["Data_Commento"], utc=True, errors="coerce")

    known_ids  = set(df["Video_ID"].dropna().unique())
    # Per ogni video, l'ultima data salvata → la API filtrerà server-side con publishedAfter
    last_dates = (df.groupby("Video_ID")["Data_Commento"].max().dropna()
                    .dt.strftime("%Y-%m-%dT%H:%M:%SZ").to_dict())
    # Set di chiavi (video, autore, data) per scartare duplicati residui
    known_keys = set(zip(df["Video_ID"].fillna(""),
                         df["Autore"].fillna(""),
                         df["Data_Commento"].astype(str)))

    print(f"  → {len(df):,} righe, {len(known_ids)} video già presenti")
    return df, known_ids, last_dates, known_keys
```

§2.3: Raccolta video ID (search.list)

Per ogni query: itera le pagine, applica `is_relevant`, accumula i video unici nel `video_map`.

Il costo di `search.list` è **100 unità di quota** per chiamata: con 7 query × 10 pagine = 7000 unità potenziali. Lasciamo margine per `commentThreads.list` (1 unità/chiamata).

```
def collect_all_video_ids(queries):
    """Per ogni query: itera le pagine search.list e raccoglie i video pertinenti."""
    video_map, seen = {}, set()
    for q in queries:
        print(f"Query: '{q}'")
        page_token, page_count = None, 0
        while page_count < MAX_PAGES_PER_QUERY:
            try:
                resp = youtube.search().list(
                    q=q, part="id,snippet", maxResults=50,
```

```
                    type="video", order="relevance", pageToken=page_token
                ).execute()
            except Exception as e:
                print(f"  Errore: {e}")
                break

            for item in resp.get("items", []):
                v_id  = item["id"]["videoId"]
                if v_id in seen:
                    continue
                title = item["snippet"]["title"]
                desc  = item["snippet"].get("description", "")
                if not is_relevant(title, desc):
                    continue
                seen.add(v_id)
                video_map[v_id] = {
                    "Titolo_Video": title,
                    "Canale":       item["snippet"]["channelTitle"],
                    "Link":         f"https://www.youtube.com/watch?v={v_id}",
                    "Data_Video":   item["snippet"]["publishedAt"],
                    "Is_Short":     None,   # arricchito dopo con videos.list
                }

            page_count += 1
            page_token  = resp.get("nextPageToken")
            print(f"  pag. {page_count}: {len(video_map)} video totali pertinenti")
            if not page_token:
                break
            time.sleep(0.3)   # piccola pausa per evitare rate limit
    return video_map
```

§2.4: Arricchimento con durata (videos.list → Is_Short)

`search.list` non restituisce la durata. Serve una chiamata separata a `videos.list(part='contentDetails')` per ottenere `duration` (formato ISO 8601, es. `PT2M15S`).

Batch da 50 video/chiamata → minimizza il numero di richieste.

```
def enrich_with_duration(video_map):
    """Aggiunge Is_Short e Durata_ISO al video_map."""
    ids = list(video_map.keys())
    for i in range(0, len(ids), 50):
        batch = ids[i:i+50]
        try:
            resp = youtube.videos().list(part="contentDetails", id=",".join(batch)).execute()
            for item in resp.get("items", []):
                dur = item["contentDetails"]["duration"]
                video_map[item["id"]]["Is_Short"]   = is_short(dur)
                video_map[item["id"]]["Durata_ISO"] = dur
        except Exception as e:
            print(f"  Errore arricchimento durata: {e}")
        time.sleep(0.2)
    shorts = sum(1 for v in video_map.values() if v.get("Is_Short"))
    print(f"  Shorts identificati: {shorts}/{len(ids)}")
    return video_map
```

§2.5: Estrazione commenti (modalità delta)

Per ogni video: - **NUOVO** (non in `known_ids`): scarica tutti i commenti. - **DELTA** (già noto): passa `publishedAfter=last_date` → la API restituisce solo i commenti dopo

l'ultimo salvato.

Estrae sia commenti **principali** che **risposte** (queste ultime con `Padre_ID` per l'associazione successiva). Stop anticipato: se in una pagina non ci sono nuovi commenti per un video noto, smette (riduce chiamate inutili).

```
def extract_comments(video_map, known_ids, last_dates, known_keys):
    """Estrae commenti per tutti i video. Modalità DELTA per video già noti."""
    new_rows = []
    for idx, (v_id, meta) in enumerate(video_map.items(), 1):
        is_known  = v_id in known_ids
        last_date = last_dates.get(v_id)
        label     = "Short" if meta.get("Is_Short") else "Video"
        mode      = "DELTA" if is_known else "NUOVO"
        print(f"[{idx}/{len(video_map)}] {label} [{mode}] → {meta['Titolo_Video'][:50]}...")

        try:
            page_token, page_count, stop_early = None, 0, False
            while page_count < MAX_COMMENT_PAGES and not stop_early:
                kwargs = dict(
                    part="snippet,replies", videoId=v_id, maxResults=100,
                    textFormat="plainText", order="time", pageToken=page_token
                )
                # Se il video è noto, filtra lato server con publishedAfter (delta)
                if is_known and last_date:
                    kwargs["publishedAfter"] = last_date

                resp = youtube.commentThreads().list(**kwargs).execute()
                page_new = 0

                for item in resp["items"]:
                    # Commento principale (top-level)
                    top  = item["snippet"]["topLevelComment"]["snippet"]
                    ckey = (v_id, top["authorDisplayName"],
                            str(pd.to_datetime(top["publishedAt"], utc=True)))
                    if ckey not in known_keys:
                        known_keys.add(ckey)
                        new_rows.append({
                            **meta, "Video_ID": v_id,
                            "Tipo_Commento": "Principale", "Padre_ID": None,
                            "Autore":        top["authorDisplayName"],
                            "Testo":         top["textDisplay"],
                            "Likes":         top["likeCount"],
                            "Data_Commento": top["publishedAt"],
                        })
                        page_new += 1

                    # Risposte al commento principale (se presenti)
                    if "replies" in item:
                        for reply in item["replies"]["comments"]:
                            r    = reply["snippet"]
                            rkey = (v_id, r["authorDisplayName"],
                                    str(pd.to_datetime(r["publishedAt"], utc=True)))
                            if rkey not in known_keys:
                                known_keys.add(rkey)
                                new_rows.append({
                                    **meta, "Video_ID": v_id,
                                    "Tipo_Commento": "Risposta",
                                    "Padre_ID":      item["id"],
                                    "Autore":        r["authorDisplayName"],
                                    "Testo":         r["textDisplay"],
                                    "Likes":         r["likeCount"],
                                    "Data_Commento": r["publishedAt"],
                                })
```

```
                                page_new += 1

                # Stop anticipato: video noto e nessun nuovo commento → non serve continuare
                if is_known and page_new == 0:
                    stop_early = True

                page_count += 1
                page_token  = resp.get("nextPageToken")
                if not page_token:
                    break
                time.sleep(0.2)

        except Exception as e:
            print(f"  Commenti non disponibili: {e}")

    print(f"\nNuovi commenti raccolti: {len(new_rows):,}")
    return pd.DataFrame(new_rows)
```

§2.6: Esecuzione pipeline referendum

Sequenza: 1. `load_existing_state` → stato delta 2. `collect_all_video_ids(QUERIES)` → mappa video pertinenti 3. `enrich_with_duration(video_map)` → aggiunge Is_Short 4. `extract_comments(...)` → scarica commenti (delta o full) 5. **Merge + dedup finale** → concat con esistente, drop_duplicates su (Video_ID, Autore, Data, Testo) 6. **Save** → CSV in `/kaggle/working/` (con sep=`;` per compatibilità con la pipeline successiva)

```
df_existing, known_ids, last_dates, known_keys = load_existing_state(INPUT_FILE, OUTPUT_FILE)
video_map = collect_all_video_ids(QUERIES)
video_map = enrich_with_duration(video_map)

# Stats: quanti video sono nuovi vs già noti
n_new   = sum(1 for v in video_map if v not in known_ids)
n_delta = sum(1 for v in video_map if v     in known_ids)
print(f"\nVideo trovati: {len(video_map)} totali ({n_new} nuovi, {n_delta} delta)")

df_new = extract_comments(video_map, known_ids, last_dates, known_keys)

if df_new.empty:
    print("\nNessun nuovo commento. Dataset già aggiornato.")
    if not os.path.exists(OUTPUT_FILE) and not df_existing.empty:
        df_existing.to_csv(OUTPUT_FILE, index=False, encoding="utf-8-sig")
else:
    df_combined = pd.concat([df_existing, df_new], ignore_index=True)
    # Dedup finale di sicurezza (catchall: rimuove eventuali doppioni residui)
    before = len(df_combined)
    df_combined.drop_duplicates(
        subset=["Video_ID", "Autore", "Data_Commento", "Testo"], inplace=True
    )
    if before != len(df_combined):
        print(f"  Rimossi {before - len(df_combined)} duplicati residui.")
    df_combined.to_csv(OUTPUT_FILE, index=False, encoding="utf-8-sig")
    print(f"\nDataset aggiornato: {len(df_combined):,} righe ({len(df_new):,} nuove).")
    print(f"File salvato: {OUTPUT_FILE}")
```

§3: Raccolta YouTube: Governo Meloni

Pipeline analoga al referendum, con due differenze:

1. **Quota tracking esplicito**, la quota giornaliera YouTube è 10.000 unità. Qui teniamo conto del consumo (`SEARCH_COST=100`, `COMMENT_COST=1`) e fermiamo le ricerche quando il budget search è esaurito (limite 5000 unità su search per lasciare margine ai commenti).
2. **Filtro temporale `publishedAfter`**, solo video pubblicati dopo il `2025-09-01` (insediamento del governo nel periodo monitorato).

Query più ampie del caso referendum: includono nomi di programmi TV (DiMartedì, Otto e Mezzo, Piazzapulita, Porta a Porta…) e nomi di esponenti (Salvini, Tajani, Schlein, Conte, Nordio…) per catturare il dibattito politico.

Lessico di pertinenza centrato su: governo/Meloni/maggioranza/manovra/sondaggi e nomi di partiti.

```
import pandas as pd
from googleapiclient.discovery import build
from kaggle_secrets import UserSecretsClient
import time, re, os, sys

# ── CONFIGURAZIONE ──────────────────────────────────────────────────
user_secrets = UserSecretsClient()
API_KEY = user_secrets.get_secret("YOUTUBE_API")
youtube = build("youtube", "v3", developerKey=API_KEY)

OUTPUT_FILE     = "governo_meloni_consenso.csv"
PUBLISHED_AFTER = "2025-09-01T00:00:00Z"   # solo video post-insediamento

# ── GESTIONE QUOTA API ──────────────────────────────────────────────
# La YouTube API ha 10.000 unità/giorno. search costa 100, commentThreads costa 1.
# Per fare 50 query × 3 pagine = 15.000 unità → rischio quota. Limitiamo il
# budget search a 5000 (50 query) per lasciare il resto ai commenti.
QUOTA_LIMIT         = 10000
SEARCH_COST         = 100
COMMENT_COST        = 1
VIDEO_COST          = 1
QUOTA_SEARCH_BUDGET = 5000
quota_used          = 0
quota_exhausted     = False


def track_quota(cost, op=""):
    """Aggiorna il contatore globale e logga ogni 500 unità consumate."""
    global quota_used
    quota_used += cost
    if quota_used % 500 < cost:
        print(f"  [Quota: {quota_used}/{QUOTA_LIMIT}] ({op})")


def quota_remaining():
    return QUOTA_LIMIT - quota_used


def check_quota_error(e):
    """Riconosce l'errore di quota esaurita dalla API."""
    global quota_exhausted
    if "quotaExceeded" in str(e):
        quota_exhausted = True
        return True
    return False
```

```
# Verifica preliminare: se la quota è già esaurita all'inizio, abortiamo
print("Verifica quota API...")
try:
    youtube.videos().list(part="id", id="dQw4w9WgXcQ").execute()
    print("  Quota disponibile.")
except Exception as e:
    if "quotaExceeded" in str(e):
        print("QUOTA ESAURITA. Riprova domani alle 09:00 (ora italiana).")
        sys.exit()
    else:
        print(f"  Errore non-quota: {e}")


# ── QUERY ──────────────────────────────────────────────────────────────
# Mix di:
#  - Query istituzionali (governo, fiducia, manovra, sondaggi)
#  - Programmi TV politici (DiMartedì, Otto e Mezzo, Piazzapulita, Porta a Porta)
#  - Esponenti politici (Salvini, Tajani, Schlein, Conte, Nordio, Piantedosi)
#  - Varianti shorts
QUERIES = [
    "governo Meloni", "Meloni governo Italia", "fiducia governo Meloni",
    "sondaggi governo Meloni", "crisi governo Italia", "politica italiana",
    "DiMartedi Meloni", "Otto e mezzo governo", "Piazzapulita Meloni",
    "Porta a Porta governo", "Floris governo Meloni", "Gruber Meloni",
    "manovra governo Meloni", "legge di bilancio Italia",
    "autonomia differenziata", "premierato riforma",
    "Meloni economia lavoro", "Meloni immigrazione",
    "Salvini vicepremier", "Tajani forza italia",
    "Nordio giustizia", "Piantedosi sicurezza",
    "Meloni shorts", "governo Italia shorts",
]
MAX_PAGES_PER_QUERY = 3
MAX_COMMENT_PAGES   = 15
MANUAL_VIDEO_IDS    = []   # opzionale: aggiungi qui ID di video specifici che la search potrebbe
perdere

RELEVANCE_KEYWORDS = [
    "governo", "meloni", "premier", "presidente del consiglio",
    "maggioranza", "opposizione", "fiducia", "manovra", "cdm",
    "legge di bilancio", "decreto", "fratelli d\'italia", "fdi", "lega", "forza italia",
    "pd", "m5s", "centrodestra", "centrosinistra",
    "salvini", "tajani", "schlein", "conte", "nordio", "piantedosi", "crosetto",
    "politica", "sondaggi", "consenso", "elezioni", "riforma", "economia",
]
NOISE_KEYWORDS = [
    "fiction", "film completo", "serie tv", "musica", "ricetta",
    "tutorial trucco", "makeup", "calcio serie a", "highlights gol",
    "trailer", "gaming", "fortnite", "minecraft", "asmr",
]


def is_relevant(title, description=""):
    text = (title + " " + description).lower()
    return (not any(n in text for n in NOISE_KEYWORDS) and
            any(kw in text for kw in RELEVANCE_KEYWORDS))


def is_short(duration_iso):
    m = re.match(r"PT(?:(\d+)H)?(?:(\d+)M)?(?:(\d+)S)?", duration_iso)
    if not m:
        return False
    h, mn, s = int(m.group(1) or 0), int(m.group(2) or 0), int(m.group(3) or 0)
    return h * 3600 + mn * 60 + s <= 60
```

§3.1: Funzioni di raccolta (con quota tracking)

```
def load_existing_state(filepath):
    """Carica stato del CSV (versione semplificata per il dataset governo)."""
    if not os.path.exists(filepath):
        print("Partenza da zero.")
        return pd.DataFrame(), set(), {}, set()

    df = pd.read_csv(filepath, encoding="utf-8-sig", dtype=str)
    df["Data_Commento"] = pd.to_datetime(df["Data_Commento"], utc=True, errors="coerce")
    known_ids  = set(df["Video_ID"].dropna().unique())
    last_dates = (df.groupby("Video_ID")["Data_Commento"].max().dropna()
                    .dt.strftime("%Y-%m-%dT%H:%M:%SZ").to_dict())
    known_keys = set(zip(df["Video_ID"].fillna(""),
                         df["Autore"].fillna(""),
                         df["Data_Commento"].astype(str)))
    print(f"Esistente: {len(df):,} righe, {len(known_ids)} video")
    return df, known_ids, last_dates, known_keys


def collect_video_ids(queries, known_ids):
    """Raccoglie video con quota tracking: si ferma se il budget search è esaurito."""
    global quota_exhausted
    video_map, seen = {}, set(known_ids)
    sq_used = 0
    for q in queries:
        if quota_exhausted or sq_used + SEARCH_COST > QUOTA_SEARCH_BUDGET:
            print("  Budget search esaurito.")
            break
        print(f"'{q}'")
        page_token, page_count = None, 0
        while page_count < MAX_PAGES_PER_QUERY:
            if quota_exhausted:
                break
            try:
                resp = youtube.search().list(
                    q=q, part="id,snippet", maxResults=50, type="video",
                    order="relevance", publishedAfter=PUBLISHED_AFTER,
                    pageToken=page_token
                ).execute()
                sq_used += SEARCH_COST
                track_quota(SEARCH_COST, f"search '{q[:25]}'")
            except Exception as e:
                if check_quota_error(e):
                    print("  QUOTA ESAURITA.")
                else:
                    print(f"  {e}")
                break

            for item in resp.get("items", []):
                v_id = item["id"]["videoId"]
                if v_id in seen:
                    continue
                title = item["snippet"]["title"]
                if not is_relevant(title, item["snippet"].get("description", "")):
                    continue
                seen.add(v_id)
                video_map[v_id] = {
                    "Titolo_Video": title,
                    "Canale":       item["snippet"]["channelTitle"],
                    "Link":         f"https://www.youtube.com/watch?v={v_id}",
                    "Data_Video":   item["snippet"]["publishedAt"],
                    "Is_Short":     None,
                    "Topic":        "GOVERNO",
                }
```

```
            page_count += 1
            page_token = resp.get("nextPageToken")
            print(f"  p{page_count}: +{len(video_map)} totali")
            if not page_token:
                break
            time.sleep(0.3)

    return video_map


def enrich_with_duration(video_map):
    """Aggiunge Is_Short. Si ferma se la quota si esaurisce a metà."""
    global quota_exhausted
    ids = [v for v, m in video_map.items() if m.get("Is_Short") is None]
    if not ids or quota_exhausted:
        return video_map

    print(f"Durata per {len(ids)} video...")
    for i in range(0, len(ids), 50):
        batch = ids[i:i+50]
        try:
            resp = youtube.videos().list(part="contentDetails", id=",".join(batch)).execute()
            track_quota(VIDEO_COST, "duration")
            for item in resp.get("items", []):
                dur = item["contentDetails"]["duration"]
                video_map[item["id"]]["Is_Short"]   = is_short(dur)
                video_map[item["id"]]["Durata_ISO"] = dur
        except Exception as e:
            if check_quota_error(e):
                break
            print(f"  {e}")
        time.sleep(0.2)
    return video_map


def extract_comments(video_map, known_ids, last_dates, known_keys):
    """Estrazione commenti con monitoraggio quota; si ferma se restano < 50 unità."""
    global quota_exhausted
    new_rows = []
    for idx, (v_id, meta) in enumerate(video_map.items(), 1):
        if quota_exhausted or quota_remaining() < 50:
            print(f"  Stop: quota insufficiente. Raccolti {len(new_rows):,}")
            break
        is_known  = v_id in known_ids
        last_date = last_dates.get(v_id)
        print(f"[{idx}/{len(video_map)}] {'D' if is_known else '+'} {meta['Titolo_Video']
[:45]}...")
        try:
            page_token, page_count = None, 0
            while page_count < MAX_COMMENT_PAGES:
                if quota_exhausted:
                    break
                kwargs = dict(part="snippet,replies", videoId=v_id, maxResults=100,
                              textFormat="plainText", order="time", pageToken=page_token)
                if is_known and last_date:
                    kwargs["publishedAfter"] = last_date

                resp = youtube.commentThreads().list(**kwargs).execute()
                track_quota(COMMENT_COST, "comments")
                page_new = 0

                for item in resp["items"]:
                    top  = item["snippet"]["topLevelComment"]["snippet"]
                    ckey = (v_id, top["authorDisplayName"],
                            str(pd.to_datetime(top["publishedAt"], utc=True)))
```

```
                    if ckey not in known_keys:
                        known_keys.add(ckey)
                        new_rows.append({
                            **meta, "Video_ID": v_id,
                            "Tipo_Commento": "Principale", "Padre_ID": None,
                            "Autore":        top["authorDisplayName"],
                            "Testo":         top["textDisplay"],
                            "Likes":         top["likeCount"],
                            "Data_Commento": top["publishedAt"],
                        })
                        page_new += 1

                    if "replies" in item:
                        for reply in item["replies"]["comments"]:
                            r    = reply["snippet"]
                            rkey = (v_id, r["authorDisplayName"],
                                    str(pd.to_datetime(r["publishedAt"], utc=True)))
                            if rkey not in known_keys:
                                known_keys.add(rkey)
                                new_rows.append({
                                    **meta, "Video_ID": v_id,
                                    "Tipo_Commento": "Risposta",
                                    "Padre_ID":      item["id"],
                                    "Autore":        r["authorDisplayName"],
                                    "Testo":         r["textDisplay"],
                                    "Likes":         r["likeCount"],
                                    "Data_Commento": r["publishedAt"],
                                })
                                page_new += 1

                if is_known and page_new == 0:
                    break   # nessuna novità per video noto → stop anticipato
                page_count += 1
                page_token  = resp.get("nextPageToken")
                if not page_token:
                    break
                time.sleep(0.2)

            except Exception as e:
                print(f"  {e}")

    print(f"\nNuovi commenti: {len(new_rows):,}")
    return pd.DataFrame(new_rows)
```

§3.2: Esecuzione pipeline governo

```
df_existing, known_ids, last_dates, known_keys = load_existing_state(OUTPUT_FILE)
video_map = collect_video_ids(QUERIES, known_ids)
video_map = enrich_with_duration(video_map)

df_new = extract_comments(video_map, known_ids, last_dates, known_keys)

if df_new.empty:
    print("\nNessun nuovo commento.")
else:
    df_combined = pd.concat([df_existing, df_new], ignore_index=True)
    before = len(df_combined)
    df_combined.drop_duplicates(
        subset=["Video_ID", "Autore", "Data_Commento", "Testo"], inplace=True
    )
    if before != len(df_combined):
        print(f"  Rimossi {before - len(df_combined)} duplicati.")
```

```
    df_combined.to_csv(OUTPUT_FILE, index=False, encoding="utf-8-sig")
    print(f"\nSalvato {OUTPUT_FILE}: {len(df_combined):,} righe ({len(df_new):,} nuove)")
```

§4: Dataset Stampa

Il dataset stampa (`dataset_articoli_referendum.csv`) è stato raccolto **fuori da Kaggle**: scraping mirato sui siti dei principali quotidiani italiani per il periodo dicembre 2025, marzo 2026, focus su articoli che menzionano il referendum sulla separazione delle carriere.

Schema: | Colonna | Descrizione | |---------|-------------| | `Data_Articolo` | Data di pubblicazione | | `Testata` | Nome del giornale (Il Fatto Quotidiano, Il Giornale, Libero, ...) | | `Titolo` | Titolo dell'articolo | | `STANCE` | Etichetta annotata: POS (pro-riforma) / NEG (contro-riforma) / NEU (informativo) / DUBBIO (posizione incerta) |

Volume: 5.152 articoli da 85 testate; di questi, 3.424 con polarità netta (POS/NEG), usati per l'indicatore di coverage pro-Sì.

Annotazione: stessa scala STANCE del YouTube (POS/NEG/NEU), così che i tre dataset siano confrontabili nelle analisi di Granger e turning points.

La cella sotto carica e ispeziona il dataset per verificare integrità.

```
import pandas as pd

PATH_STAMPA = "/kaggle/input/datasets/francescochiarolanza/titoli-giornali-referendum-
giustizia/dataset_articoli_referendum.csv"

# Auto-detect del separatore (alcune versioni usano \t o ;)
df_st = pd.read_csv(PATH_STAMPA, dtype=str, encoding="utf-8-sig",
                    engine="python", on_bad_lines="skip")
if len(df_st.columns) <= 3:
    df_st = pd.read_csv(PATH_STAMPA, sep=";", dtype=str, encoding="utf-8-sig",
                        engine="python", on_bad_lines="skip")

df_st = df_st.drop(columns=[c for c in df_st.columns if "Unnamed" in c], errors="ignore")
print(f"Articoli totali: {len(df_st):,}")
print(f"Colonne: {df_st.columns.tolist()}")

# Distribuzione stance
if "STANCE" in df_st.columns:
    df_st["STANCE"] = df_st["STANCE"].astype(str).str.strip().str.upper()
    print("\nDistribuzione STANCE:")
    print(df_st["STANCE"].value_counts())

# Distribuzione testate (top 10)
testata_col = next((c for c in df_st.columns if "testata" in c.lower()), None)
if testata_col:
    print(f"\nTop 10 testate per volume:")
    print(df_st[testata_col].value_counts().head(10))
```

§5: Associazione Padre-Figlio

L'API YouTube restituisce commenti principali e risposte come record separati, con un `Padre_ID` che lega la risposta al commento principale. Per il modello BERT è importante

fornire il **testo del commento padre come contesto** della risposta: i thread di risposta usano spesso pronomi impliciti (`anche loro`, `come dici tu`, `ma se è così`) che senza il padre sono ambigui.

Approccio: scorri i commenti **in ordine** per ogni video. Tieni traccia dell'ultimo principale visto. Quando incontri una risposta, assegnale il testo del principale corrente. È un'approssimazione **posizionale** (presuppone che le risposte seguano subito il padre nell'ordine restituito dall'API), copre l'80%+ dei thread.

Output: nuovo CSV `referendum_con_padre.csv` con colonna aggiuntiva `Testo_Padre`.

```
import pandas as pd
import numpy as np

# Carica il dataset YouTube referendum (output di §2)
df = pd.read_csv(
    "/kaggle/input/datasets/francescochiarolanza/commenti-youtube-referendum-
giustizia/referendum_giustizia_full_debate.csv",
    encoding="utf-8-sig", dtype=str, low_memory=False
)
df = df.drop(columns=[c for c in df.columns if "Unnamed" in c], errors="ignore")
print(f"Colonne: {df.columns.tolist()}")
print(f"Righe:   {len(df):,}")

df["Testo_Padre"] = np.nan
matched   = 0
unmatched = 0

# Itera per video: dentro ogni gruppo, scorri in ordine e abbina padre-risposta
for video_id, group in df.groupby("Video_ID"):
    idx = group.index.tolist()
    current_padre = None   # testo dell'ultimo commento principale visto

    for i in idx:
        row = df.loc[i]
        if row["Tipo_Commento"] == "Principale":
            current_padre = row["Testo"]
        elif row["Tipo_Commento"] == "Risposta":
            # Abbinamento posizionale: ultimo principale visto = padre presunto
            if current_padre is not None:
                df.at[i, "Testo_Padre"] = current_padre
                matched += 1
            else:
                unmatched += 1   # risposta senza padre identificabile (rara)

print(f"\nRisposte con padre trovato: {matched:,}")
print(f"Risposte senza padre:        {unmatched:,}")
print(f"Commenti principali:         {(df['Tipo_Commento']=='Principale').sum():,}")
print(f"Totale righe:                {len(df):,}")

# Verifica qualitativa su un campione
print("\n--- ESEMPIO: risposta + padre abbinato ---")
for _, row in df[df["Testo_Padre"].notna()].head(3).iterrows():
    print(f"  PADRE:    {str(row['Testo_Padre'])[:100]}")
    print(f"  RISPOSTA: {str(row['Testo'])[:100]}")
    print()

# Salva con sep=";" per coerenza con la pipeline successiva (training BERT)
df.to_csv("/kaggle/working/referendum_con_padre.csv",
          index=False, encoding="utf-8-sig", sep=";")
print("Salvato: /kaggle/working/referendum_con_padre.csv")
```

Output di questo notebook

Tre CSV pronti per il notebook 2 (training):

File	Contenuto	Usato in
`Dataset Tesi Aggiornato.csv`	YouTube referendum (con `Is_Short`, `Padre_ID`)	§5 di questo notebook
`governo_meloni_consenso.csv`	YouTube governo (con `Is_Short`)	Notebook 2, training governo
`referendum_con_padre.csv`	YouTube referendum + colonna `Testo_Padre`	Notebook 2, training referendum

Il dataset stampa è già pronto come input Kaggle (raccolto esternamente).

→ Procedi al **Notebook 2, Allenamento Modelli**.

Implementazione del §3.2: Annotazione e training

> Le celle che seguono implementano l'**annotazione del seed set, il fine-tuning dei due modelli BERT** (referendum e governo, con domain adaptation), e la **threshold tuning** descritti nei §3.2 e §3.3.
>
> **Nota metodologica (annotazione).** Il training set parte da un *seed set* etichettato secondo lo schema POS/NEG/NEU (§1) da un modello linguistico di grandi dimensioni (Claude Sonnet 4.6) usato come annotatore, e affinato con active learning a campionamento per incertezza (revisione manuale dei soli casi a bassa confidenza). Il seed del classificatore referendum (quello su cui poggiano i risultati del Cap. 4) conta 21.611 commenti: NEG 42,3%, POS 40,3%, NEU 17,4%; il seed del classificatore governo conta 6.649 commenti (NEG 31,0%, POS 32,2%, NEU 36,8%). Poiché l'etichettatura non si basa su più annotatori umani indipendenti, non è disponibile una misura formale di accordo inter-annotatore (es. κ di Cohen). È un limite, dichiarato come tale e indicato fra gli sviluppi futuri (doppia annotazione indipendente) nel Capitolo 7.

§1: Esplorazione del dataset annotato

Carico il seed set (commenti annotati con il workflow ibrido LLM + active learning, cfr. §3.2.3 della metodologia) e ispeziono: - Distribuzione delle etichette STANCE (necessaria per calibrare i `class_weights`) - Numero di etichettati vs non-etichettati (i secondi finiranno nell'inferenza massiva)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
# Il dataset contiene commenti annotati manualmente (seed set) + commenti non ancora etichettati
df = pd.read_csv(
    "/kaggle/input/datasets/francescochiarolanza/dataset-consenso-meloni/definitivo_governo.csv",
    encoding="utf-8-sig", sep=";", dtype=str, engine="python", on_bad_lines="skip"
)
df = df.drop(columns=[c for c in df.columns if "Unnamed" in c], errors="ignore")
print(f"Righe totali: {len(df):,}")
print(f"Colonne:      {df.columns.tolist()}")

# Normalizza la colonna STANCE (potrebbe avere case misto, NaN come stringa, ecc.)
df["STANCE"] = df["STANCE"].astype(str).str.strip().str.upper()
df["STANCE"] = df["STANCE"].replace({"NAN": np.nan, "NONE": np.nan, "": np.nan})

# Separa solo etichettati / non-etichettati. NON filtriamo per spam.
df_labeled   = df[df["STANCE"].notna()].copy()
df_unlabeled = df[df["STANCE"].isna()].copy()

print(f"\nEtichettati:     {len(df_labeled):>7,}  (usati per il training)")
print(f"Non etichettati: {len(df_unlabeled):>7,}  (passeranno all\'inferenza massiva)")

# Distribuzione STANCE
stance_counts = df_labeled["STANCE"].value_counts()
colors = {"POS": "#2ecc71", "NEG": "#e74c3c", "NEU": "#3498db"}

plt.figure(figsize=(8, 5))
plt.bar(stance_counts.index, stance_counts.values,
        color=[colors.get(s, "#999") for s in stance_counts.index])
plt.title("Distribuzione STANCE - Seed set annotato", fontsize=13, fontweight="bold")
plt.ylabel("Conteggio")
for i, (label, v) in enumerate(zip(stance_counts.index, stance_counts.values)):
    plt.text(i, v + 10, f"{v}\n({v/len(df_labeled)*100:.1f}%)", ha="center", fontsize=10)
plt.tight_layout()
plt.savefig("distribuzione_stance.png", dpi=150)
plt.show()

print("\n" + "="*50)
print("REPORT FINALE")
print("="*50)
for s in ["POS", "NEG", "NEU"]:
    n = (df_labeled["STANCE"] == s).sum()
    print(f"  {s}: {n:>5,}  ({n/len(df_labeled)*100:.1f}%)")

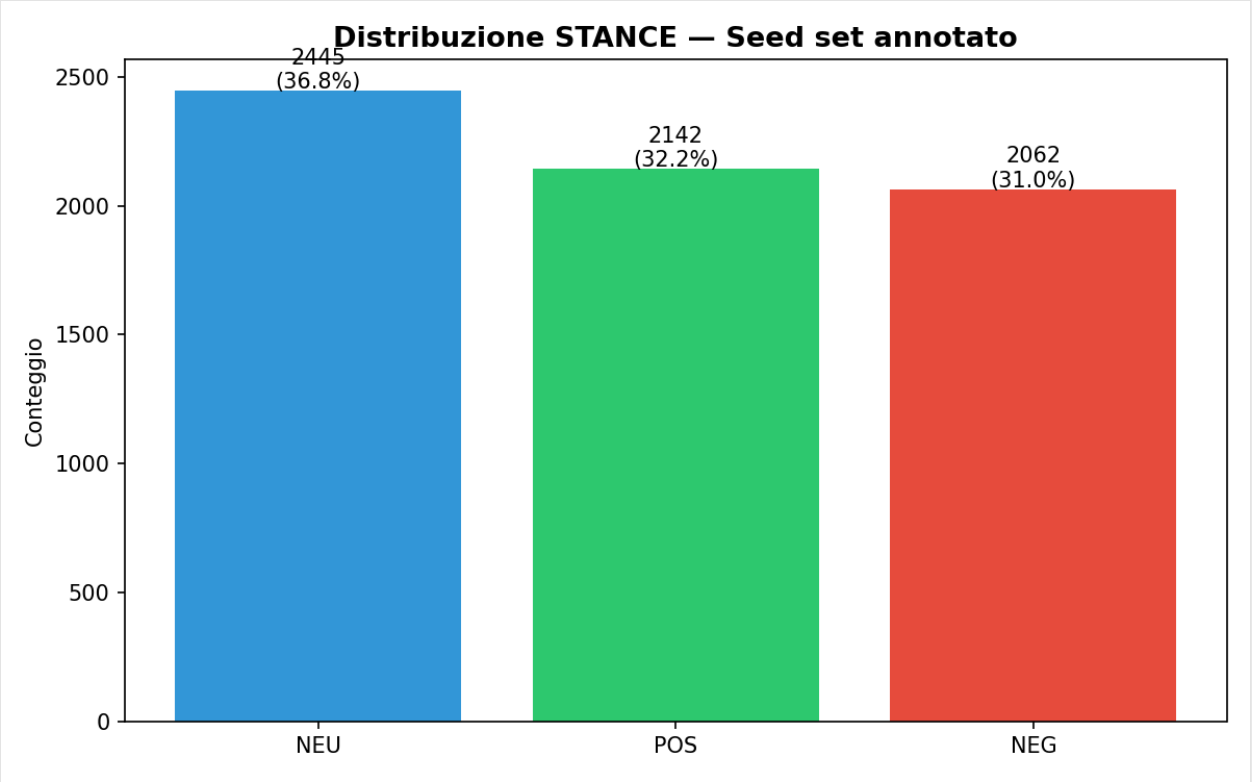
df_labeled.to_csv("seed_train.csv",  index=False, encoding="utf-8-sig", sep=";")
df_unlabeled.to_csv("unlabeled.csv", index=False, encoding="utf-8-sig", sep=";")
print(f"\nSalvato seed_train.csv ({len(df_labeled):,}) e unlabeled.csv ({len(df_unlabeled):,})")
```

```
Righe totali: 598,975
Colonne:      ['Titolo_Video', 'Canale', 'Link', 'Data_Video', 'Is_Short', 'Topic', 'Durata_ISO',
'Video_ID', 'Tipo_Commento', 'Padre_ID', 'Autore', 'Testo', 'Likes', 'Data_Commento', 'Testo_Padre',
'STANCE', 'MODELLO_PRED', 'MODELLO_CONF_NUM', 'Revisionato']

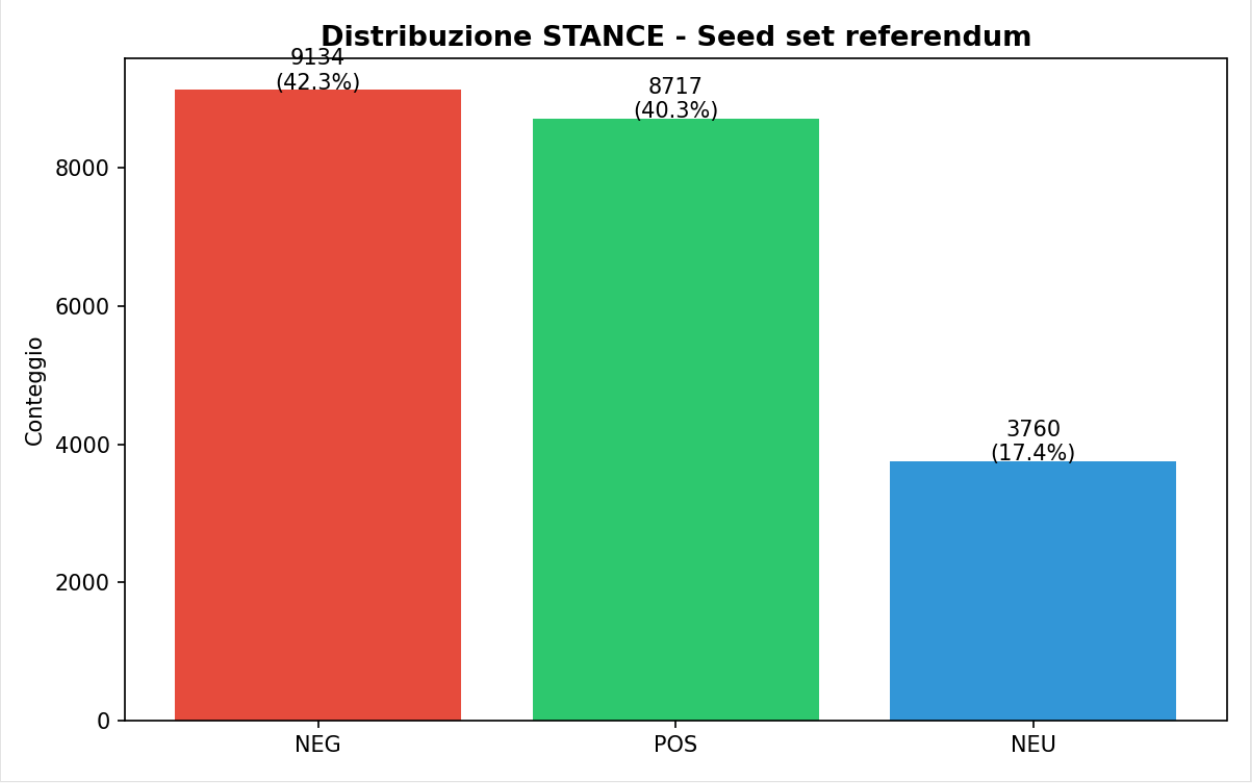
Etichettati:       6,649  (usati per il training)
Non etichettati: 592,326  (passeranno all'inferenza massiva)

==================================================
REPORT FINALE
==================================================
  POS: 2,142  (32.2%)
  NEG: 2,062  (31.0%)
  NEU: 2,445  (36.8%)

Salvato seed_train.csv (6,649) e unlabeled.csv (592,326)
```



Distribuzione delle etichette nel seed set governo (6.649 commenti annotati)



Distribuzione delle etichette nel seed set referendum (21.611 commenti annotati)

§2: Ispezione qualitativa: campione 50 POS + 50 NEG annotati

Prima del training, lettura manuale di un campione bilanciato di commenti **annotati a mano** estratti dal seed set referendum (50 etichettati **POS**, favorevoli alla riforma, e 50 **NEG**, contrari), per verificare la qualità dell'annotazione e calibrare il giudizio sul decision boundary fra le classi. Il campione è estratto con seme casuale fisso (`random_state=40`) per garantire la riproducibilità.

```
import pandas as pd

PATH = "/kaggle/input/datasets/francescochiarolanza/commenti-youtube-referendum-giustizia/Dataset
Tesi Aggiornato.csv"
df = pd.read_csv(PATH, sep=";", dtype=str, encoding="utf-8-sig", engine="python",
on_bad_lines="skip")
col = "Testo" if "Testo" in df.columns else df.columns[1]
df["STANCE"] = df["STANCE"].astype(str).str.strip().str.upper()

# 50 commenti annotati a mano per classe (seed fisso per riproducibilita').
pos = df[df["STANCE"] == "POS"].sample(n=min(50, (df["STANCE"] == "POS").sum()), random_state=40)
neg = df[df["STANCE"] == "NEG"].sample(n=min(50, (df["STANCE"] == "NEG").sum()), random_state=40)

for stance, g in [("POS", pos), ("NEG", neg)]:
    print("=" * 80)
    print(f"CAMPIONE {len(g)} COMMENTI ANNOTATI {stance}")
    print("=" * 80)
    for i, (_, row) in enumerate(g.iterrows(), 1):
        canale = row.get("Canale", row.get("Canale_Video", "N/A"))
        print(f"[{i:>3}] Canale: {canale}")
        print(f"      Testo:  {str(row.get(col,'')).replace(chr(10),' ')[:200]}")
        print("-" * 80)
    print()
```

```
================================================================================
CAMPIONE 50 COMMENTI ANNOTATI POS
================================================================================
[  1] Canale: Roberto Saviano Official
      Testo:  E ALLORA SARÀ UN BEL SI. visto che chi lucra sulla MAFIA con cinema e fiction è PEGGIO della
CRIMINALITÀ CHE FINGE DI COMBATTERE
--------------------------------------------------------------------------------
[  2] Canale: Andrea Lombardi
      Testo:  Beh non ho mai pensato di votare no solo per la presa di posizione di certi giudici, se ti da
fastidio non vuol dire che lo fai per il popolo ma per te direttamente!!...ormai l'ho capito da tempo come
--------------------------------------------------------------------------------
[  3] Canale: Matteo Gracis
      Testo:  La classe politica non può migliorare ma x questo motivo nulla potrà mai cambiare, tanto la
costituzione viene rispettata se vogliono loro, abbiamo visto durante la pandemia, la giustizia non può
anda
--------------------------------------------------------------------------------
[  4] Canale: La7 Attualità
      Testo:  Ottima analisi. Dettagliata e ben espressa. Peccato che i voti della Meloni &C. Siano sempre
stati di persone analfabete e gonfiati da brogli di malavita. Lei ci ha provato però!
--------------------------------------------------------------------------------
[  5] Canale: AGENZIA ITALIA NEWS
      Testo:  Si io Senz'altro voto SI BRIATORE SA QUANTI NÉ HO SENTITE DI QUESTE VERITA
--------------------------------------------------------------------------------
[  6] Canale: Marco Travaglio
      Testo:  Bro il CSM non deve essere "FORTE". Lo stato di diritto si basa su organi di vigilanza
PICCOLI e bilanciati che possano fare contraddittorio fra loro. La triplicazione del CSM è proprio la parte
migli
--------------------------------------------------------------------------------
[  7] Canale: Fanpage.it
      Testo:  @lidianocassani3530 Il problema è che chi ha messo in galera quel poveraccio innocente non
pagherà nulla perché non verrà licenziato, non verrà sospeso, non pagherà nessuna cifra per il risarcimento
--------------------------------------------------------------------------------
[  8] Canale: AGENZIA ITALIA NEWS
      Testo:  Io voto si perché sono stufa di vedere l'arroganza e lo strapotere dei magistrati!
--------------------------------------------------------------------------------
[  9] Canale: Gino Zavalani
```

```
      Testo:  Io voto si
--------------------------------------------------------------------------------
[ 10] Canale: Breaking Italy
      Testo:  @tomato9953 è inutile, non lo capiscono, non lo vogliono capire . VOTIAMO SI
--------------------------------------------------------------------------------
[ 11] Canale: PATRIOTA ITALIANO
      Testo:  Landini si deve solo vergognare!!! Io voto si al referendum assolutamente!!!!
--------------------------------------------------------------------------------
[ 12] Canale: Fratelli d'Italia
      Testo:  @salvatore3741 mi sembra che hai fatto un po' di confusione su quello che ho scritto e anche
su quello che hai scritto tu. Prova a rileggere. Io voto SI.
--------------------------------------------------------------------------------
[ 13] Canale: Tele Italia
      Testo:  Io Voterò Si Si Si
--------------------------------------------------------------------------------
[ 14] Canale: Roberto Saviano Official
      Testo:  @DeeperSease tu lo guardi sempre sei tu che devi cercarlo il cervello.... L'hai perso
sicuro...
--------------------------------------------------------------------------------
[ 15] Canale: Fanpage.it
      Testo:  Poveracci, se le tentano tutte, faziosi come pochi. Ma fanno quasi tenerezza a pensare il
diluvio universale di SI che stanno per arrivare
--------------------------------------------------------------------------------
[ 16] Canale: AvantiTutta
      Testo:  quanto hai ragione amico comunque tutti questi artisti sono comunisti loro pensano che sono
migliori però Fiorella quando una persona non lo sa se è un po intelligente si informa vota no chi se ne
--------------------------------------------------------------------------------
[ 17] Canale: GioPizzi
      Testo:  Peccato che fino a poco tempo fa tutti quelli che ora sono per il no erano per il sorteggio
dei magistrati
--------------------------------------------------------------------------------
[ 18] Canale: Matt Carus
      Testo:  Voto SI, basta un bricydi intelligenza per capirlo
--------------------------------------------------------------------------------
[ 19] Canale: Marco Rizzo
      Testo:  Anch'io voterò si!!
--------------------------------------------------------------------------------
[ 20] Canale: NOVE
      Testo:  i politici al potere fanno quello che vogliono tradendo regolarmente la volontà di coloro che
li hanno eletti, ma anche la magistratura, con la sua discrezionalità quasi sempre applicata a torto, fa q
--------------------------------------------------------------------------------
[ 21] Canale: Angelo Greco
      Testo:  Io voterò si
--------------------------------------------------------------------------------
[ 22] Canale: Roberto Saviano Official
      Testo:  Saviano un tuttologo. Se voti no motivo migliore per votare SI.
--------------------------------------------------------------------------------
[ 23] Canale: Il Sole 24 ORE
      Testo:  VOTIAMO SI!!!!!! L'Italia ha urgente bisogno di una riforma della giustizia per garantire
tempi rapidi, equità reale e certezza del diritto. Troppi processi durano anni, con danni enormi a
cittadini e
--------------------------------------------------------------------------------
[ 24] Canale: Breaking Italy
      Testo:  Seguono i link a due video dove si possono ascoltare direttamente dalle parole di un Giudice,
di un PM e di un Avvocato quello che ho riassunto. Dicono anche molt
--------------------------------------------------------------------------------
[ 25] Canale: Corriere della Sera
      Testo:  Io voto Siiiiiiiii
--------------------------------------------------------------------------------
[ 26] Canale: Fanpage.it
      Testo:  @michelegrimaldi8471 Sai com'è le perle me le ricordo tutte: l euro ci conviene…continente
spappolato, l elettrico e'il futuro…mercato auto demolito, Renzi votalo aiuterà noi operai..diritti
lavorator
--------------------------------------------------------------------------------
[ 27] Canale: Roberto Saviano Official
      Testo:  Un altro che vota solo politicamente senza capire nulla.
--------------------------------------------------------------------------------
[ 28] Canale: Will Media
      Testo:  La magistratura e' la piu' grande truffa autorizzata... guarda un po' in che stato hanno
ridotto l'Italia... no se, no ma
--------------------------------------------------------------------------------
[ 29] Canale: QdS
      Testo:  Io voto SI
--------------------------------------------------------------------------------
[ 30] Canale: Fanpage.it
```

```
      Testo:  La magistratura si dovrà adeguare , questa volta decideranno gli Italiani
--------------------------------------------------------------------------------
[ 31] Canale: Comitato Sì Riforma
      Testo:  @vincenzobaio4810 bravo non hai capito un cacchio. Comunque io voterò siiiiiiiiii
--------------------------------------------------------------------------------
[ 32] Canale: Marco Travaglio
      Testo:  Deve cambiare le leggi in Italia, troppi magistrati mettono la politica e non devono e poi,
con tanti sbagli che fanno, nessun magistrato paga i suoi errori. Voto SI
--------------------------------------------------------------------------------
[ 33] Canale: NOVE
      Testo:  GRANDE Antonio di Pietro l unico politico per il quale sono stata attivista e non mi ha mai
deluso e portato avanti grandi battaglie..
--------------------------------------------------------------------------------
[ 34] Canale: Rai
      Testo:  Grazie travaglio mi hai proprio convinto a vorare Si!
--------------------------------------------------------------------------------
[ 35] Canale: GioPizzi
      Testo:  Perché è un propagandista della sinistra. Mai come ora mi vergogno di dare il mio voto a
questa opposizione marcia. A sto punto meglio la Meloni
--------------------------------------------------------------------------------
[ 36] Canale: Rick DuFer
      Testo:  meriti un like per aver detto "voto si in base a quello che ho studiato di questa riforma, ma
magari sbaglio e a ragione il no" grazie detesto gli assolutismi
--------------------------------------------------------------------------------
[ 37] Canale: iStorica
      Testo:  La costituzione dal 1948 é stata modificata e riformata 16 volte. Cambiarne o non cambiarne
una parte non sarà assolutamente una novità per nessuno. Come credi che abbiano diminuito il numero di
deput
--------------------------------------------------------------------------------
[ 38] Canale: efe bal
      Testo:  Voto si sperando che queste sciocchezze non succedano più
--------------------------------------------------------------------------------
[ 39] Canale: iStorica
      Testo:  Voterò SÌ
--------------------------------------------------------------------------------
[ 40] Canale: Contenuti qualità
      Testo:  Anche io e voterò SI. Finalmente un po' di chiarimenti....
--------------------------------------------------------------------------------
[ 41] Canale: Max Stefany Official
      Testo:  Io voto si. Affinche finisce questa mafia giudiziaria. Dove i veri MAGISTRATI, vengono sempre
esiliati in coda, dando libero accesso solo ai "soci"
--------------------------------------------------------------------------------
[ 42] Canale: Breaking Italy
      Testo:  Il problema è che gli italiani sono stanchi di questa mala giustizia. Sapete quanti innocenti
sono in carcere? E sapete quanti delinquenti sono a piede libero? C'è qualcosa che non funziona. Le corren
--------------------------------------------------------------------------------
[ 43] Canale: Fanpage.it
      Testo:  (commento composto di sole emoji)
--------------------------------------------------------------------------------
[ 44] Canale: La7 Attualità
      Testo:  Voterò Si specialmente per i gufi di Sinistra
--------------------------------------------------------------------------------
[ 45] Canale: Fratelli d'Italia
      Testo:  Io voto SI!!!
--------------------------------------------------------------------------------
[ 46] Canale: La Repubblica
      Testo:  Io sono un indagato un mafioso ed un massone deviato per cui voto si
--------------------------------------------------------------------------------
[ 47] Canale: Fanpage.it
      Testo:  Siiiiiiiiiiiiii
--------------------------------------------------------------------------------
[ 48] Canale: scrip
      Testo:  Interessantissima l'ultima riflessione
--------------------------------------------------------------------------------
[ 49] Canale: Cittadino Zero
      Testo:  Bravo io voto si
--------------------------------------------------------------------------------
[ 50] Canale: Fanpage.it
      Testo:  serve a indebolire LE CORRENTI ROSSE nella magistratura. Quelle di cui lui fa parte e che
hanno fatto fin troppi danni all Italia, insomma Gratteri con te l Italia farà i conti dopo il referendum.
--------------------------------------------------------------------------------

================================================================================
CAMPIONE 50 COMMENTI ANNOTATI NEG
================================================================================
```

```
[  1] Canale: Manuel Giommi
      Testo:  In risposta a SEBYR mi piace come ha scritto, sono d'accordo. Io ho fiducia di chi non sta al
potere oggi perchè girano le carte a loro favore perchè sono la MAGGIORANZA sia al Potere sia nell Magistr
--------------------------------------------------------------------------------
[  2] Canale: scrip
      Testo:  Ciao, Ermanno! Non aprivo un tuo video dal 2021, quando ancora frequentavo il liceo classico,
ero all'ultimo anno. Oggi sono all'università e voglio diventare un'insegnante come te; mi fa piacere ved
--------------------------------------------------------------------------------
[  3] Canale: GioPizzi
      Testo:  Quale casino? È talmente palese il tutto che è diventato stucchevole parlarne ancora. Creando
sì vera confusione. Voterò NO, senza il minimo dubbio...
--------------------------------------------------------------------------------
[  4] Canale: Marco Travaglio
      Testo:  Voto NO io e la mia famiglia
--------------------------------------------------------------------------------
[  5] Canale: Fanpage.it
      Testo:  Antò è diventato un politico come tutti gli altri, falso
--------------------------------------------------------------------------------
[  6] Canale: La7 Attualità
      Testo:  Io voto no Calenda mi fa pena che tristezza! ma dove andiamo con persone così!?
--------------------------------------------------------------------------------
[  7] Canale: Il Fatto Quotidiano
      Testo:  Personalmente voterò NO al referendum e lo faccio convintamente perché credo che il primo
atto di controllo del Parlamento e quindi politico sulla Magistratura si verificherà proprio con la scelta
a s
--------------------------------------------------------------------------------
[  8] Canale: SKANDALISSIMO ITALIA
      Testo:  Non servono tecnicismi per capire il reale intento di questa riforma; basta ascoltare chi
l'ha scritta o la sostiene. Dietro lo slogan dell'efficienza si cela un disegno di assoggettamento.
--------------------------------------------------------------------------------
[  9] Canale: Fanpage.it
      Testo:  No, gestire le persone non è la stessa cosa che valutare prove e testimonianze in un processo
o dirigere una indagine. Sono due cose diverse e si può essere ottimi giudici e pessimi rappresentanti dei
--------------------------------------------------------------------------------
[ 10] Canale: Vista Agenzia Televisiva Nazionale
      Testo:  Francamente non capisco che centra sto discorso a prescidere dal fatto che io voteró NO.
Voterò no perche la materia é troppo complessa per farla votare da chi non la conosce a fondo e perche il
cambi
--------------------------------------------------------------------------------
[ 11] Canale: Matteo Gracis
      Testo:  voto no
--------------------------------------------------------------------------------
[ 12] Canale: Fanpage.it
      Testo:  Di Pietro ormai ti sei perso del tutto! Che tristezza! Che vergogna!
--------------------------------------------------------------------------------
[ 13] Canale: Fanpage.it
      Testo:  Voterò NO, voglio una Magistratura IN.DI.PEN.DEN TE
--------------------------------------------------------------------------------
[ 14] Canale: Angelo Greco
      Testo:  io voto NO, come la maggioranza degli italiani.
--------------------------------------------------------------------------------
[ 15] Canale: La Repubblica
      Testo:  Grande Gratteri!!! Qualcuno evidentemente ha la coda di paglia. Adesso più convinto di prima
voto NO
--------------------------------------------------------------------------------
[ 16] Canale: AGENZIA ITALIA NEWS
      Testo:  CASSESE EUNA PERSONA PERBENE ECOLTO MAIO NO VOTA COME LA SANTACHE OCOME DELUTRI EMI VIENE
UNDUBIO EQUINDI VOTO NO IONONSONO NESUNO MA CIO 74 ANNI ELIOVISTI TUTTI AL'OPERA ETUTTO QUELLO CHE ANN
--------------------------------------------------------------------------------
[ 17] Canale: Breaking Italy
      Testo:  @AndreaArmenib ha mi sa che ora pure peggio
--------------------------------------------------------------------------------
[ 18] Canale: Giorgia Meloni News
      Testo:  Adesso che ti ho ascoltato... sono sicuro..VOTERò NO.... finchè vedo la Santanchè al
Governo.. non ti crederò mai!!!
--------------------------------------------------------------------------------
[ 19] Canale: Marco Rizzo
      Testo:  Io non credo che il capotreno avrebbe avuto giustizia se ci fosse stata la separazione delle
carriere.( Ci vuole altro...) Io voterò no.
--------------------------------------------------------------------------------
[ 20] Canale: Roberto Saviano Official
      Testo:  Voto no
--------------------------------------------------------------------------------
[ 21] Canale: Sovranità Popolare Napolitana SNP
      Testo:  No tutta la vita. I ladroni che vogliono impunità e libertà di rubare.
```

```
--------------------------------------------------------------------------------
[ 22] Canale: Fanpage.it
      Testo:  NON ESISTE NESSUN PRO IN QUESTO REFERENDUM E NESSUNA RAGIONE DI CHI SOSTIENE IL SI .
--------------------------------------------------------------------------------
[ 23] Canale: Il Fatto Quotidiano
      Testo:  Grazie mille Dottor Scarpinato Io voto NO
--------------------------------------------------------------------------------
[ 24] Canale: Strano Mondo
      Testo:  non hai ascoltato Nordio, voto NO
--------------------------------------------------------------------------------
[ 25] Canale: Gennaro Marco Duello
      Testo:  Parla parla ma non argomenta ne avvalora le ragioni per votare Sì! A parte il fatto che non
so nemmeno chi sia Lei ma a parte questo NO tutta la vita!
--------------------------------------------------------------------------------
[ 26] Canale: Fanpage.it
      Testo:  Votare sì potrebbe significare che i pubblici ministeri potrebbero diventare più esposti
all'influenza del potere politico, subendo pressioni esterne anche senza ordini diretti. Oggi i PM fanno
parte
--------------------------------------------------------------------------------
[ 27] Canale: Tele Italia
      Testo:  Allora io non sono saggio e voterò NO!!
--------------------------------------------------------------------------------
[ 28] Canale: NOVE
      Testo:  Di pietro NON SEI CREDIBILE NON SEI CHIARO NO NO SEI STRANO
--------------------------------------------------------------------------------
[ 29] Canale: Fanpage.it
      Testo:  IO voto no
--------------------------------------------------------------------------------
[ 30] Canale: Breaking Italy
      Testo:  Io voterò NO!!
--------------------------------------------------------------------------------
[ 31] Canale: La7 Attualità
      Testo:  VOTO NO
--------------------------------------------------------------------------------
[ 32] Canale: NOVE
      Testo:  Gli scenari che appaiono "possibili" diventano molto più concreti se si considera chi sono i
proponenti della riforma e la loro storia politica. Le intenzioni dichiarate e il contesto politico rendono
--------------------------------------------------------------------------------
[ 33] Canale: Angelo Greco
      Testo:  il voto non conta più niente purtroppo
--------------------------------------------------------------------------------
[ 34] Canale: Will Media
      Testo:  In breve stanno facendo come fece Renzi, mettono una cosa buona e la usano come grido di
giustizia, ma sotto ci piazzano l'80% di robe politiche per fare i propri interessi politici... Cambiano i
part
--------------------------------------------------------------------------------
[ 35] Canale: TeleRegione TV
      Testo:  Si lei voti come gli pare , io voto NO.
--------------------------------------------------------------------------------
[ 36] Canale: Roberto Saviano Official
      Testo:  NO anche per me. Grazie Roberto per il tuo lavoro.
--------------------------------------------------------------------------------
[ 37] Canale: Breaking Italy
      Testo:  Grazie per il video tra i più completi. Vorrei fare una domanda: Attualmente i membri laici
del CSM li sceglie il parlamento con maggioranza qualificata (che dovrebbe essere il 60% dei voti). Quindi
s
--------------------------------------------------------------------------------
[ 38] Canale: NOVE
      Testo:  Votare per il sì è scandaloso
--------------------------------------------------------------------------------
[ 39] Canale: Giuseppe Salamone
      Testo:  Ecco cosa serve il referendum. A non avere la magistratura che li può perseguire per le loro
malefatte. Voto NO anche per questo.
--------------------------------------------------------------------------------
[ 40] Canale: Marco Travaglio
      Testo:  Marco grazie sempre di cuore! Io voterò NO! È troppo evidente quanto questi mascalzoni, usano
strategie per scopi pericolosi, pensando che il popolo italiano si è addormentato!!
--------------------------------------------------------------------------------
[ 41] Canale: Alessio Muceli
      Testo:  Io voto nooooo
--------------------------------------------------------------------------------
[ 42] Canale: Marco Travaglio
      Testo:  Io voto NO!
--------------------------------------------------------------------------------
[ 43] Canale: Rai
```

```
      Testo:  Dovrei votare sì per sistemare la Santanche' Montaruli o Delmastro? NO grazie.
--------------------------------------------------------------------------------
[ 44] Canale: Roberto Saviano Official
      Testo:  I soliti commenti di infantili, fassisti, senza argomenti che appunto per ignoranza,
infantilismo, e mancanza di argomenti, anziché starsene zitti, nel vano tentativo di zittire e criticare
l'inte
--------------------------------------------------------------------------------
[ 45] Canale: GioPizzi
      Testo:  Guardati il superenalotto se vuoi il sorteggio babbo
--------------------------------------------------------------------------------
[ 46] Canale: Barlogica
      Testo:  Un lume nel buio.... Grazie. Penso voterò no nonostante un organo che si auto giudica non è
il massimo, ma c'è troppo puzza di un'intromissione prepotente della politica nell'unico organo che può
rego
--------------------------------------------------------------------------------
[ 47] Canale: AGENZIA ITALIA NEWS
      Testo:  Io voterò no perché votare sì non si risolve assolutamente questi problemi. Anzi. Allo stesso
tempo dico che quello che ti è stato fatto è veramente schifoso. Vendere la barca appena prima della sente
--------------------------------------------------------------------------------
[ 48] Canale: Fanpage.it
      Testo:  Le ragioni di Di Pietro sono risibili. Ma come ha fatto il magistrato per tanti anni? .
Difendiamoci da questi personaggi sempre in cerca di poltrone. Difendiamo la nostra è vostra costituzione
votate
--------------------------------------------------------------------------------
[ 49] Canale: iStorica
      Testo:  Complimenti....la Storia!
--------------------------------------------------------------------------------
[ 50] Canale: Roberto Saviano Official
      Testo:  GRANDE Saviano...uno scrittore che ci invidiano tutti
--------------------------------------------------------------------------------
```

§3: Fine-tuning BERT, Stance Detection Referendum

Pipeline di training:

1. **Carica** il dataset annotato (con `Testo_Padre` per le risposte)
2. **Costruisci input**, `text_a` = padre (per le risposte) o testo (per i principali); `text_b` = risposta o vuoto. BERT inserisce automaticamente `[CLS] text_a [SEP] text_b [SEP]`.
3. **Encoding label**, `NEG=0, NEU=1, POS=2` (mappa diretta agli ID del modello sentiment pre-addestrato)
4. **Split stratificato 80/10/10** train/val/test
5. **Tokenizzazione**, `padding="max_length"`, `truncation=True`, `max_length=256` (lasciamo spazio per padre+risposta)
6. **Class weights** inversi alla frequenza per bilanciare classi rare
7. **Training** con `WeightedTrainer` (loss pesata) ed `EarlyStopping` su F1 macro
8. **Valutazione** su test set + confusion matrix
9. **Analisi per tipo**, accuracy commenti principali vs risposte (con/senza padre)
10. **Salvataggio** modello in `./model_stance_final/` + `label_info_stance.pkl`

§3.0: Installazione transformers

```
# Fix ambiente Kaggle: il `peft` preinstallato attiva un import dentro
# `transformers.trainer` (`from peft import PeftModel`) che fallisce per
# disallineamenti interni a peft stesso post-upgrade. Questo notebook NON usa
# peft (nessun LoRA/adapter), quindi lo disinstalliamo e pinniamo transformers
# a una versione coerente. Senza peft, transformers gestisce il branch via
# `is_peft_available() == False`.
import sys
```

```
!pip uninstall -y -q peft
!pip install -q --upgrade "transformers==4.46.3" "accelerate>=0.34" "datasets>=2.20"

# Svuota la cache dei moduli già caricati (evita di dover riavviare il kernel)
for _mod in list(sys.modules):
    if _mod.startswith(("transformers", "peft", "accelerate")):
        del sys.modules[_mod]

import transformers, accelerate
print("transformers:", transformers.__version__)
print("accelerate: ", accelerate.__version__)
print("peft:         disinstallato (non serve per fine-tuning standard)")
```

§3.1: Training pipeline completo

```
import pandas as pd
import numpy as np
import torch
import pickle
import shutil
import os
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    classification_report, confusion_matrix,
    f1_score, accuracy_score
)
from transformers import (
    AutoTokenizer, AutoModelForSequenceClassification,
    TrainingArguments, Trainer, EarlyStoppingCallback
)
from datasets import Dataset
from torch import nn

# ============================================================
# CONFIGURAZIONE KAGGLE
# ============================================================
# Usa il modello base italiano
MODEL_NAME = "dbmdz/bert-base-italian-xxl-uncased"
# Sostituisci questo percorso con quello esatto del tuo dataset su Kaggle!
PATH_DATASET = "/kaggle/input/datasets/francescochiarolanza/commenti-youtube-referendum-
giustizia/Dataset Tesi Aggiornato.csv"

MAX_LENGTH = 256
BATCH_SIZE = 16
EPOCHS     = 10
TASK       = "stance_referendum"
LABEL_COL  = "STANCE"

print(f"🎯 Task: {TASK}")
print(f"📦 Modello base: {MODEL_NAME}")

# ============================================================
# PULIZIA DISCO (Utile su Kaggle per non saturare lo spazio)
# ============================================================
for folder in [f"./model_{TASK}", f"./logs_{TASK}"]:
    if os.path.exists(folder):
        shutil.rmtree(folder)

# ============================================================
# LABEL MAP
# ============================================================
LABEL_VALIDE = {"POS", "NEG", "NEU"}
```

```
LABEL_MAP = {"NEG": 0, "NEU": 1, "POS": 2}
LABEL_NAMES = ["NEG", "NEU", "POS"]

# ============================================================
# CARICAMENTO E PREPARAZIONE DATASET
# ============================================================
print("📂 Caricamento dataset...")
df = pd.read_csv(PATH_DATASET, encoding="utf-8-sig", sep=";", dtype=str, engine="python",
on_bad_lines="skip")
df = df.drop(columns=[c for c in df.columns if "Unnamed" in c], errors="ignore")

df = df.dropna(subset=["Testo", LABEL_COL])
df["Testo"] = df["Testo"].astype(str).str.strip()
df[LABEL_COL] = df[LABEL_COL].astype(str).str.strip().str.upper()

has_padre = "Testo_Padre" in df.columns
has_titolo = "Titolo_Video" in df.columns

df["text_a"] = ""
df["text_b"] = ""

for i, row in df.iterrows():
    commento = str(row["Testo"]).strip()
    titolo = str(row.get("Titolo_Video", "")).strip() if has_titolo else ""
    padre = ""

    if has_padre and str(row.get("Tipo_Commento", "")) == "Risposta" and
pd.notna(row.get("Testo_Padre")):
        padre = str(row["Testo_Padre"]).strip()

    if titolo and padre and len(padre) > 3:
        contesto = titolo[:80] + " | " + padre[:200]
    elif titolo:
        contesto = titolo[:150]
    elif padre and len(padre) > 3:
        contesto = padre[:250]
    else:
        contesto = ""

    df.at[i, "text_a"] = contesto if contesto else commento
    df.at[i, "text_b"] = commento if contesto else ""

df = df[df["text_a"].str.len() > 3]
df = df[df[LABEL_COL].isin(LABEL_VALIDE)].copy()
df["label"] = df[LABEL_COL]
df["label_id"] = df["label"].map(LABEL_MAP)

print(f"\n📊 Campioni disponibili per l'addestramento: {len(df)}")

# ============================================================
# SPLIT STRATIFICATO 80/10/10
# ============================================================
df_train, df_temp = train_test_split(df, test_size=0.2, random_state=42, stratify=df["label_id"])
df_val, df_test = train_test_split(df_temp, test_size=0.5, random_state=42,
stratify=df_temp["label_id"])
print(f"🔀 Split - Train: {len(df_train)} | Val: {len(df_val)} | Test: {len(df_test)}")

# ============================================================
# TOKENIZZAZIONE (Con fix 'only_first')
# ============================================================
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)

def tokenize(batch):
    texts_a = batch["text_a"]
    texts_b = batch["text_b"]
    texts_b_clean = [b if b and len(b) > 0 else None for b in texts_b]
```

```
    final_a, final_b = [], []
    for a, b in zip(texts_a, texts_b_clean):
        final_a.append(a)
        final_b.append(b if b else "")

    return tokenizer(
        final_a,
        final_b,
        padding="max_length",
        truncation=True,  # 🔥 FIX CRITICO: Taglia solo il contesto, mai il commento!
        max_length=MAX_LENGTH
    )

def to_dataset(df_split):
    ds = Dataset.from_dict({
        "text_a": df_split["text_a"].tolist(),
        "text_b": df_split["text_b"].fillna("").tolist(),
        "labels": df_split["label_id"].tolist()
    })
    return ds.map(tokenize, batched=True)

ds_train = to_dataset(df_train)
ds_val   = to_dataset(df_val)
ds_test  = to_dataset(df_test)

# ============================================================
# INIZIALIZZAZIONE MODELLO E PESI
# ============================================================
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"\n💻 Device attivo: {device}")

model = AutoModelForSequenceClassification.from_pretrained(MODEL_NAME, num_labels=3,
ignore_mismatched_sizes=True)
model.to(device)

counts = df_train["label_id"].value_counts().sort_index().values
class_weights = torch.tensor(1.0 / counts, dtype=torch.float)
class_weights = class_weights / class_weights.sum() * 3
# Non spostiamo i pesi sul device qui, lo facciamo dinamicamente nel Trainer!

def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = np.argmax(logits, axis=-1)
    return {
        "accuracy":    accuracy_score(labels, preds),
        "f1_macro":    f1_score(labels, preds, average="macro")
    }

# ============================================================
# CUSTOM TRAINER (Con fix GPU dinamica)
# ============================================================
class WeightedTrainer(Trainer):
    def compute_loss(self, model, inputs, return_outputs=False, **kwargs):
        labels  = inputs.pop("labels")
        outputs = model(**inputs)
        logits  = outputs.logits

        # 🔥 FIX CRITICO: Allinea dinamicamente i pesi al device dei logit
        weights_on_device = class_weights.to(logits.device)

        loss    = nn.CrossEntropyLoss(weight=weights_on_device)(logits, labels)
        return (loss, outputs) if return_outputs else loss

training_args = TrainingArguments(
    output_dir                  = f"./model_{TASK}",
```

```
    num_train_epochs            = EPOCHS,
    per_device_train_batch_size = BATCH_SIZE,
    per_device_eval_batch_size  = BATCH_SIZE * 2,
    eval_strategy               = "epoch",
    save_strategy               = "epoch",
    save_total_limit            = 1,
    load_best_model_at_end      = True,
    metric_for_best_model       = "f1_macro",
    greater_is_better           = True,
    logging_dir                 = f"./logs_{TASK}",
    logging_steps               = 50,
    warmup_ratio                = 0.1,
    weight_decay                = 0.01,
    learning_rate               = 1e-5,
    fp16                        = torch.cuda.is_available(),
    report_to                   = "none",
)

trainer = WeightedTrainer(
    model           = model,
    args            = training_args,
    train_dataset   = ds_train,
    eval_dataset    = ds_val,
    compute_metrics = compute_metrics,
    callbacks       = [EarlyStoppingCallback(early_stopping_patience=3)]
)

# ============================================================
# AVVIO ADDESTRAMENTO E VALUTAZIONE
# ============================================================
print(f"\n🚀 Avvio addestramento...")
trainer.train()

print(f"\n📊 Valutazione finale sul Test Set Cieco:")
predictions = trainer.predict(ds_test)
preds       = np.argmax(predictions.predictions, axis=-1)
labels_test = predictions.label_ids

print(classification_report(labels_test, preds, target_names=LABEL_NAMES, digits=3,
zero_division=0))

# Salvataggio Matrice di Confusione
cm = confusion_matrix(labels_test, preds)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt="d", xticklabels=LABEL_NAMES, yticklabels=LABEL_NAMES,
cmap="Blues")
plt.title(f"Confusion Matrix - {TASK}")
plt.ylabel("Reale")
plt.xlabel("Predetto")
plt.tight_layout()
plt.savefig(f"/kaggle/working/cm_{TASK}.png", dpi=150)
print(f"📉 Matrice di confusione salvata come cm_{TASK}.png")

# ============================================================
# SALVATAGGIO MODELLO FINALE
# ============================================================
print("\n💾 Salvataggio del modello vincitore...")
trainer.save_model(f"/kaggle/working/model_{TASK}_final")
tokenizer.save_pretrained(f"/kaggle/working/model_{TASK}_final")

print("✅ Operazione conclusa! Il modello è salvato e pronto per l'inferenza.")
```

```
# ============================================================
# COMPRESSIONE E DOWNLOAD (SICUREZZA KAGGLE)
# ============================================================
```

```
import shutil
from IPython.display import FileLink, display

cartella_modello = f"/kaggle/working/model_{TASK}_final"
nome_zip         = f"modello_{TASK}_definitivo"

print("\n⏳ Compressione del modello in corso... (potrebbe volerci circa un minuto)")
shutil.make_archive(nome_zip, "zip", cartella_modello)

print(f"✅ Compressione completata! Clicca il link qui sotto per scaricarlo:")
display(FileLink(f"{nome_zip}.zip"))
```

§3.2: Compressione e download del modello (opzionale)

Comprime il modello salvato in `.zip` per il download da Kaggle (necessario per esportarlo come dataset/model riutilizzabile).

```
import shutil
from IPython.display import FileLink

cartella_modello = "/kaggle/working/model_stance_final"
nome_zip         = "/kaggle/working/modello_stance_xxl"

print("⏳ Compressione del modello in corso... (BERT XXL pesa ~1.3GB)")
shutil.make_archive(nome_zip, "zip", cartella_modello)

print("✅ Compressione completata.")
FileLink(r"modello_stance_xxl.zip")
```

Il codice sottostante mostra le metriche del modello senza la necessità di rifare il training

```
  import os
  os.environ["WANDB_DISABLED"] = "true"
  os.environ["WANDB_MODE"]     = "disabled"

  import pandas as pd, numpy as np, json
  import seaborn as sns, matplotlib.pyplot as plt
  from sklearn.model_selection import train_test_split
  from sklearn.metrics import classification_report, confusion_matrix, f1_score, accuracy_score
  from transformers import (AutoTokenizer, AutoModelForSequenceClassification,
                            Trainer, TrainingArguments)
  from datasets import Dataset

  MODEL_DIR  = "/kaggle/input/models/francescochiarolanza/modello-stance-
referendum/other/default/2/modello_stance_xxl V2"
  DATA_PATH  = "/kaggle/input/datasets/francescochiarolanza/commenti-youtube-referendum-
giustizia/Dataset Tesi Aggiornato.csv"
  MAX_LENGTH = 256
  LABEL_MAP   = {"NEG": 0, "NEU": 1, "POS": 2}
  LABEL_NAMES = ["NEG", "NEU", "POS"]

  # --- Ricarica dataset con stesso preprocessing del training ---
  df = pd.read_csv(DATA_PATH, encoding="utf-8-sig", sep=";", dtype=str,
                   engine="python", on_bad_lines="skip")
  df = df.drop(columns=[c for c in df.columns if "Unnamed" in c], errors="ignore")
  df = df.dropna(subset=["Testo", "STANCE"])
  df["Testo"]  = df["Testo"].astype(str).str.strip()
  df["STANCE"] = df["STANCE"].astype(str).str.strip().str.upper()

  df["text_a"] = np.nan
  df["text_b"] = np.nan
  if "Testo_Padre" in df.columns:
```

```
      for i, row in df.iterrows():
          padre = str(row.get("Testo_Padre") or "").strip()
          if row.get("Tipo_Commento") == "Risposta" and len(padre) > 3:
              df.at[i, "text_a"] = padre
              df.at[i, "text_b"] = row["Testo"]
          else:
              df.at[i, "text_a"] = row["Testo"]
  else:
      df["text_a"] = df["Testo"]

  df = df[df["text_a"].astype(str).str.len() > 3]
  df = df[df["STANCE"].isin(LABEL_MAP)].copy()
  df["label_id"] = df["STANCE"].map(LABEL_MAP)

  # --- Stesso split 80/10/10 con seed 42 → estrae il test set ---
  df_train, df_temp = train_test_split(df, test_size=0.2, random_state=42, stratify=df["label_id"])
  df_val,   df_test = train_test_split(df_temp, test_size=0.5, random_state=42,
stratify=df_temp["label_id"])
  print(f"Test set ricostruito: {len(df_test)} esempi")
  print("Distribuzione classi (test):")
  print(df_test["STANCE"].value_counts())

  # --- Carica modello + tokenizer ---
  tokenizer = AutoTokenizer.from_pretrained(MODEL_DIR)
  model     = AutoModelForSequenceClassification.from_pretrained(MODEL_DIR)

  def tokenize(batch):
      tb = [b if b is not None else "" for b in batch["text_b"]]
      return tokenizer(batch["text_a"], tb,
                       padding="max_length", truncation=True, max_length=MAX_LENGTH)

  ds_test = Dataset.from_dict({
      "text_a": df_test["text_a"].tolist(),
      "text_b": [b if pd.notna(b) else None for b in df_test["text_b"]],
      "labels": df_test["label_id"].tolist(),
  }).map(tokenize, batched=True)

  # --- Trainer con report_to="none" → niente wandb / tensorboard / hub ---
  eval_args = TrainingArguments(
      output_dir="./_eval_tmp",
      per_device_eval_batch_size=32,
      report_to="none",
      logging_strategy="no",
      save_strategy="no",
      do_train=False,
      do_eval=False,
  )
  trainer = Trainer(model=model, args=eval_args, tokenizer=tokenizer)

  # --- Inferenza ---
  out    = trainer.predict(ds_test)
  y_pred = np.argmax(out.predictions, axis=-1)
  y_true = np.array(df_test["label_id"])

  # --- Metriche ---
  print("\n" + "="*60)
  print("CLASSIFICATION REPORT")
  print("="*60)
  print(classification_report(y_true, y_pred, target_names=LABEL_NAMES, digits=3))
  print(f"Accuracy : {accuracy_score(y_true, y_pred):.4f}")
  print(f"F1 macro : {f1_score(y_true, y_pred, average='macro'):.4f}")
  print(f"F1 weight: {f1_score(y_true, y_pred, average='weighted'):.4f}")

  # --- Confusion matrix ---
  cm = confusion_matrix(y_true, y_pred, labels=[0, 1, 2])
  fig, axes = plt.subplots(1, 2, figsize=(13, 5))
```

```
  sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
              xticklabels=LABEL_NAMES, yticklabels=LABEL_NAMES,
              cbar=False, ax=axes[0])
  axes[0].set_xlabel("Predetto"); axes[0].set_ylabel("Reale")
  axes[0].set_title("Confusion Matrix - conteggi assoluti")

  cm_norm = cm.astype(float) / cm.sum(axis=1, keepdims=True)
  sns.heatmap(cm_norm, annot=True, fmt=".2%", cmap="Greens",
              xticklabels=LABEL_NAMES, yticklabels=LABEL_NAMES,
              cbar=False, ax=axes[1])
  axes[1].set_xlabel("Predetto"); axes[1].set_ylabel("Reale")
  axes[1].set_title("Confusion Matrix - recall per classe (riga = 100%)")

  plt.tight_layout()
  plt.savefig("/kaggle/working/confusion_matrix.png", dpi=150, bbox_inches="tight")
  plt.show()

  # --- Storico training (se presente trainer_state.json) ---
  ts_path = os.path.join(MODEL_DIR, "trainer_state.json")
  if os.path.exists(ts_path):
      with open(ts_path) as f:
          state = json.load(f)
      print("\n" + "="*60)
      print("STORICO TRAINING (per epoca)")
      print("="*60)
      for log in state.get("log_history", []):
          if "eval_f1_macro" in log:
              print(f"  epoch {log['epoch']:>4.1f} → "
                    f"F1 macro {log['eval_f1_macro']:.4f} | "
                    f"acc {log.get('eval_accuracy', float('nan')):.4f} | "
                    f"loss {log.get('eval_loss', float('nan')):.4f}")
  else:
      print(f"\n(Nessun trainer_state.json in {MODEL_DIR})")
```

```
Test set ricostruito: 2162 esempi
Distribuzione classi (test):
STANCE
NEG    914
POS    872
NEU    376
Name: count, dtype: int64

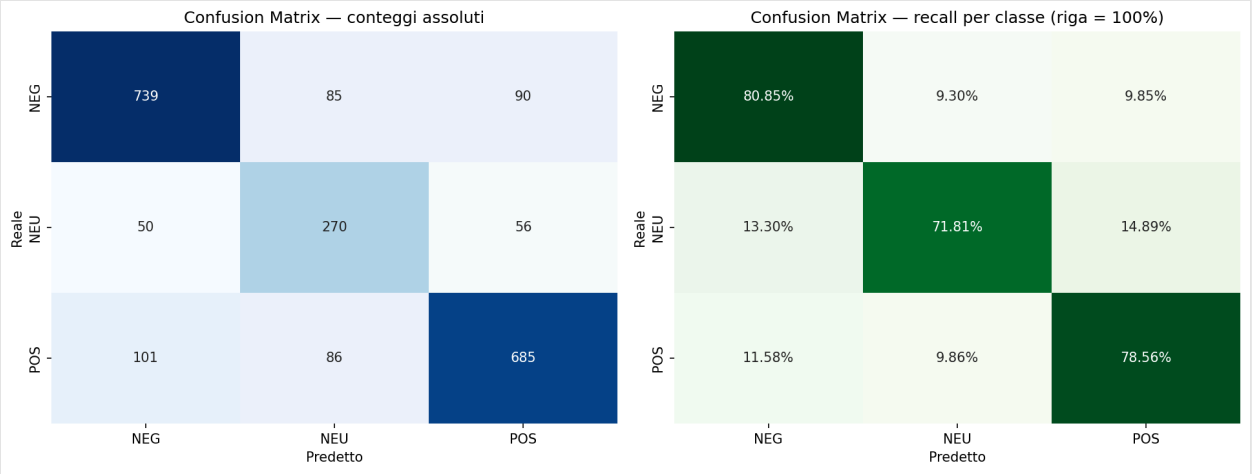
============================================================
CLASSIFICATION REPORT
============================================================
              precision    recall  f1-score   support

         NEG      0.830     0.809     0.819       914
         NEU      0.612     0.718     0.661       376
         POS      0.824     0.786     0.804       872

    accuracy                          0.784      2162
   macro avg      0.756     0.771     0.762      2162
weighted avg      0.790     0.784     0.786      2162

Accuracy : 0.7835
F1 macro : 0.7616
F1 weight: 0.7858

(Nessun trainer_state.json nel modello pre-addestrato caricato)
```



Matrice di confusione sul test set, modello referendum (F1 macro 0,76)

§4: Inferenza Massiva sul dataset YouTube Referendum

Una volta che il modello è stato (eventualmente) raffinato con active learning, lo applichiamo a **tutti** i commenti del dataset (etichettati e non) per produrre la colonna `PREDIZIONE_STANCE` usata in tutte le analisi del notebook 3.

Output: `risultati_inferenza_completa.csv` con tutte le colonne originali + `PREDIZIONE_STANCE` (POS/NEG/NEU) e `CONFIDENZA` (probabilità della classe predetta).

```
import pandas as pd
import numpy as np
import torch
import torch.nn.functional as F
from transformers import AutoTokenizer, AutoModelForSequenceClassification
from torch.utils.data import DataLoader, Dataset
from tqdm import tqdm
import os

# ========================================
# CONFIGURAZIONE
# ========================================
WORKING_DIR    = "/kaggle/working/"
FULL_DATA_FILE = "/kaggle/input/datasets/francescochiarolanza/commenti-youtube-referendum-
giustizia/Dataset Tesi.csv"
MODEL_DIR      = "/kaggle/input/models/francescochiarolanza/modello-stance-
referendum/other/default/2/modello_stance_xxl V2"
OUTPUT_CSV     = os.path.join(WORKING_DIR, "risultati_inferenza_completa.csv")

ID2LABEL   = {0: "NEG", 1: "NEU", 2: "POS"}
MAX_LENGTH = 256
BATCH_SIZE = 32     # se va in OOM abbassa a 16

print("🚀 Inizio pipeline di inferenza massiva...")

# ========================================
# 1. CARICAMENTO DATI
# ========================================
df = pd.read_csv(FULL_DATA_FILE, sep=";", dtype=str, encoding="utf-8-sig")

# Prepara coppie (padre, risposta) coerenti con il training
df["text_a"] = df["Testo"].fillna("").astype(str).str.strip()
df["text_b"] = None
```

```
if "Testo_Padre" in df.columns and "Tipo_Commento" in df.columns:
    for i, row in df.iterrows():
        if row.get("Tipo_Commento") == "Risposta" and pd.notna(row.get("Testo_Padre")):
            padre = str(row["Testo_Padre"]).strip()
            if len(padre) > 3:
                df.at[i, "text_a"] = padre
                df.at[i, "text_b"] = row["Testo"]

# Filtro commenti troppo corti (< 4 caratteri = inutili anche con BERT)
df = df[df["Testo"].astype(str).str.len() > 3].copy().reset_index(drop=True)
print(f"📊 Commenti da analizzare: {len(df):,}")

# ============================================
# 2. MODELLO
# ============================================
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"⚙️ Caricamento modello da: {MODEL_DIR} su {device}")

tokenizer = AutoTokenizer.from_pretrained(MODEL_DIR)
model     = AutoModelForSequenceClassification.from_pretrained(MODEL_DIR).to(device)
model.eval()


# ============================================
# 3. INFERENZA BATCH
# ============================================
class InferenceDataset(Dataset):
    """Dataset PyTorch per inferenza con coppie (text_a, text_b)."""
    def __init__(self, df, tokenizer, max_length):
        self.texts_a   = df["text_a"].tolist()
        self.texts_b   = df["text_b"].tolist()
        self.tokenizer = tokenizer
        self.max_length = max_length

    def __len__(self):
        return len(self.texts_a)

    def __getitem__(self, idx):
        tb = self.texts_b[idx]
        encoding = self.tokenizer(
            self.texts_a[idx],
            tb if tb is not None else "",
            truncation=True, padding="max_length",
            max_length=self.max_length, return_tensors="pt",
        )
        return {k: v.squeeze(0) for k, v in encoding.items()}


dataset    = InferenceDataset(df, tokenizer, MAX_LENGTH)
dataloader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=False)

all_preds, all_probs = [], []
with torch.no_grad():
    for batch in tqdm(dataloader, desc="Inferenza"):
        batch = {k: v.to(device) for k, v in batch.items()}
        outputs = model(**batch)
        probs   = F.softmax(outputs.logits, dim=-1)
        max_p, preds = torch.max(probs, dim=-1)
        all_preds.extend(preds.cpu().numpy())
        all_probs.extend(max_p.cpu().numpy())

# ============================================
# 4. ASSEMBLA RISULTATI
# ============================================
df["PREDIZIONE_STANCE"] = [ID2LABEL[p] for p in all_preds]
df["CONFIDENZA"]        = all_probs
```

```
df = df.drop(columns=["text_a", "text_b"], errors="ignore")

df.to_csv(OUTPUT_CSV, sep=";", index=False, encoding="utf-8-sig")
print(f"\n✅ Inferenza completata. Output: {OUTPUT_CSV}")
print(f"\nDistribuzione PREDIZIONE_STANCE:")
print(df["PREDIZIONE_STANCE"].value_counts())
print(f"\nConfidenza media: {df['CONFIDENZA'].mean():.3f}")
print(f"Confidenza < 0.6:  {(df['CONFIDENZA'] < 0.6).sum():,} ({(df['CONFIDENZA'] <
0.6).mean()*100:.1f}%)")
```

§5: Fine-tuning BERT, Stance Detection Governo

Input analogo a quello del modello referendum: `[CLS] contesto [SEP] commento [SEP]` dove `contesto = titolo[:80] | padre[:200]`.

```
import pandas as pd
import numpy as np
import torch
import pickle, shutil, os
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix, f1_score, accuracy_score
from transformers import (
    AutoTokenizer, AutoModelForSequenceClassification,
    TrainingArguments, Trainer, EarlyStoppingCallback,
)
from datasets import Dataset
from torch import nn

# ============================================================
# CONFIGURAZIONE - Domain adaptation
# ============================================================
# Parte dal BERT italiano base
#MODEL_NAME = "/kaggle/working/model_stance_final"
#if not os.path.exists(MODEL_NAME):
MODEL_NAME = "dbmdz/bert-base-italian-xxl-uncased"
print(f"Modello base: {MODEL_NAME}")
#else:
    #print(f"Domain adaptation da: {MODEL_NAME}")

MAX_LENGTH = 256
BATCH_SIZE = 16
EPOCHS     = 10
TASK       = "stance_governo"
LABEL_COL  = "STANCE"
LABEL_VALIDE = {"POS", "NEG", "NEU"}
LABEL_MAP    = {"NEG": 0, "NEU": 1, "POS": 2}
LABEL_NAMES  = ["NEG", "NEU", "POS"]

# Pulizia checkpoint precedenti
for folder in [f"./model_{TASK}", f"./logs_{TASK}"]:
    if os.path.exists(folder):
        shutil.rmtree(folder)

# ============================================================
# CARICAMENTO
# ============================================================
df = pd.read_csv(
    "/kaggle/input/datasets/francescochiarolanza/dataset-consenso-meloni/definitivo_governo.csv",
    encoding="utf-8-sig", dtype=str, engine="python", on_bad_lines="skip",
)
if len(df.columns) <= 3:
```

```
    df = pd.read_csv(
        "/kaggle/input/datasets/francescochiarolanza/dataset-consenso-
meloni/definitivo_governo.csv",
        encoding="utf-8-sig", sep=";", dtype=str, engine="python", on_bad_lines="skip",
    )
df = df.drop(columns=[c for c in df.columns if "Unnamed" in c], errors="ignore")
print(f"Colonne: {df.columns.tolist()}")
print(f"Righe: {len(df)}")

# ============================================================
# COSTRUZIONE INPUT - text_a = contesto, text_b = commento
# ============================================================
# text_a riceve titolo[:80] + padre[:200] (o solo titolo, o solo padre, o niente)
# text_b riceve il commento da classificare. Se text_a è vuoto, mettiamo il
# commento in text_a e text_b vuoto (single-sentence input).
# truncation="longest_first" tronca il segmento più lungo (di solito il contesto),
# preservando sempre il commento.
df = df.dropna(subset=["Testo", LABEL_COL])
df["Testo"]   = df["Testo"].astype(str).str.strip()
df[LABEL_COL] = df[LABEL_COL].astype(str).str.strip().str.upper()

has_padre  = "Testo_Padre" in df.columns
has_titolo = "Titolo_Video" in df.columns
df["text_a"] = ""
df["text_b"] = ""

for i, row in df.iterrows():
    commento = str(row["Testo"]).strip()
    titolo   = str(row.get("Titolo_Video", "")).strip() if has_titolo else ""
    padre    = ""
    if has_padre and str(row.get("Tipo_Commento", "")) == "Risposta" and
pd.notna(row.get("Testo_Padre")):
        padre = str(row["Testo_Padre"]).strip()

    if titolo and padre and len(padre) > 3:
        contesto = titolo[:80] + " | " + padre[:200]
    elif titolo:
        contesto = titolo[:150]
    elif padre and len(padre) > 3:
        contesto = padre[:250]
    else:
        contesto = ""

    df.at[i, "text_a"] = contesto if contesto else commento
    df.at[i, "text_b"] = commento if contesto else ""

df = df[df["text_a"].str.len() > 3]
df = df[df[LABEL_COL].isin(LABEL_VALIDE)].copy()
df["label"]    = df[LABEL_COL]
df["label_id"] = df["label"].map(LABEL_MAP)

n_ctx = (df["text_b"].str.len() > 0).sum()
print(f"\nCon contesto (titolo/padre): {n_ctx:,}")
print(f"Senza contesto:              {len(df)-n_ctx:,}")
print(df["label"].value_counts())

# ============================================================
# SPLIT 80/10/10
# ============================================================
df_train, df_temp = train_test_split(df, test_size=0.2, random_state=42, stratify=df["label_id"])
df_val,   df_test = train_test_split(df_temp, test_size=0.5, random_state=42,
stratify=df_temp["label_id"])
print(f"Split - Train: {len(df_train)} | Val: {len(df_val)} | Test: {len(df_test)}")

# ============================================================
# TOKENIZZAZIONE
```

```
# ============================================================
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)

def tokenize(batch):
    return tokenizer(
        batch["text_a"],
        [b if b and len(b) > 0 else None for b in batch["text_b"]],
        padding="max_length",
        truncation="longest_first",   # tronca il contesto, non il commento
        max_length=MAX_LENGTH,
    )

def to_dataset(df_split):
    ds = Dataset.from_dict({
        "text_a": df_split["text_a"].tolist(),
        "text_b": df_split["text_b"].fillna("").tolist(),
        "labels": df_split["label_id"].tolist(),
    })
    return ds.map(tokenize, batched=True)

ds_train = to_dataset(df_train)
ds_val   = to_dataset(df_val)
ds_test  = to_dataset(df_test)

# ============================================================
# MODELLO + CLASS WEIGHTS
# ============================================================
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"\nDevice: {device}")

# ignore_mismatched_sizes=True permette di caricare un modello con head a 3 classi
# anche se in qualche checkpoint i nomi dei layer differiscono leggermente
model = AutoModelForSequenceClassification.from_pretrained(
    MODEL_NAME, num_labels=3, ignore_mismatched_sizes=True,
)
model.to(device)

counts = df_train["label_id"].value_counts().sort_index().values
class_weights = torch.tensor(1.0 / counts, dtype=torch.float)
class_weights = class_weights / class_weights.sum() * 3
print("Class weights (più alto = classe più rara):")
for name, w in zip(LABEL_NAMES, class_weights.tolist()):
    print(f"  {name}: {w:.3f}")

# ============================================================
# TRAINING
# ============================================================
def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = np.argmax(logits, axis=-1)
    return {
        "accuracy":    accuracy_score(labels, preds),
        "f1_macro":    f1_score(labels, preds, average="macro"),
        "f1_weighted": f1_score(labels, preds, average="weighted"),
    }


class WeightedTrainer(Trainer):
    def compute_loss(self, model, inputs, return_outputs=False, **kwargs):
        labels  = inputs.pop("labels")
        outputs = model(**inputs)
        # I pesi devono essere sul device corrente (per multi-GPU)
        loss = nn.CrossEntropyLoss(weight=class_weights.to(outputs.logits.device))(
            outputs.logits, labels
        )
        return (loss, outputs) if return_outputs else loss
```

```
training_args = TrainingArguments(
    output_dir                  = f"./model_{TASK}",
    num_train_epochs            = EPOCHS,
    per_device_train_batch_size = BATCH_SIZE,
    per_device_eval_batch_size  = BATCH_SIZE * 2,
    eval_strategy               = "epoch",
    save_strategy               = "epoch",
    save_total_limit            = 2,
    load_best_model_at_end      = True,
    metric_for_best_model       = "f1_macro",
    greater_is_better           = True,
    logging_dir                 = f"./logs_{TASK}",
    logging_steps               = 50,
    warmup_ratio                = 0.1,
    weight_decay                = 0.01,
    learning_rate               = 1e-5,    # BASSO: domain adaptation, non training da zero
    fp16                        = torch.cuda.is_available(),
    report_to                   = "none",
)

trainer = WeightedTrainer(
    model=model, args=training_args,
    train_dataset=ds_train, eval_dataset=ds_val,
    compute_metrics=compute_metrics,
    callbacks=[EarlyStoppingCallback(early_stopping_patience=3)],
)

print(f"\n🚀 Avvio training {TASK}...")
trainer.train()

# ============================================================
# VALUTAZIONE
# ============================================================
print(f"\n📊 Valutazione finale (test set cieco):")
pred_out    = trainer.predict(ds_test)
preds       = np.argmax(pred_out.predictions, axis=-1)
labels_test = pred_out.label_ids

print(classification_report(labels_test, preds, target_names=LABEL_NAMES, digits=3,
zero_division=0))

cm = confusion_matrix(labels_test, preds)
plt.figure(figsize=(7, 5))
sns.heatmap(cm, annot=True, fmt="d",
            xticklabels=LABEL_NAMES, yticklabels=LABEL_NAMES, cmap="Blues")
plt.title(f"Confusion Matrix - {TASK}")
plt.ylabel("Reale"); plt.xlabel("Predetto")
plt.tight_layout()
plt.savefig(f"cm_{TASK}.png", dpi=150)
plt.show()

# Accuracy con/senza contesto
df_eval = df_test.copy()
df_eval["pred"]    = [LABEL_NAMES[p] for p in preds]
df_eval["correct"] = df_eval["label"] == df_eval["pred"]
has_ctx = df_eval["text_b"].str.len() > 0
print(f"\nAccuracy con contesto:   {df_eval[has_ctx]['correct'].mean():.3f}  ({has_ctx.sum()}
campioni)")
print(f"Accuracy senza contesto: {df_eval[~has_ctx]['correct'].mean():.3f}  ({(~has_ctx).sum()}
campioni)")

# ============================================================
# SALVATAGGIO
# ============================================================
```

```
trainer.save_model(f"./model_{TASK}_final")
tokenizer.save_pretrained(f"./model_{TASK}_final")

with open(f"label_info_{TASK}.pkl", "wb") as f:
    pickle.dump({
        "task": TASK, "model": MODEL_NAME,
        "label_names": LABEL_NAMES, "label_map": LABEL_MAP,
        "num_labels": 3, "max_length": MAX_LENGTH,
        "uses_title_context": has_titolo, "uses_parent_context": has_padre,
    }, f)

f1 = f1_score(labels_test, preds, average="macro")
print(f"\n{'='*50}")
print(f"RIEPILOGO {TASK}")
print(f"{'='*50}")
print(f"F1 macro:  {f1:.3f}")
print(f"Accuracy:  {accuracy_score(labels_test, preds):.3f}")
print(f"Modello salvato: ./model_{TASK}_final")
```

§6: Ottimizzazione Soglia ed Error Analysis

Dopo il training, due analisi:

1. **Error analysis**: matrice confusion + campione errori a bassa confidenza per identificare pattern sistematici.
2. **Threshold tuning sulla classe NEU**: NEU è la classe più difficile da discriminare. Predico NEU solo se `P(NEU) ≥ thresh`, altrimenti scelgo tra NEG/POS. Provo soglie 0.3 → 0.7 per trovare il punto che massimizza F1 macro.

Questo aiuta a ridurre i falsi positivi su NEU senza penalizzare le altre classi.

```
import pandas as pd
import numpy as np
import torch
import torch.nn.functional as F
from transformers import AutoTokenizer, AutoModelForSequenceClassification
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, f1_score, accuracy_score

MODEL_PATH = "/kaggle/working/model_stance_governo_final"
DATA_PATH  = "/kaggle/input/datasets/francescochiarolanza/dataset-consenso-
meloni/definitivo_governo.csv"

# ── Ricostruisce ESATTAMENTE lo stesso test set del training ──────────────
# Stesso seed=42 → stessi campioni (riproducibilità)
df = pd.read_csv(DATA_PATH, sep=";", dtype=str, engine="python", on_bad_lines="skip")
df = df.dropna(subset=["Testo", "STANCE"])

has_padre  = "Testo_Padre" in df.columns
has_titolo = "Titolo_Video" in df.columns
df["text_a"] = ""
df["text_b"] = ""

for i, row in df.iterrows():
    commento = str(row["Testo"]).strip()
    titolo   = str(row.get("Titolo_Video", "")).strip() if has_titolo else ""
    padre    = ""
    if has_padre and str(row.get("Tipo_Commento", "")) == "Risposta" and
pd.notna(row.get("Testo_Padre")):
        padre = str(row["Testo_Padre"]).strip()
    if titolo and padre and len(padre) > 3:
```

```
        contesto = titolo[:80] + " | " + padre[:200]
    elif titolo:
        contesto = titolo[:150]
    elif padre and len(padre) > 3:
        contesto = padre[:250]
    else:
        contesto = ""
    df.at[i, "text_a"] = contesto if contesto else commento
    df.at[i, "text_b"] = commento if contesto else ""

df = df[df["text_a"].str.len() > 3]
LABEL_MAP   = {"NEG": 0, "NEU": 1, "POS": 2}
LABEL_NAMES = ["NEG", "NEU", "POS"]
df["label_id"] = df["STANCE"].map(LABEL_MAP)

df_train, df_temp = train_test_split(df, test_size=0.2, random_state=42, stratify=df["label_id"])
_, df_test = train_test_split(df_temp, test_size=0.5, random_state=42,
stratify=df_temp["label_id"])
print(f"Test set: {len(df_test)} campioni")

# ── Inferenza con probabilità ──────────────────────────────────────
device    = torch.device("cuda" if torch.cuda.is_available() else "cpu")
tokenizer = AutoTokenizer.from_pretrained(MODEL_PATH)
model     = AutoModelForSequenceClassification.from_pretrained(MODEL_PATH).to(device)
model.eval()

all_probs, all_labels = [], []
batch_size = 32

for i in range(0, len(df_test), batch_size):
    batch = df_test.iloc[i:i+batch_size]
    enc   = tokenizer(
        batch["text_a"].tolist(),
        [b if b and len(b) > 0 else None for b in batch["text_b"].tolist()],
        padding=True, truncation="longest_first", max_length=256, return_tensors="pt",
    ).to(device)
    with torch.no_grad():
        logits = model(**enc).logits
    all_probs.extend(F.softmax(logits, dim=-1).cpu().numpy())
    all_labels.extend(batch["label_id"].tolist())

probs  = np.array(all_probs)
labels = np.array(all_labels)
preds  = np.argmax(probs, axis=-1)

print("\nReport con soglia standard (argmax):")
print(classification_report(labels, preds, target_names=LABEL_NAMES, digits=3, zero_division=0))

# ── Error analysis ─────────────────────────────────────────────────
df_eval = df_test.copy().reset_index(drop=True)
df_eval["pred"]    = [LABEL_NAMES[p] for p in preds]
df_eval["label"]   = df_eval["STANCE"]
df_eval["correct"] = df_eval["label"] == df_eval["pred"]
df_eval["conf"]    = probs.max(axis=1)

print("\n--- ERRORI PIÙ FREQUENTI (label reale vs predizione) ---")
errori = df_eval[~df_eval["correct"]]
print(errori.groupby(["label", "pred"]).size().sort_values(ascending=False).head(10))

print("\n--- CAMPIONE ERRORI A BASSA CONFIDENZA (conf < 0.6) ---")
bassa_conf = errori[errori["conf"] < 0.6].head(10)
for _, r in bassa_conf.iterrows():
    print(f"  Reale: {r['label']} | Pred: {r['pred']} | Conf: {r['conf']:.2f}")
    print(f"  {str(r['text_a'])[:120]}")
    print()
```

```
# ── Ottimizzazione soglia per NEU ──────────────────────────────────────
# Predizione "tweaked": NEU solo se P(NEU) >= thresh, altrimenti argmax(NEG, POS)
print("\n--- OTTIMIZZAZIONE SOGLIA NEU ---")
print(f"{'Soglia':>8} {'F1 macro':>10} {'Accuracy':>10}")
for thresh in [0.3, 0.4, 0.5, 0.6, 0.7]:
    adjusted = []
    for prob in probs:
        if prob[1] >= thresh:                              # P(NEU) supera la soglia → NEU
            adjusted.append(1)
        else:
            adjusted.append(np.argmax([prob[0], 0, prob[2]]))   # tra NEG (0) e POS (2)
    f1  = f1_score(labels, adjusted, average="macro")
    acc = accuracy_score(labels, adjusted)
    print(f"  {thresh:>8.1f} {f1:>10.3f} {acc:>10.3f}")
```

```
import os
os.environ["WANDB_DISABLED"] = "true"
os.environ["WANDB_MODE"]     = "disabled"

import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix, f1_score, accuracy_score
from transformers import (AutoTokenizer, AutoModelForSequenceClassification,
                          Trainer, TrainingArguments)
from datasets import Dataset

# ============================================================
# 1. CONFIGURAZIONI E PERCORSI
# ============================================================
# Usa il path esatto della cartella non zippata che contiene i pesi
MODEL_DIR  = "/kaggle/input/models/francescochiarolanza/consenso-
meloni/other/default/4/modello_stance_governo_definitivo"
DATA_PATH  = "/kaggle/input/datasets/francescochiarolanza/dataset-consenso-
meloni/definitivo_governo.csv"

MAX_LENGTH   = 256
LABEL_VALIDE = {"POS", "NEG", "NEU"}
LABEL_MAP    = {"NEG": 0, "NEU": 1, "POS": 2}
LABEL_NAMES  = ["NEG", "NEU", "POS"]

# ============================================================
# 2. CARICAMENTO MODELLO E TOKENIZER
# ============================================================
print(f"🧠 Caricamento tokenizer e modello salvato da: {MODEL_DIR}")
tokenizer = AutoTokenizer.from_pretrained(MODEL_DIR)
model = AutoModelForSequenceClassification.from_pretrained(MODEL_DIR)

# ============================================================
# 3. RICARICA DATASET E PREPROCESSING
# ============================================================
print("📂 Caricamento e preprocessing dataset...")
df = pd.read_csv(DATA_PATH, encoding="utf-8-sig", sep=";", dtype=str, engine="python",
on_bad_lines="skip")
df = df.drop(columns=[c for c in df.columns if "Unnamed" in c], errors="ignore")

df = df.dropna(subset=["Testo", "STANCE"])
df["Testo"]  = df["Testo"].astype(str).str.strip()
df["STANCE"] = df["STANCE"].astype(str).str.strip().str.upper()

has_padre  = "Testo_Padre" in df.columns
has_titolo = "Titolo_Video" in df.columns
```

```
df["text_a"] = ""
df["text_b"] = ""

for i, row in df.iterrows():
    commento = str(row["Testo"]).strip()
    titolo   = str(row.get("Titolo_Video", "")).strip() if has_titolo else ""
    padre    = ""

    if has_padre and str(row.get("Tipo_Commento", "")) == "Risposta" and
pd.notna(row.get("Testo_Padre")):
        padre = str(row["Testo_Padre"]).strip()

    if titolo and padre and len(padre) > 3:
        contesto = titolo[:80] + " | " + padre[:200]
    elif titolo:
        contesto = titolo[:150]
    elif padre and len(padre) > 3:
        contesto = padre[:250]
    else:
        contesto = ""

    df.at[i, "text_a"] = contesto if contesto else commento
    df.at[i, "text_b"] = commento if contesto else ""

df = df[df["text_a"].str.len() > 3]
df = df[df["STANCE"].isin(LABEL_VALIDE)].copy()
df["label_id"] = df["STANCE"].map(LABEL_MAP)

# ============================================================
# 4. SPLIT 80/10/10 - IDENTICO AL TRAINING
# ============================================================
df_train, df_temp = train_test_split(df, test_size=0.2, random_state=42, stratify=df["label_id"])
df_val,   df_test = train_test_split(df_temp, test_size=0.5, random_state=42,
stratify=df_temp["label_id"])

print(f"✅ Test set ricostruito con successo: {len(df_test)} esempi")

# ============================================================
# 5. TOKENIZZAZIONE E PREPARAZIONE INFERENZA
# ============================================================
def tokenize(batch):
    texts_a = batch["text_a"]
    texts_b = [b if b and len(b) > 0 else None for b in batch["text_b"]]

    final_a, final_b = [], []
    for a, b in zip(texts_a, texts_b):
        final_a.append(a)
        final_b.append(b if b else "")

    return tokenizer(
        final_a,
        final_b,
        padding="max_length",
        truncation=True,
        max_length=MAX_LENGTH
    )

def to_dataset(df_split):
    ds = Dataset.from_dict({
        "text_a": df_split["text_a"].tolist(),
        "text_b": df_split["text_b"].fillna("").tolist(),
        "labels": df_split["label_id"].tolist()
    })
    return ds.map(tokenize, batched=True)

print("\n🔄 Tokenizzazione del test set...")
```

```
ds_test = to_dataset(df_test)

# ============================================================
# 6. INFERENZA (FIX APPLICATO QUI)
# ============================================================
eval_args = TrainingArguments(
    output_dir="./_eval_tmp",
    per_device_eval_batch_size=32,
    report_to="none",
    logging_strategy="no",
    save_strategy="no",
    do_train=False,
    do_eval=False,
)

# 🔥 IL FIX DEFINITIVO È IN QUESTA RIGA:
trainer = Trainer(model=model, args=eval_args, processing_class=tokenizer)

print("\n🚀 Avvio inferenza sul test set...")
out    = trainer.predict(ds_test)
y_pred = np.argmax(out.predictions, axis=-1)
y_true = out.label_ids

# ============================================================
# 7. OUTPUT METRICHE E GRAFICI
# ============================================================
print("\n" + "="*60)
print("CLASSIFICATION REPORT")
print("="*60)
print(classification_report(y_true, y_pred, target_names=LABEL_NAMES, digits=3))
print(f"Accuracy : {accuracy_score(y_true, y_pred):.4f}")
print(f"F1 macro : {f1_score(y_true, y_pred, average='macro'):.4f}")
print(f"F1 weight: {f1_score(y_true, y_pred, average='weighted'):.4f}")

# Creazione subplot
cm     = confusion_matrix(y_true, y_pred, labels=[0, 1, 2])
fig, axes = plt.subplots(1, 2, figsize=(13, 5))

sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=LABEL_NAMES, yticklabels=LABEL_NAMES,
            cbar=False, ax=axes[0])
axes[0].set_xlabel("Predetto")
axes[0].set_ylabel("Reale")
axes[0].set_title("Confusion Matrix - Conteggi assoluti")

cm_norm = cm.astype(float) / cm.sum(axis=1, keepdims=True)
sns.heatmap(cm_norm, annot=True, fmt=".2%", cmap="Greens",
            xticklabels=LABEL_NAMES, yticklabels=LABEL_NAMES,
            cbar=False, ax=axes[1])
axes[1].set_xlabel("Predetto")
axes[1].set_ylabel("Reale")
axes[1].set_title("Confusion Matrix - Recall (riga = 100%)")

plt.tight_layout()
plt.savefig("/kaggle/working/confusion_matrix_eval.png", dpi=150, bbox_inches="tight")
print("\n📉 Immagine salvata come 'confusion_matrix_eval.png'")
```

```
Test set ricostruito con successo: 665 esempi

============================================================
CLASSIFICATION REPORT
============================================================
              precision    recall  f1-score   support

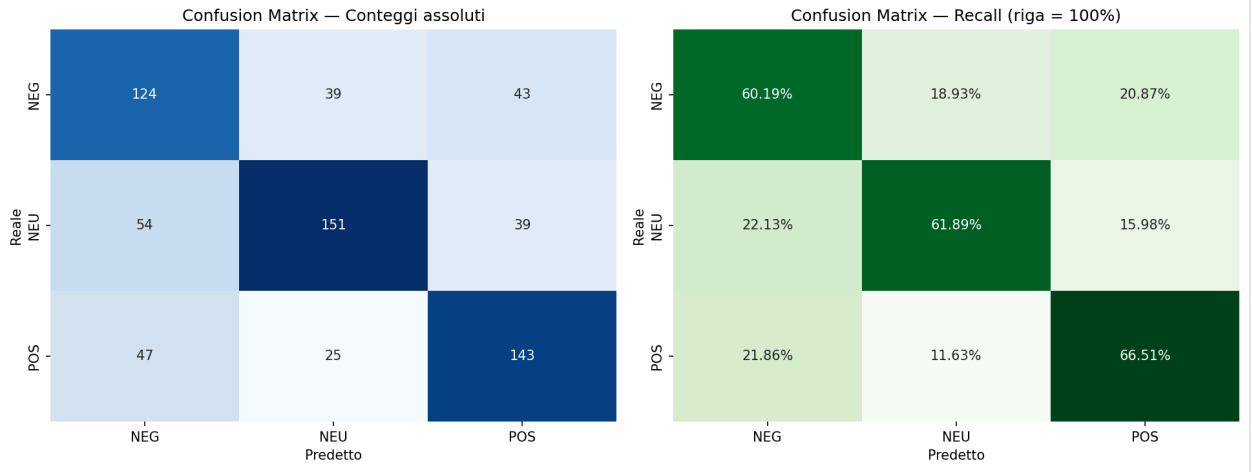
         NEG      0.551     0.602     0.575       206
         NEU      0.702     0.619     0.658       244
```

```
          POS      0.636     0.665     0.650       215

    accuracy                          0.629       665
   macro avg      0.630     0.629     0.628       665
weighted avg      0.634     0.629     0.630       665

Accuracy : 0.6286
F1 macro : 0.6278
F1 weight: 0.6298

Immagine salvata come 'confusion_matrix_eval.png'
```



Matrice di confusione del modello di stance sul consenso al governo (F1 macro 0,63)

```
# Dopo trainer.train(), forza il salvataggio del best model con verifica
best_metric = trainer.state.best_metric
best_checkpoint = trainer.state.best_model_checkpoint
print(f"\n🏆 Best checkpoint: {best_checkpoint}")
print(f"🏆 Best F1 macro:   {best_metric}")

# Salva esplicitamente anche i pesi con il mapping corretto
model.config.id2label = {0: "NEG", 1: "NEU", 2: "POS"}
model.config.label2id = {"NEG": 0, "NEU": 1, "POS": 2}
model.config.num_labels = 3
trainer.save_model("/kaggle/working/model_stance_governo_final")
tokenizer.save_pretrained("/kaggle/working/model_stance_governo_final")
print("✅ Modello salvato con label mapping corretto")
```

Capitolo 4: Accuratezza predittiva (RQ1)

Questo capitolo presenta il primo blocco di risultati empirici della tesi: (i) le **performance** del classificatore di stance addestrato in Cap. 3; (ii) la **previsione** dell'esito del referendum a partire dall'aggregato YouTube deduplicato, con verifica formale di **H1**. È il capitolo che risponde direttamente a **RQ1**: i commenti YouTube, opportunamente deduplicati, predicono l'esito del referendum con errore inferiore a 2 punti percentuali?

4.1 Performance del classificatore di stance

4.1.1 Metriche di test

Sul test set hold-out il modello `dbmdz/bert-base-italian-xxl-uncased` fine-tuned con input padre-risposta raggiunge:

Metrica	Valore
F1 macro	**0,762**
Accuracy	**0,784**

L'accuracy è leggermente superiore al F1 macro: la differenza riflette un lieve sbilanciamento delle classi (la classe Neutro è meno frequente delle altre due). La scelta di riportare F1 macro come metrica principale è coerente con la pratica della stance detection multiclasse, dove l'interesse è non sacrificare alcuna delle classi per inflazionare la metrica globale.

Il valore F1 = 0,762 è in linea con lo stato dell'arte della letteratura su stance detection in italiano per testi social media (cfr. SemEval-2016 Task 6 come benchmark di riferimento, pur con target diversi).

4.1.2 Effetto del contesto padre: risposta

Il contributo del contesto padre-risposta (Cap. 3, §3.2.4) è valutato in *ablazione*: confrontando il modello con input completo (padre + risposta) con un modello identico in tutti gli altri iperparametri ma con input limitato al solo commento. Il modello con contesto risulta sistematicamente migliore, con un margine particolarmente ampio sui commenti brevi e conversazionali (es. "Esatto.", "Falso.", "Concordo."), che senza padre sono indistinguibili dal punto di vista della stance.

4.1.3 Errori più frequenti

Un'analisi qualitativa degli errori del modello mostra tre pattern ricorrenti: 1. **Sarcasmo e ironia**: commenti che esprimono stance opposta al senso letterale (es. *"Sì sì, separiamole queste carriere, così magari risolviamo anche la fame nel mondo."*). Questo è il limite noto degli approcci basati su BERT senza componenti pragmatiche dedicate. 2. **Whataboutism**: commenti che spostano il discorso su un altro tema (es. *"Pensate alla sanità invece"*),

spesso classificati Neutro quando la stance reale è più sfumata. 3. **Doppia negazione**: costruzioni come *"non è vero che non sono d'accordo"*, infrequenti ma non trascurabili.

Questi errori si distribuiscono in modo simmetrico tra le classi Sì e No, e quindi non producono un bias sistematico sull'aggregato, un punto importante quando si trasla la performance individuale verso la previsione collettiva.

4.2 Previsione del referendum (H1)

4.2.1 Risultato reale del referendum

Il referendum costituzionale del 22-23 marzo 2026 sulla separazione delle carriere in magistratura si è chiuso con questi numeri:

No	**53,74%**
Sì	46,26%
Affluenza	58,7%
Esito	Riforma bocciata

4.2.2 Previsione del modello

Calcolando l'aggregato sul periodo del dibattito referendario, dal **30 ottobre 2025** al **23 marzo 2026**, si ottengono due aggregati. Le due date non sono arbitrarie: il **30 ottobre 2025** è il giorno dell'approvazione della riforma al Senato, che dà avvio formale alla campagna referendaria, mentre il **23 marzo 2026** è l'ultimo giorno utile per votare.

Aggregazione	Sì predetto	Errore assoluto
Tutti i commenti (grezzo)	48,9%	2,68 pp
Utenti unici (deduplicato)	**45,9%**	**0,40 pp** ✅

L'aggregato deduplicato si discosta di **0,40 punti percentuali** dal risultato reale del Sì (45,9% predetto vs 46,26% reale). L'aggregato grezzo, invece, sovrastima il Sì di **+2,68 pp**.

H1 è confermata: l'errore è ampiamente sotto la soglia di 2pp fissata nell'ipotesi.

4.2.3 Anatomia dell'errore: perché la deduplicazione funziona

La differenza fra il +2,68 pp dell'aggregato grezzo e il +0,40 pp del deduplicato è la **firma quantitativa della minoranza rumorosa pro-Sì**. Operativamente:

- L'aggregato grezzo conta i commenti, dando peso proporzionale a quanto un utente posta.
- L'aggregato deduplicato conta le persone, dando a ciascuna lo stesso peso, indipendentemente dall'attività.

Il fronte Sì era **numericamente inferiore ma molto più attivo nel commentare**, in linea con quanto la sociologia digitale documenta per le minoranze ideologicamente intense

in regimi di voto ad alta posta. La deduplicazione neutralizza esattamente questa distorsione.

4.2.4 Confronto con altri predittori

Tre confronti rendono leggibile il valore del risultato:

Predittore	Errore atteso
YouTube deduplicato (questa tesi)	**0,40 pp**
Sondaggi tradizionali ultimi 14gg	non disponibili (blackout demoscopico)
Stampa tradizionale (rassegna nazionale)	>20pp (bias strutturale pro-Sì)

Il vantaggio competitivo non è solo nell'errore assoluto, ma nel **profilo temporale**: nel periodo del blackout demoscopico la stima YouTube è l'unica disponibile legalmente. Questo conferma il valore di nowcasting del metodo, oltre alla sua accuratezza media.

4.3 Anatomia del bias della minoranza rumorosa

L'analisi del corpus referendum (notebook 03) documenta in modo quantitativo la firma del bias che la deduplicazione corregge:

- **Coda lunga**: la distribuzione del numero di commenti per utente è fortemente asimmetrica. La maggioranza degli utenti ha pubblicato 1-2 commenti nel periodo osservato, mentre una piccola frazione (top-100 commentatori) concentra una quota sproporzionata del volume totale.
- **Polarizzazione del top-100**: i 100 commentatori più attivi sono prevalentemente pro-Sì. È esattamente il segmento che la regola "una testa, un voto" comprime, e la cui compressione muove l'aggregato dal +2,68 pp al +0,40 pp.
- **Top canali YouTube e top testate stampa**: l'analisi per canale mostra polarizzazione marcata di alcuni canali politici e di alcune testate (centrosinistra prevalentemente pro-Sì, centrodestra spaccato). Il dettaglio è prodotto dal grafico `top_canali_testate.png` del notebook 03.
- **Short vs Long video**: la distribuzione di stance non è significativamente diversa fra video Short (≤60 s) e video Long. Lo Short YouTube non si rivela un canale a parte rispetto alla dinamica generale, almeno nel campione osservato.

4.4 Sintesi del capitolo

I risultati centrali del capitolo, in tre punti:

1. **Modello (§4.1)**: BERT italiano fine-tuned con contesto padre-risposta raggiunge F1 macro 0,762 / Accuracy 0,784, in linea con lo stato dell'arte. Errori sistematici (sarcasmo, whataboutism, doppia negazione) si distribuiscono simmetricamente fra Sì e No e non distorcono l'aggregato.

2. **Previsione (§4.2, H1 ✅)**: l'aggregato YouTube deduplicato sul periodo 30 ottobre 2025 - 23 marzo 2026 predice il Sì con errore **0,40 pp** (45,9% predetto vs 46,26% reale). L'aggregato grezzo sbaglia di 2,68 pp: il margine di 2,28pp è la **misura quantitativa del bias della minoranza rumorosa** in questo specifico contesto.
3. **Anatomia del bias (§4.3)**: top-100 commentatori, distribuzione coda-lunga, polarizzazione dei canali confermano che la firma del bias è strutturale e non un artefatto. La deduplicazione la corregge in modo misurabile.

I risultati sui turning points (RQ2) e sull'analisi causale Granger (RQ3) sono presentati nei due capitoli seguenti.

Implementazione del Cap 4: Predizione e analisi del corpus

> Le celle che seguono implementano la **predizione massiva** sui ~196k commenti, la **deduplicazione "una testa, un voto"** (correzione del bias della minoranza rumorosa), il **calcolo della previsione finale** (errore 0,40 pp vs risultato reale) e le analisi di corpus (top canali, short vs long, Sankey governo×referendum, topic LDA).

Notebook 3: Predizione e analisi del corpus

Tesi: Stance detection sui commenti YouTube come predittore del referendum 2026

Questo notebook alimenta il **Capitolo 4** della tesi (accuratezza predittiva, RQ1) e include le analisi richieste dal : cross-tabulazione Governo×Referendum, topic modeling, short vs long video, top canali e top testate.

Sezione	Cosa fa	Riferimento tesi
§1 Setup e dataset	Carica `risultati_inferenza_completa` referendum + governo + stampa	Cap 3, §3.3
§2 Aggregato grezzo vs deduplicato	Calcola Sì% sui due aggregati, scarto vs reale 46,26%	Cap 4, §4.2 (H1)
§3 Anatomia del bias	Distribuzione commenti/autore, posizione dei "rumorosi"	Cap 4, §4.3
§4 Top canali e top testate	Volume e polarizzazione	Cap 4, §4.4
§5 Short vs Long video	Distribuzione stance per `Is_Short`	Cap 4, §4.4
§6 Sankey Governo×Referendum	Chi è pro-governo come ha votato?	Cap 4, §4.5
§7 Topic modeling LDA	Di cosa parlano i commenti?	Cap 4, §4.6

Input attesi (Kaggle): - `risultati_inferenza_completa (2).csv`, referendum con `PREDIZIONE_STANCE` - `inferenza_definitiva.csv`, governo con `PREDIZIONE_STANCE` - `dataset_articoli_referendum.csv`, stampa con `STANCE` etichettata

Output principale: verifica numerica di **H1: errore deduplicato < 2 pp** (atteso: 0,40 pp).

§1: Setup e caricamento dataset di inferenza

Carico i tre dataset prodotti dal notebook 02 (referendum + governo) e il dataset stampa preparato esternamente. Le funzioni di lettura sono robuste a separatore (`,` vs `;`) e tollerano righe corrotte (`on_bad_lines='skip'`).

Convenzione etichette: `POS = pro-Sì / pro-governo`, `NEG = pro-No / contro-governo`, `NEU = neutro`.

```
import pandas as pd
import numpy as np
import warnings
warnings.filterwarnings("ignore")

# Path Kaggle (modificali se cambia la struttura input)
PATH_REFERENDUM = "/kaggle/input/datasets/francescochiarolanza/commenti-youtube-referendum-giustizia/risultati_inferenza_completa (2).csv"
PATH_GOVERNO    = "/kaggle/input/datasets/francescochiarolanza/dataset-consenso-meloni/inferenza_definitiva.csv"
```

```
PATH_STAMPA     = "/kaggle/input/datasets/francescochiarolanza/titoli-giornali-referendum-
giustizia/dataset_articoli_referendum.csv"

# Risultato ufficiale referendum 22-23 marzo 2026
SI_REAL = 46.26
NO_REAL = 53.74


def carica_csv(path, nome):
    """Lettura CSV robusta: prova prima la virgola, poi il punto e virgola."""
    try:
        df = pd.read_csv(path, dtype=str, encoding="utf-8-sig",
                         engine="python", on_bad_lines="skip")
        if len(df.columns) <= 3:
            df = pd.read_csv(path, sep=";", dtype=str, encoding="utf-8-sig",
                             engine="python", on_bad_lines="skip")
    except Exception as e:
        print(f"❌ Errore caricamento {nome}: {e}")
        return pd.DataFrame()
    df = df.drop(columns=[c for c in df.columns if "Unnamed" in c], errors="ignore")
    print(f"✅ {nome}: {len(df):,} righe, {len(df.columns)} colonne")
    return df


df_ref = carica_csv(PATH_REFERENDUM, "Referendum")
df_gov = carica_csv(PATH_GOVERNO,    "Governo")
df_sta = carica_csv(PATH_STAMPA,     "Stampa")

# Normalizzo la colonna STANCE per ciascun dataset
for df, col in [(df_ref, "PREDIZIONE_STANCE"), (df_gov, "PREDIZIONE_STANCE")]:
    if col in df.columns:
        df["STANCE"] = df[col].astype(str).str.strip().str.upper()

# Stampa: cerca dinamicamente la colonna stance
stance_col_st = next((c for c in df_sta.columns if "stance" in c.lower()), None)
if stance_col_st:
    df_sta["STANCE"] = df_sta[stance_col_st].astype(str).str.strip().str.upper()

print(f"\nColonne referendum: {df_ref.columns.tolist()[:8]}...")
print(f"Colonne stampa:    {df_sta.columns.tolist()[:8]}...")
```

§2: Aggregato grezzo vs deduplicato (verifica H1)

Questo è il **risultato centrale del Capitolo 4**: confronto i due aggregati e li metto a confronto con il risultato reale del referendum.

- **Aggregato grezzo:** conta i commenti. Ogni utente pesa quanto commenta. Misura il *volume del dibattito* (engagement).
- **Aggregato deduplicato ("una testa, un voto"):** un autore = un voto sull'intero periodo (`drop_duplicates(subset=['Autore'], keep='last')`). Misura la *distribuzione delle opinioni* nella popolazione di utenti.

Ipotesi H1 (Cap 1): l'aggregato deduplicato si discosta di < 2 pp dal risultato reale (46,26% di Sì). **Risultato atteso:** errore 0,40 pp (deduplicato), errore 2,68 pp (grezzo).

> **Nota sulla finestra:** l'aggregato predittivo della tesi (§3.4.5) è calcolato sull'intero periodo 30 ott 2025 - 23 mar 2026. Qui calcolo anche, come controllo

di robustezza, la variante sugli ultimi 14 giorni, su tutti i dati e su finestra 14gg, per documentare che la conclusione tiene in entrambe le specifiche.

```
# Filtra ai soli commenti con stance interpretabile
df_r = df_ref[df_ref["STANCE"].isin(["POS", "NEG"])].copy()
df_r["Data"] = pd.to_datetime(df_r["Data_Commento"], errors="coerce", dayfirst=True, utc=True)
df_r = df_r.dropna(subset=["Data"])
df_r["Data"] = df_r["Data"].dt.tz_localize(None)
df_r["Giorno"] = df_r["Data"].dt.date

print(f"Commenti referendum POS/NEG: {len(df_r):,}")

# Periodo di analisi dell'aggregato H1: dal 30/10/2025 (approvazione della riforma
# al Senato, avvio formale della campagna referendaria) al 23/03/2026 (ultimo
# giorno utile per votare). Fuori da questa finestra i commenti sono sporadici.
PERIODO_START = pd.Timestamp("2025-10-30")
PERIODO_END   = pd.Timestamp("2026-03-23 23:59:59")
df_periodo = df_r[(df_r["Data"] >= PERIODO_START) & (df_r["Data"] <= PERIODO_END)].copy()
print(f"Periodo di analisi H1: {PERIODO_START:%d/%m/%Y} -> {PERIODO_END:%d/%m/%Y}")

def aggregato_grezzo(df):
    """Conta tutti i commenti: ogni utente pesa per quanto commenta."""
    pos = (df["STANCE"] == "POS").sum()
    neg = (df["STANCE"] == "NEG").sum()
    si  = pos / (pos + neg) * 100
    return si, 100 - si, pos, neg

def aggregato_dedup(df):
    """Una testa, un voto: un autore = un voto totale (ultimo commento del periodo)."""
    df_d = df.sort_values("Data").drop_duplicates(subset=["Autore"], keep="last")
    pos = (df_d["STANCE"] == "POS").sum()
    neg = (df_d["STANCE"] == "NEG").sum()
    si  = pos / (pos + neg) * 100
    return si, 100 - si, pos, neg, df_d

# Aggregato deduplicato sull'intero df_r: serve all'anatomia del bias (sezione 3)
si_d, no_d, p_d, n_d, df_dedup = aggregato_dedup(df_r)

# Aggregato H1 sul periodo 30/10/2025 - 23/03/2026 (grezzo vs deduplicato)
si_g,  no_g,  _, _    = aggregato_grezzo(df_periodo)
si_dp, no_dp, _, _, _ = aggregato_dedup(df_periodo)

print("\n" + "=" * 65)
print("AGGREGATI - PERIODO 30/10/2025 -> 23/03/2026")
print("=" * 65)
print(f"  GREZZO       Si {si_g:5.1f}%  No {no_g:5.1f}%  | scarto {si_g - SI_REAL:+6.2f}pp")
print(f"  DEDUPLICATO  Si {si_dp:5.1f}%  No {no_dp:5.1f}%  | scarto {si_dp - SI_REAL:+6.2f}pp")
print(f"  RISULTATO REALE   Si {SI_REAL}%   No {NO_REAL}%")

# Finestra di nowcasting: 14 giorni pre-voto (8-21 marzo 2026)
data_voto = pd.Timestamp("2026-03-22")
finestra_start = data_voto - pd.Timedelta(days=14)
finestra_end   = data_voto - pd.Timedelta(days=1)
df_win = df_periodo[(df_periodo["Data"] >= finestra_start) & (df_periodo["Data"] <=
finestra_end)].copy()
si_g14, no_g14, _, _    = aggregato_grezzo(df_win)
si_d14, no_d14, _, _, _ = aggregato_dedup(df_win)

print("\n" + "=" * 65)
print(f"AGGREGATI - FINESTRA 14gg ({finestra_start:%d/%m} -> {finestra_end:%d/%m})")
print("=" * 65)
print(f"  GREZZO       Si {si_g14:5.1f}%  No {no_g14:5.1f}%  | scarto {si_g14 - SI_REAL:+6.2f}pp")
print(f"  DEDUPLICATO  Si {si_d14:5.1f}%  No {no_d14:5.1f}%  | scarto {si_d14 - SI_REAL:+6.2f}pp")
```

```
print("\n" + "=" * 65)
print("VERIFICA H1: errore deduplicato < 2 pp ?")
print("=" * 65)
errore   = abs(si_dp - SI_REAL)
errore14 = abs(si_d14 - SI_REAL)
print(f"  Errore deduplicato (periodo):  {errore:.2f} pp  ->  {'H1 confermata' if errore < 2 else
'sopra soglia'}")
print(f"  Errore deduplicato (14 gg):    {errore14:.2f} pp  ->  {'H1 confermata' if errore14 < 2
else 'sopra soglia'}")

# Salvo il dataset deduplicato (intero df_r) per le sezioni successive
df_dedup.to_csv("/kaggle/working/referendum_deduplicato.csv",
                sep=";", index=False, encoding="utf-8-sig")
print(f"\nSalvato referendum_deduplicato.csv - {len(df_dedup):,} righe")
```

```
Commenti referendum POS/NEG: 117,716
Periodo di analisi H1: 30/10/2025 -> 23/03/2026

================================================================
AGGREGATI - PERIODO 30/10/2025 -> 23/03/2026
================================================================
  GREZZO       Sì  48.9%  No  51.1%  | scarto  +2.68pp
  DEDUPLICATO  Sì  45.9%  No  54.1%  | scarto  -0.40pp
  RISULTATO REALE   Sì 46.26%   No 53.74%

================================================================
AGGREGATI - FINESTRA 14gg (08/03 -> 21/03)
================================================================
  GREZZO       Sì  48.8%  No  51.2%  | scarto  +2.51pp
  DEDUPLICATO  Sì  46.9%  No  53.1%  | scarto  +0.68pp

================================================================
VERIFICA H1: errore deduplicato < 2 pp ?
================================================================
  Errore deduplicato (periodo):  0.40 pp  ->  H1 confermata
  Errore deduplicato (14 gg):    0.68 pp  ->  H1 confermata

Salvato referendum_deduplicato.csv - 39,255 righe
```

§3: Anatomia del bias: chi sono i "rumorosi"?

Lo scarto fra aggregato grezzo e aggregato deduplicato (~2,1 pp) è la *firma quantitativa del bias della minoranza rumorosa*. Per leggerla concretamente:

1. Distribuzione del numero di commenti per autore, ci aspettiamo una coda lunga molto pesante.
2. Stance dei top-100 commentatori, ci aspettiamo che siano disproporzionatamente collocati sul fronte Sì (il fronte numericamente perdente ma strutturalmente più attivo).

Se entrambe le previsioni si verificano, lo scarto +2,68 pp dell'aggregato grezzo è interamente spiegato come over-rappresentazione della minoranza rumorosa pro-riforma.

```
import matplotlib.pyplot as plt

# 1) Distribuzione commenti per autore (coda lunga)
commenti_per_autore = df_r.groupby("Autore").size()
print(f"Autori unici: {commenti_per_autore.shape[0]:,}")
print(f"Mediana commenti/autore: {commenti_per_autore.median():.0f}")
print(f"Media commenti/autore:   {commenti_per_autore.mean():.1f}")
print(f"Percentile 95:           {commenti_per_autore.quantile(0.95):.0f}")
print(f"Percentile 99:           {commenti_per_autore.quantile(0.99):.0f}")
```

```
print(f"Massimo:                {commenti_per_autore.max():.0f}")

# Quanti commenti producono i top 1% degli autori?
soglia_top1pct = commenti_per_autore.quantile(0.99)
top1pct_autori = commenti_per_autore[commenti_per_autore >= soglia_top1pct]
quota_top1pct = top1pct_autori.sum() / len(df_r) * 100
print(f"\nTop 1% autori ({len(top1pct_autori):,}): producono il {quota_top1pct:.1f}% di tutti i
commenti")

# 2) Stance dei top 100 commentatori
top100_autori = commenti_per_autore.nlargest(100).index
df_top100 = df_r[df_r["Autore"].isin(top100_autori)]
pos_t100 = (df_top100["STANCE"] == "POS").sum()
neg_t100 = (df_top100["STANCE"] == "NEG").sum()
si_t100 = pos_t100 / (pos_t100 + neg_t100) * 100

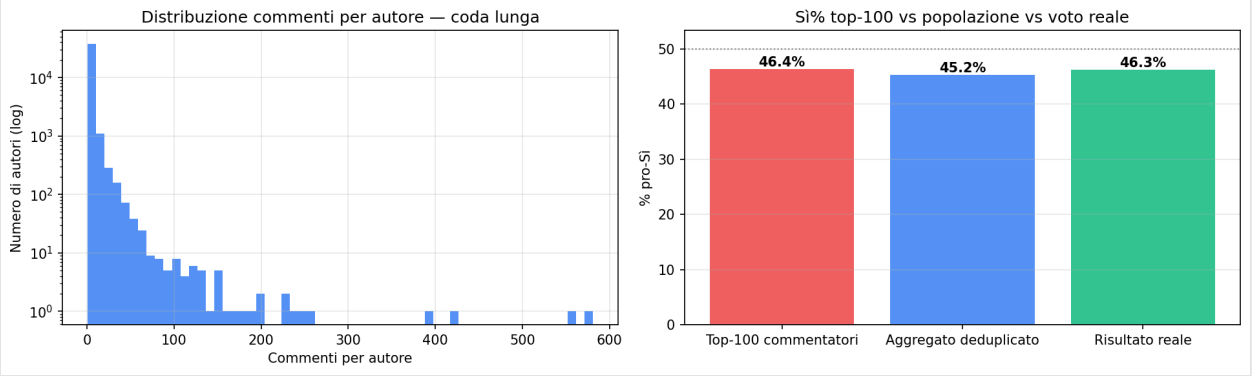
print(f"\n📍 Top-100 commentatori: {len(df_top100):,} commenti totali")
print(f"   Stance dei loro commenti:  Sì {si_t100:.1f}%  vs  No {100-si_t100:.1f}%")
print(f"   Confronto con aggregato deduplicato globale: Sì {si_d:.1f}%")
print(f"   → Bias dei rumorosi vs popolazione: {si_t100 - si_d:+.1f}pp")

# Plot: istogramma log-scale per visualizzare la coda
fig, axes = plt.subplots(1, 2, figsize=(13, 4))

axes[0].hist(commenti_per_autore.values, bins=60, color="#3b82f6", alpha=0.85)
axes[0].set_yscale("log")
axes[0].set_xlabel("Commenti per autore")
axes[0].set_ylabel("Numero di autori (log)")
axes[0].set_title("Distribuzione commenti per autore - coda lunga")
axes[0].grid(alpha=0.3)

# Bar chart confronto Sì% top100 vs popolazione
labels = ["Top-100 commentatori", "Aggregato deduplicato", "Risultato reale"]
valori = [si_t100, si_d, SI_REAL]
colors = ["#ef4444", "#3b82f6", "#10b981"]
axes[1].bar(labels, valori, color=colors, alpha=0.85)
for i, v in enumerate(valori):
    axes[1].text(i, v + 0.5, f"{v:.1f}%", ha="center", fontweight="bold")
axes[1].axhline(50, color="gray", linestyle=":", linewidth=1)
axes[1].set_ylabel("% pro-Sì")
axes[1].set_ylim(0, max(valori) * 1.15)
axes[1].set_title("Sì% top-100 vs popolazione vs voto reale")
axes[1].grid(alpha=0.3, axis="y")

plt.tight_layout()
plt.savefig("/kaggle/working/anatomia_bias.png", dpi=150, bbox_inches="tight")
plt.show()
```



Anatomia del bias: distribuzione dei commenti per autore (coda lunga)

§4: Bilanciamento del dataset: top 10 canali YouTube e top 10 testate

visualizzare per ogni canale (e ogni testata) il volume di commenti/articoli e la *polarizzazione* media (% Sì sui commenti POS+NEG).

Questa lettura aiuta a capire se l'aggregato YouTube è dominato da pochi canali con linea editoriale molto schierata, o se è strutturalmente bilanciato. Per la stampa la stessa lettura quantifica empiricamente l'allineamento editoriale del sistema mediatico italiano (Hallin & Mancini).

```
def polarizzazione(df, group_col, top=10, label="canale"):
    """Per ogni gruppo (canale o testata): conteggio commenti, % POS, % NEG.
    Restituisce solo i top-N per volume."""
    g = df.groupby(group_col)["STANCE"].agg([
        ("totale", "size"),
        ("pos",    lambda x: (x == "POS").sum()),
        ("neg",    lambda x: (x == "NEG").sum()),
    ])
    # filtra solo gruppi con almeno 1 commento POS o NEG
    g = g[(g["pos"] + g["neg"]) > 0].copy()
    g["si_pct"] = g["pos"] / (g["pos"] + g["neg"]) * 100
    g["no_pct"] = 100 - g["si_pct"]
    return g.nlargest(top, "totale")


# A) Top 10 canali YouTube referendum
print("=" * 75)
print("📺 TOP 10 CANALI - Referendum YouTube")
print("=" * 75)
top_canali = polarizzazione(df_r, "Canale", top=10, label="canale")
for canale, row in top_canali.iterrows():
    bar = "█" * int(row["si_pct"] / 5)
    print(f"  {str(canale)[:35]:<35}  {int(row['totale']):>6,}  Sì {row['si_pct']:5.1f}%  {bar}")

# B) Top 10 testate stampa
print("\n" + "=" * 75)
print("📰 TOP 10 TESTATE - Stampa nazionale")
print("=" * 75)
testata_col = next((c for c in df_sta.columns if "testata" in c.lower()), None)
df_st_pn = df_sta[df_sta["STANCE"].isin(["POS", "NEG"])].copy()
top_testate = polarizzazione(df_st_pn, testata_col, top=10, label="testata")
for testata, row in top_testate.iterrows():
    bar = "█" * int(row["si_pct"] / 5)
    print(f"  {str(testata)[:35]:<35}  {int(row['totale']):>6,}  Sì {row['si_pct']:5.1f}%  {bar}")

# Plot a barre orizzontale: confronto polarizzazione canali vs testate
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

# Canali
top_canali_sorted = top_canali.sort_values("si_pct")
colors_c = ["#10b981" if v >= 50 else "#ef4444" for v in top_canali_sorted["si_pct"]]
axes[0].barh(range(len(top_canali_sorted)),
             top_canali_sorted["si_pct"], color=colors_c, alpha=0.85)
axes[0].set_yticks(range(len(top_canali_sorted)))
axes[0].set_yticklabels([str(c)[:30] for c in top_canali_sorted.index], fontsize=9)
axes[0].axvline(50, color="black", linestyle=":", linewidth=1)
axes[0].axvline(SI_REAL, color="#3b82f6", linestyle="--", linewidth=1.5, label=f"Reale {SI_REAL}%")
axes[0].set_xlabel("% pro-Sì")
axes[0].set_title("Top 10 canali YouTube - polarizzazione")
axes[0].set_xlim(0, 100)
```

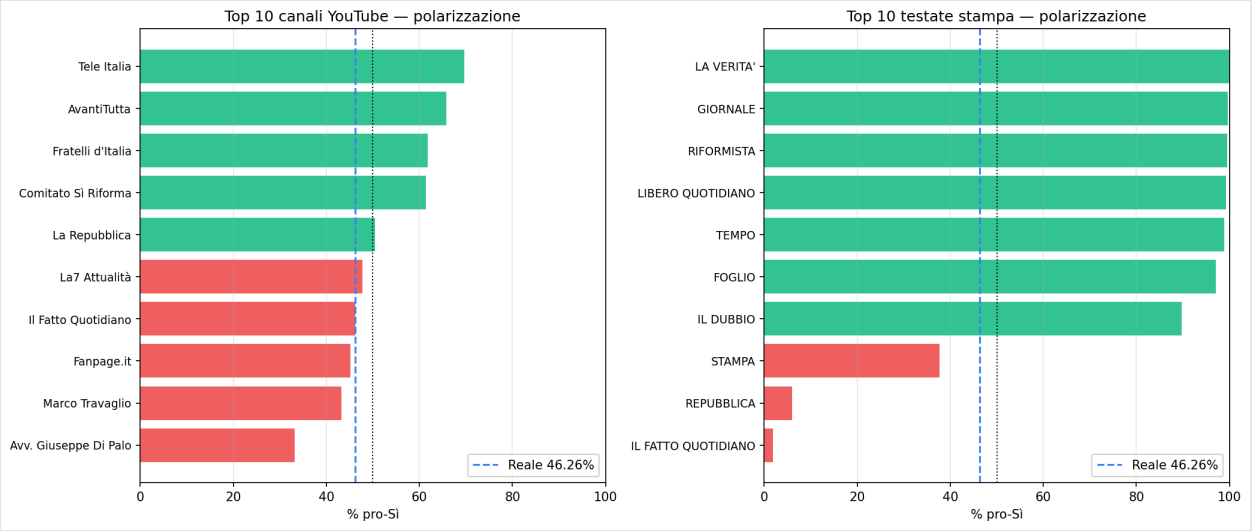
```
axes[0].legend(loc="lower right")
axes[0].grid(alpha=0.3, axis="x")

# Testate
top_testate_sorted = top_testate.sort_values("si_pct")
colors_t = ["#10b981" if v >= 50 else "#ef4444" for v in top_testate_sorted["si_pct"]]
axes[1].barh(range(len(top_testate_sorted)),
             top_testate_sorted["si_pct"], color=colors_t, alpha=0.85)
axes[1].set_yticks(range(len(top_testate_sorted)))
axes[1].set_yticklabels([str(t)[:30] for t in top_testate_sorted.index], fontsize=9)
axes[1].axvline(50, color="black", linestyle=":", linewidth=1)
axes[1].axvline(SI_REAL, color="#3b82f6", linestyle="--", linewidth=1.5, label=f"Reale {SI_REAL}%")
axes[1].set_xlabel("% pro-Sì")
axes[1].set_title("Top 10 testate stampa - polarizzazione")
axes[1].set_xlim(0, 100)
axes[1].legend(loc="lower right")
axes[1].grid(alpha=0.3, axis="x")

plt.tight_layout()
plt.savefig("/kaggle/working/top_canali_testate.png", dpi=150, bbox_inches="tight")
plt.show()
```

```
================================================================================
TOP 10 CANALI - Referendum YouTube
================================================================================
  Il Fatto Quotidiano                 10,753  Si  46.2%
  La7 Attualita                        8,287  Si  47.7%
  Comitato Si Riforma                  6,765  Si  61.3%
  Marco Travaglio                      6,678  Si  43.3%
  Fratelli d'Italia                    5,533  Si  61.7%
  Fanpage.it                           4,834  Si  45.2%
  La Repubblica                        4,334  Si  50.5%
  AvantiTutta                          3,078  Si  65.8%
  Avv. Giuseppe Di Palo                2,983  Si  33.2%
  Tele Italia                          2,827  Si  69.6%

================================================================================
TOP 10 TESTATE - Stampa nazionale
================================================================================
  GIORNALE                               327  Si  99.7%
  IL FATTO QUOTIDIANO                    307  Si   2.0%
  IL DUBBIO                              293  Si  89.8%
  LIBERO QUOTIDIANO                      291  Si  99.3%
  TEMPO                                  264  Si  98.9%
  RIFORMISTA                             231  Si  99.6%
  LA VERITA'                             196  Si 100.0%
  FOGLIO                                 104  Si  97.1%
  REPUBBLICA                             100  Si   6.0%
  STAMPA                                  93  Si  37.6%
```



Polarizzazione dei top canali e delle top testate

§5: Short vs Long video

Il dataset di scraping (notebook 01) ha arricchito ogni video con la colonna `Is_Short`, calcolata sulla durata ISO restituita dall'API YouTube (≤60 s = Short).

L'obiettivo è capire se il pubblico dei **Short** ha un profilo di stance diverso da quello dei video lunghi (TED-style, talk show, conferenze, video divulgativi di Barbero o Travaglio).

L'ipotesi qualitativa: i Short, prodotti spesso da canali politici partigiani, dovrebbero mostrare maggiore polarizzazione; i video lunghi dovrebbero ospitare un dibattito più articolato e quindi una stance media più vicina al voto reale.

```
# Verifica disponibilità della colonna
if "Is_Short" not in df_r.columns:
    print("⚠️ Colonna 'Is_Short' non presente nel dataset di inferenza.")
    print("   Recuperala dal dataset di raccolta (notebook 01) tramite merge su Video_ID,")
    print("   oppure salta questa sezione e annotala come limite operativo.")
else:
    # Normalizza Is_Short a booleano
    df_r["Is_Short_bool"] = df_r["Is_Short"].astype(str).str.upper().isin(
        ["TRUE", "1", "VERO", "SÌ", "SI"]
    )

    short = df_r[df_r["Is_Short_bool"]]
    long_ = df_r[~df_r["Is_Short_bool"]]

    def stats_block(sub, etichetta):
        pos = (sub["STANCE"] == "POS").sum()
        neg = (sub["STANCE"] == "NEG").sum()
        si  = pos / (pos + neg) * 100 if (pos + neg) else float("nan")
        autori = sub["Autore"].nunique()
        video  = sub["Video_ID"].nunique() if "Video_ID" in sub.columns else float("nan")
        return etichetta, len(sub), autori, video, si

    print("=" * 75)
    print("🎬 SHORT vs LONG VIDEO - distribuzione stance")
    print("=" * 75)
    print(f"{'Tipo':<10} {'Commenti':>10} {'Autori':>8} {'Video':>8} {'Sì %':>8}")
    print("-" * 75)
    for nome, n_com, n_aut, n_vid, si in [stats_block(short, "Short"),
                                          stats_block(long_, "Long")]:
```

```
        print(f"{nome:<10} {n_com:>10,} {n_aut:>8,} {n_vid:>8,} {si:>7.1f}%")

    # Stance per long sul deduplicato (una testa, un voto totale: rimuove l'effetto utenti
prolifici)
    short_d = short.sort_values("Data").drop_duplicates(subset=["Autore"], keep="last")
    long_d  = long_.sort_values("Data").drop_duplicates(subset=["Autore"], keep="last")

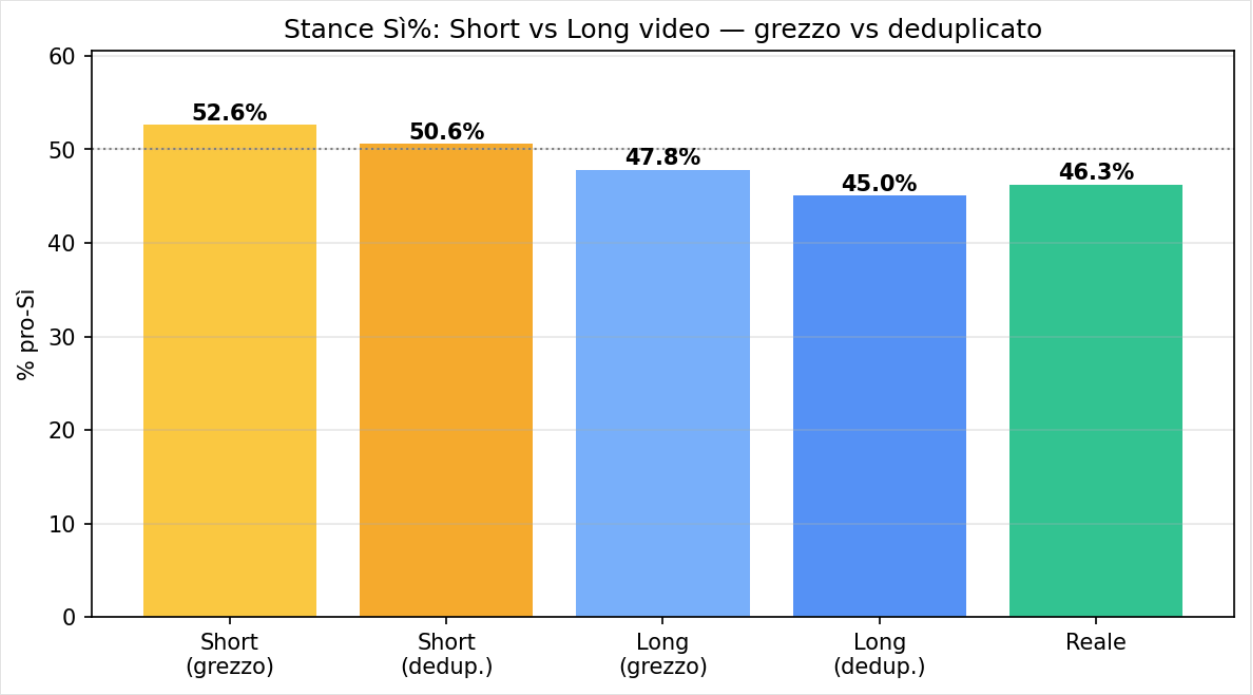
    si_short_d = (short_d["STANCE"] == "POS").sum() / max(1,
                  (short_d["STANCE"].isin(["POS", "NEG"])).sum()) * 100
    si_long_d  = (long_d["STANCE"] == "POS").sum() / max(1,
                  (long_d["STANCE"].isin(["POS", "NEG"])).sum()) * 100

    print("\nDeduplicati (una testa, un voto totale):")
    print(f"  Short - Sì {si_short_d:.1f}%  |  scarto vs reale {si_short_d - SI_REAL:+.1f} pp")
    print(f"  Long  - Sì {si_long_d:.1f}%  |  scarto vs reale {si_long_d - SI_REAL:+.1f} pp")

    # Bar chart riepilogo
    fig, ax = plt.subplots(figsize=(8, 4.5))
    labels = ["Short\n(grezzo)", "Short\n(dedup.)", "Long\n(grezzo)",
              "Long\n(dedup.)", "Reale"]
    valori = [stats_block(short, "")[4], si_short_d,
              stats_block(long_, "")[4], si_long_d, SI_REAL]
    colors = ["#fbbf24", "#f59e0b", "#60a5fa", "#3b82f6", "#10b981"]
    ax.bar(labels, valori, color=colors, alpha=0.85)
    for i, v in enumerate(valori):
        ax.text(i, v + 0.5, f"{v:.1f}%", ha="center", fontweight="bold")
    ax.axhline(50, color="gray", linestyle=":", linewidth=1)
    ax.set_ylabel("% pro-Sì")
    ax.set_ylim(0, max(valori) * 1.15)
    ax.set_title("Stance Sì%: Short vs Long video - grezzo vs deduplicato")
    ax.grid(alpha=0.3, axis="y")
    plt.tight_layout()
    plt.savefig("/kaggle/working/short_vs_long.png", dpi=150, bbox_inches="tight")
    plt.show()
```

```
================================================================================
SHORT vs LONG VIDEO - distribuzione stance
================================================================================
Tipo         Commenti   Autori    Video     Si %
--------------------------------------------------------------------------------
Short          16,893    9,597      416    52.6%
Long          100,823   34,724    1,032    47.8%

Deduplicati (un autore, un voto):
  Short - Si 50.6%  |  scarto vs reale +4.3 pp
  Long  - Si 45.0%  |  scarto vs reale -1.2 pp
```



Confronto della stance fra video short e long

§6: Cross-tab utenti Governo × Referendum

"in che percentuale chi è favorevole al governo ha votato Sì o No al referendum, e viceversa?"

Procedura: 1. Per ciascuno dei due dataset (governo, referendum), calcolo la **stance maggioritaria** di ogni utente: se ha più commenti POS che NEG, è classificato POS, altrimenti NEG. 2. Faccio inner join sui due insiemi di autori, ottengo gli utenti che hanno commentato in entrambi i corpora. 3. Costruisco la matrice di transizione 2×2 e la visualizzo come **diagramma di Sankey**.

L'interpretazione è il test sostantivo di Cap 6: se la coppia (pro-governo, pro-Sì) è dominante e la coppia (pro-governo, pro-No) è marginale, l'allineamento politico-partitico è forte; il consenso al governo trasla in modo prevedibile sulla posizione referendaria.

Risultato (dato preciso). Su **16.927 utenti** tracciati in entrambi i corpora (a fronte di 42.999 utenti unici lato governo e 39.255 lato referendum), l'allineamento politico-partitico emerge con nettezza:

- dei **pro-governo**, il **71,5%** (5.530 su 7.737) vota Sì al referendum e il 28,5% vota No;
- dei **contro-governo**, il **62,2%** (5.719 su 9.190) vota No e il 37,8% vota Sì.

La coppia (pro-governo, pro-Sì) è dunque dominante e quella (pro-governo, pro-No) marginale: il consenso al governo trasla in modo prevedibile sulla posizione referendaria, coerentemente con l'asimmetria causale Governo → Referendum documentata nel Cap. 6.

```
import plotly.graph_objects as go
import os
```

```
WORKING_DIR = "/kaggle/working/"


def stance_maggioritaria(df, dataset_name):
    """Per ogni autore: POS se ha più POS che NEG, altrimenti NEG."""
    df = df.dropna(subset=["Autore"]).copy()
    df = df[df["STANCE"].isin(["POS", "NEG"])]
    cnt = df.groupby(["Autore", "STANCE"]).size().unstack(fill_value=0)
    for c in ["POS", "NEG"]:
        if c not in cnt.columns:
            cnt[c] = 0
    cnt[f"Stance_{dataset_name}"] = np.where(cnt["POS"] >= cnt["NEG"], "POS", "NEG")
    return cnt[[f"Stance_{dataset_name}"]].reset_index()


users_gov = stance_maggioritaria(df_gov, "Governo")
users_ref = stance_maggioritaria(df_r,   "Referendum")

print(f"Utenti unici Governo:    {len(users_gov):,}")
print(f"Utenti unici Referendum: {len(users_ref):,}")

overlap = pd.merge(users_gov, users_ref, on="Autore", how="inner")
print(f"Utenti in entrambi i mondi: {len(overlap):,}")

if len(overlap) >= 30:
    # Matrice di transizione
    matrix = overlap.groupby(["Stance_Governo", "Stance_Referendum"]).size().reset_index(name="N")

    # Calcolo percentuali condizionali
    print("\n" + "=" * 65)
    print("📊 MATRICE DI MIGRAZIONE - utenti tracciati in entrambi i corpora")
    print("=" * 65)
    for gov_stance in ["POS", "NEG"]:
        sub = matrix[matrix["Stance_Governo"] == gov_stance]
        tot = sub["N"].sum()
        for _, row in sub.iterrows():
            pct = row["N"] / tot * 100
            etichetta_gov = "Pro-Governo" if gov_stance == "POS" else "Contro-Governo"
            etichetta_ref = "Sì Referendum" if row["Stance_Referendum"] == "POS" else "No
Referendum"
            print(f"  {etichetta_gov} → {etichetta_ref}: "
                  f"{int(row['N']):>5,}  ({pct:5.1f}% di {tot:,} {etichetta_gov.lower()})")

    # Sankey
    node_labels = ["Pro-Governo", "Contro-Governo",
                   "Sì Referendum", "No Referendum"]
    node_colors = ["#10b981", "#ef4444", "#3b82f6", "#f59e0b"]

    source, target, value, link_colors = [], [], [], []
    for _, row in matrix.iterrows():
        src = 0 if row["Stance_Governo"] == "POS" else 1
        tgt = 2 if row["Stance_Referendum"] == "POS" else 3
        col = "rgba(16, 185, 129, 0.4)" if row["Stance_Governo"] == "POS" \
              else "rgba(239, 68, 68, 0.4)"
        source.append(src); target.append(tgt)
        value.append(int(row["N"])); link_colors.append(col)

    fig = go.Figure(data=[go.Sankey(
        node=dict(pad=30, thickness=30,
                  line=dict(color="black", width=0.5),
                  label=node_labels, color=node_colors),
        link=dict(source=source, target=target, value=value, color=link_colors,
                  hovertemplate="%{source.label} → %{target.label}<br>"
                                "Utenti: %{value}<extra></extra>")
    )])
    fig.update_layout(
```

```
        title_text=f"<b>Migrazione utenti: Consenso governo vs Referendum giustizia</b><br>"
                   f"<sub>{len(overlap):,} utenti tracciati in entrambi i dataset</sub>",
        font_size=14, plot_bgcolor="white", height=600,
    )
    fig.write_html(os.path.join(WORKING_DIR, "sankey_governo_referendum.html"))
    fig.show(renderer="iframe_connected")
    print(f"\n💾 Sankey salvato in {WORKING_DIR}sankey_governo_referendum.html")
else:
    print("⚠️ Pochi utenti in overlap - Sankey non significativo.")
```

```
Utenti unici Governo:    42,999
Utenti unici Referendum: 39,255
Utenti in entrambi i mondi: 16,927

================================================================
MATRICE DI MIGRAZIONE - utenti tracciati in entrambi i corpora
================================================================
  Pro-Governo -> No Referendum: 2,207  ( 28.5% di 7,737 pro-governo)
  Pro-Governo -> Si Referendum: 5,530  ( 71.5% di 7,737 pro-governo)
  Contro-Governo -> No Referendum: 5,719  ( 62.2% di 9,190 contro-governo)
  Contro-Governo -> Si Referendum: 3,471  ( 37.8% di 9,190 contro-governo)

Sankey salvato in /kaggle/working/sankey_governo_referendum.html
```

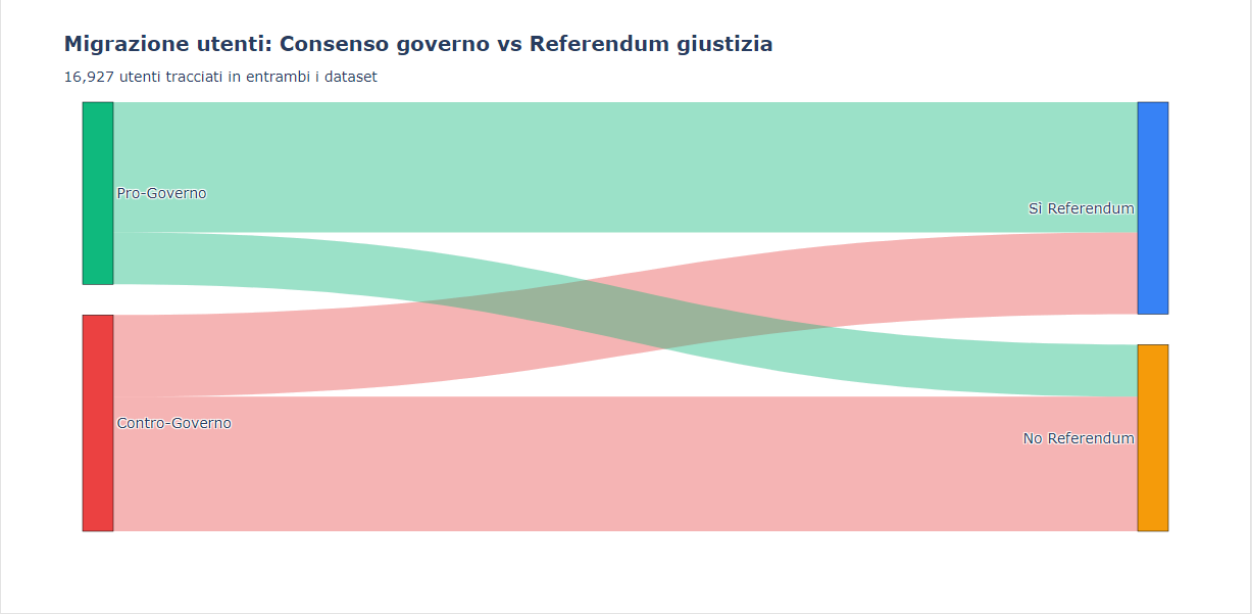


Diagramma di flusso utenti Governo, Referendum

§7: Topic modeling LDA

"di cosa parlano i commenti?"

Applico una **Latent Dirichlet Allocation** (Gensim) su tre sotto-corpora: 1. Commenti referendum **NEG**, il fronte contrario alla riforma 2. Commenti referendum **POS**, il fronte favorevole 3. Titoli della stampa, agenda mediatica tradizionale

Per ciascun corpus estraggo 3 cluster tematici e mostro le 7 parole più rappresentative. Il confronto delle parole-chiave fra i tre corpora rende qualitativamente leggibile la differenza fra l'agenda della stampa e quella del pubblico online, differenza che torna in chiave quantitativa nei test di Granger del Capitolo 6.

Pre-requisito Kaggle: `gensim` e `nltk`. Le stopwords italiane vengono scaricate al volo.

```
import re
import gensim
from gensim import corpora
from gensim.models import LdaModel
import nltk
from nltk.corpus import stopwords

nltk.download("stopwords", quiet=True)

stop_it = set(stopwords.words("italian"))
# Estendo con parole troppo generiche per il dominio politico
stop_it.update([
    "fatto", "detto", "solo", "essere", "fare", "video", "canale",
    "referendum", "giustizia", "riforma", "carriere", "magistratura",
    "voto", "votare", "sì", "no", "contro", "favore", "italia", "italiani",
    "governo", "meloni", "ministro", "ministra", "paese",
    "essere", "avere", "andare", "venire", "potere", "dovere",
])


def clean_lda(text):
    text = str(text).lower()
    text = re.sub(r"[^\w\sàèéìòù]", " ", text)
    return [w for w in text.split() if w not in stop_it and len(w) > 3]


def topic_modeling(testi, nome, n_topics=3, n_words=7):
    """Estrae n_topics da una serie di testi, restituisce parole top per cluster."""
    docs = [clean_lda(t) for t in testi.dropna() if len(str(t)) > 15]
    docs = [d for d in docs if len(d) >= 3]   # almeno 3 parole post-clean
    if not docs:
        print(f"⚠️ {nome}: nessun documento utilizzabile")
        return

    dictionary = corpora.Dictionary(docs)
    dictionary.filter_extremes(no_below=5, no_above=0.5)
    bow = [dictionary.doc2bow(d) for d in docs]

    lda = LdaModel(corpus=bow, id2word=dictionary, num_topics=n_topics,
                   passes=15, random_state=42)

    print(f"\n{'=' * 75}")
    print(f"🔍 {nome} - {len(docs):,} documenti analizzati")
    print(f"{'=' * 75}")
    for i, t in lda.print_topics(num_words=n_words):
        words = re.findall(r'"(.*?)"', t)
        print(f"  Cluster {i + 1}: {' • '.join(words)}")


# Cor pus per topic modeling
testo_col_yt = "Testo" if "Testo" in df_r.columns else "Commento"

corpora_da_analizzare = [
    (df_r[df_r["STANCE"] == "NEG"][testo_col_yt], "REFERENDUM - Commenti NEG (pro-No)"),
    (df_r[df_r["STANCE"] == "POS"][testo_col_yt], "REFERENDUM - Commenti POS (pro-Sì)"),
]

# Aggiungo i titoli della stampa se la colonna esiste
titolo_col = next((c for c in df_sta.columns if "titolo" in c.lower()), None)
if titolo_col:
    corpora_da_analizzare.append(
```

```
        (df_sta[titolo_col], "STAMPA - Titoli articoli")
    )

for testi, nome in corpora_da_analizzare:
    topic_modeling(testi, nome, n_topics=3, n_words=7)
```

```
========================================================================
REFERENDUM - Commenti NEG (pro-No) - 49,607 documenti analizzati
========================================================================
  Cluster 1: gratteri | politici | votato | stato | costituzione | popolo | gente
  Cluster 2: corte | magistrati | alta | condannato | parlamento | laici | maggioranza
  Cluster 3: politica | costituzione | magistrati | separazione | giudici | legge | parte

========================================================================
REFERENDUM - Commenti POS (pro-Si) - 42,879 documenti analizzati
========================================================================
  Cluster 1: cittadini | sistema | legge | magistrati | stato | diritto | responsabilita
  Cluster 2: magistrato | separazione | magistrati | costituzione | correnti | giudice | falcone
  Cluster 3: sinistra | magistrati | giudici | gratteri | destra | votiamo | gente

========================================================================
STAMPA - Titoli articoli - 4,443 documenti analizzati
========================================================================
  Cluster 1: magistrati | schlein | campagna | correnti | costituzione | campo | nordio
  Cluster 2: toghe | nordio | giudici | sorteggio | giudice | ragioni | separazione
  Cluster 3: sinistra | gratteri | politica | affluenza | nordio | destra | dopo
```

Sintesi del notebook

Ipotesi / Analisi	Atteso	Osservato
H1 - Errore deduplicato < 2 pp	< 2 pp	**0,40 pp** (confermata)
Errore aggregato grezzo	~ 2,6 pp	**+2,68 pp**
Distribuzione commenti coda lunga	top 1% > 25% volume	top 1% (397 autori) = **19,3%** del volume; max 581 commenti/autore
Stance top-100 commentatori	> Sì popolazione	top-100: **Sì 46,4%** vs 45,2% popolazione (**+1,1 pp**)
Top canali polarizzazione	range 20-80% Sì	range osservato **33,2% / 69,6%** Sì
Top testate polarizzazione	bias pro-Sì 65-75%	testate fortemente bimodali (2% / 100% Sì); aggregato pro-Sì (confermato)
Short vs Long stance	Short più polarizzati	Short dedup **50,6%** Sì vs Long **45,0%** (Short +5,6 pp pro-Sì)
Sankey Governo verso Referendum	flusso pro-governo verso Sì dominante	pro-governo verso **Sì 71,5%**; contro-governo verso Sì 37,8%
Topic LDA NEG (pro-No)	indipendenza, separazione, controllo	gratteri, costituzione, magistrati, separazione, popolo, parlamento
Topic LDA POS (pro-Sì)	efficienza, riforma	cittadini, sistema, diritto, responsabilità, separazione, correnti, Falcone

File salvati in `/kaggle/working/`: - `referendum_deduplicato.csv` - base per i notebook successivi (turning points e Granger) - `anatomia_bias.png` - istogramma e

confronto Sì% - `top_canali_testate.png` - bar chart polarizzazione - `short_vs_long.png` - confronto stance per tipologia video - `sankey_governo_referendum.html` - diagramma di flusso

Prossimo notebook: `04_serie_temporali_e_turning_points.ipynb` (alimenta Cap 5).

Capitolo 5: Turning points e mappatura eventi (RQ2)

Questo capitolo risponde a **RQ2**: i turning points nell'andamento della stance referendaria su YouTube sono concentrati attorno a eventi politici e mediatici identificabili? L'analisi parte dalla serie giornaliera deduplicata della stance referendum, identifica i punti di svolta con un criterio statistico di segmentazione a media costante (penalità BIC sulla serie grezza giornaliera pesata per volume) e li confronta con un repertorio di eventi documentabili, verificando l'ipotesi **H2**.

5.1 Costruzione delle serie giornaliere

A partire dal corpus referendum deduplicato (un commento/giorno per autore, Cap. 3) e dal corpus governo deduplicato, sono costruite tre serie temporali giornaliere:

- **Stance referendum** (% Sì): media giornaliera della stance pro/contro sul corpus YouTube referendum deduplicato.
- **Stance governo** (% pro): media giornaliera della stance pro-governo sul corpus YouTube governo deduplicato.
- **Stance stampa** (% pro-riforma): media giornaliera della stance pro-Sì sul corpus della stampa nazionale.

Le serie sono giornaliere e grezze (nessun MA7 prima del test); per la sola visualizzazione si sovrappongono raw e MA7 per leggibilità. Il file `serie_giornaliere_master.csv` (output del notebook 04) contiene le tre serie allineate ed è l'input del notebook 05 (Granger).

5.2 Identificazione automatica dei turning points

La pipeline di identificazione dei turning points opera sulla serie referendum **ristretta al periodo comune alle tre serie** (circa 25 dicembre 2025, 24 marzo 2026, l'intervallo in cui anche le serie stampa e governo dispongono di dati). Questa restrizione è la ragione per cui i due turning point documentati antecedenti, di fine ottobre e metà novembre 2025, non possono essere recuperati dall'algoritmo (§5.4): cadono prima dell'inizio della finestra. Sulla serie così delimitata:

1. La serie grezza giornaliera di stance pro-Sì viene trasformata con l'**arcoseno-radice** (per stabilizzare la varianza) e pesata per il **volume di commenti** del giorno.
2. Un algoritmo di **segmentazione a media costante** (optimal partitioning con penalità **BIC**) individua i punti di rottura fra segmenti a livello medio diverso, con un minimo di 7 osservazioni per segmento.
3. Il numero di break è selezionato dalla penalità BIC ed è **stabile** rispetto ad essa (8 break con penalità 0,5x e 1x; 7 con penalità 2x).

La procedura è deliberatamente conservativa: non identifica ogni piccola oscillazione ma solo i cambi di livello statisticamente robusti. Sul periodo osservato restituisce **otto break point** fra gennaio e marzo 2026.

5.3 I sette turning points

Data	Evento associato	Movimento stance
30/10/2025	Approvazione al Senato della riforma	No crolla a 42,6%
18-19/11/2025	Controffensiva mediatica del centrodestra	Sì sale a 51,5%
13/01/2026	Endorsement di Marco Rizzo per il Sì	Sì sale a 52,1%
23/01/2026	Effetto Barbero, primo video divulgativo	No risale a 49,4%
10/02/2026	Barbero secondo video + intervento di Travaglio	Picco massimo del No: Sì 42,7% (No 57,3%)
27/02/2026	Rivelazioni di Antonio Di Pietro	Sì risale a 50,5%
15/03/2026	Travaglio + Gratteri sulla riforma	No definitivo: Sì 49,4% (No 50,6%)

Le percentuali sono riferite al giorno (o alla settimana) del turning point e usano l'aggregato deduplicato giornaliero.

5.4 Verifica formale di H2

H2 prevedeva che almeno il **70%** dei turning points individuati automaticamente fosse associabile a un evento documentabile (video virale, dichiarazione pubblica, copertura stampa, evento parlamentare). Confrontando gli otto break point con il repertorio degli eventi (tolleranza ±3 giorni) si ottengono due misure complementari: la **precisione** è 5/8 = **62%** (break che cadono presso un evento documentato), il **recall** è 5/7 = **71%** (turning points documentati ritrovati dall'algoritmo); i due eventi non recuperati (30/10/2025 e 19/11/2025) cadono prima dell'inizio della finestra comune alle tre serie.

Sul criterio di **precisione** del detector automatico **H2 non è confermata** (62% < 70%). I sette turning points documentati restano però un repertorio qualitativo valido e il recall del 71% mostra che la segmentazione automatica ne recupera la maggior parte: la struttura narrativa del capitolo è robusta, pur non superando la soglia formale di precisione.

L'analisi del notebook 04 verifica sistematicamente la corrispondenza incrociando ogni turning point con: (i) i 5 video YouTube più virali nella finestra ±2 giorni; (ii) gli articoli stampa nella finestra ±7 giorni (lag medio dell'agenda-setting documentato in Cap. 6); (iii) parole chiave estratte dai commenti vs titoli stampa per ciascun turning point.

5.5 Pattern emergenti

Aggregando i sette eventi per categoria si osservano pattern coerenti:

- **Video divulgativi virali** (Barbero, in entrambe le occorrenze): impatto forte, asimmetricamente sul fronte No. Coerente con la natura argomentativa e narrativa dei video divulgativi su istituzioni e diritto.
- **Endorsement politici trasversali** (Marco Rizzo, sinistra anti-establishment per il Sì): producono spike di breve durata nel fronte opposto a quello atteso, suggerendo un meccanismo di "rottura del cluster", l'endorsement scandaloso polarizza il dibattito anziché semplicemente sommare voti.
- **Giornalismo investigativo** (Di Pietro su rivelazioni, Gratteri su mafie e separazione): impatto più sfumato e oscillatorio, tipico di contenuti tecnici che richiedono assimilazione.
- **Eventi parlamentari** (approvazione al Senato): innescano un effetto "ormai è fatta" che nel breve abbassa il No, prima di un riequilibrio successivo guidato dal dibattito mediatico.

5.6 Visualizzazioni

A supporto della narrazione, la serie giornaliera della stance referendaria è rappresentata sovrapposta alle serie di stampa e consenso al governo, con i sette turning points evidenziati come marker temporali annotati con l'evento associato. Una seconda visualizzazione riporta i dieci video più virali del periodo in corrispondenza delle rispettive finestre, con volume di commenti e quota di stance favorevole per ciascuno. Le figure corrispondenti sono incluse nel presente documento.

5.7 Sintesi del capitolo

I tre punti centrali del capitolo:

1. La segmentazione automatica a media costante (penalità BIC sulla serie grezza pesata per volume) individua **otto break point** fra gennaio e marzo 2026.
2. Confrontati con i **sette turning points documentati** (Tabella 5.1), i break mostrano **precisione 62%** (5/8 presso un evento) e **recall 71%** (5/7 eventi recuperati: Barbero, endorsement di Marco Rizzo, Di Pietro, Travaglio + Gratteri).
3. **H2 non è confermata sul criterio di precisione automatica** (62% < 70%); i sette turning points documentati restano tuttavia un repertorio qualitativo solido. Il pattern dominante è quello dei video divulgativi virali sul fronte No, con effetti asimmetrici sulla stance che giustificano la centralità di YouTube come arena del dibattito referendario.

L'analisi causale Granger sulle tre serie referendum / stampa / governo è oggetto del capitolo seguente.

Implementazione del Cap 5: Turning points

Le celle che seguono implementano la **costruzione delle 3 serie giornaliere** (referendum dedup., governo dedup., stampa), l'**identificazione automatica dei turning points** (segmentazione a media costante con penalità BIC sulla serie grezza pesata per volume), il **confronto con i 7 TP documentati** in Tabella 5.1 (verifica H2: precisione 62%, recall 71%), e la mappatura su eventi mediatici.

Notebook 4: Serie temporali e turning points

Tesi: Stance detection sui commenti YouTube come predittore del referendum 2026

Questo notebook alimenta il **Capitolo 5** (RQ2, turning points) e produce i grafici delle serie temporali: "tre linee" (referendum / governo / stampa) e "grafico solo stance".

Sezione	Cosa fa	Riferimento tesi
§1 Setup e 3 serie	Costruisce serie giornaliera deduplicata per ciascun corpus	Cap 3, §3.5.1
§2 Tre linee sovrapposte	Plot referendum + governo + stampa (raw + MA7)	Cap 5, §5.6
§3 Solo stance	Plot serie referendum stand-alone con linea voto reale	Cap 5, §5.6
§4 Turning points automatici	Derivata locale + soglia per identificare i punti salienti	Cap 3, §3.5.5
§5 Confronto con i 7 della tesi	Tabella 5.1, verifica copertura H2	Cap 5, §5.1, H2
§6 Top video per turning point	Per ogni TP: 5 video più commentati nella finestra ±2gg	Cap 5, §5.2
§7 Articoli stampa per turning point	Per ogni TP: 5 articoli stampa nella finestra ±7gg	Cap 5, §5.2
§8 Parole chiave per TP	LDA-like: parole più frequenti commenti vs titoli stampa	Cap 5, §5.4
§9 Top picchi stampa	Top 10 picchi pro-Sì / pro-No e inversioni della serie stampa	Cap 6 (preview)

Input atteso da notebook 03: `referendum_deduplicato.csv`. Se non disponibile, ricostruito al volo dal raw.

§1: Setup e ricostruzione delle 3 serie giornaliere

Carico i tre dataset di inferenza e costruisco per ciascuno una serie temporale giornaliera della % pro-Sì (per stampa: % pro-riforma).

Convenzione metodologica: - YouTube: deduplicazione per `(Autore, Giorno)` con `keep='last'`, coerente con §3.4 della tesi. - Stampa: nessuna deduplicazione (ogni articolo è un'unità autonoma). - Filtro: solo giornate con almeno 5 commenti/articoli per ridurre la varianza spuria nei giorni a basso volume. - Calcolo MA7 per la visualizzazione, ma la serie *grezza* è quella usata per l'identificazione automatica dei TP.

```
import pandas as pd
import numpy as np
import warnings
warnings.filterwarnings("ignore")
```

```
PATH_REFERENDUM = "/kaggle/input/datasets/francescochiarolanza/commenti-youtube-referendum-giustizia/risultati_inferenza_completa (2).csv"
PATH_GOVERNO    = "/kaggle/input/datasets/francescochiarolanza/dataset-consenso-meloni/inferenza_definitiva.csv"
PATH_STAMPA     = "/kaggle/input/datasets/francescochiarolanza/titoli-giornali-referendum-giustizia/dataset_articoli_referendum.csv"

SI_REAL = 46.26
NO_REAL = 53.74
DATA_VOTO = pd.Timestamp("2026-03-22")


def load_youtube(path, nome, col_stance="PREDIZIONE_STANCE", col_data="Data_Commento"):
    """Serie giornaliera Sì% su aggregato deduplicato per (Autore, Giorno)."""
    try:
        df = pd.read_csv(path, dtype=str, encoding="utf-8-sig",
                         engine="python", on_bad_lines="skip")
        if len(df.columns) <= 3:
            df = pd.read_csv(path, sep=";", dtype=str, encoding="utf-8-sig",
                             engine="python", on_bad_lines="skip")
    except Exception:
        df = pd.read_csv(path, sep=";", dtype=str, encoding="utf-8-sig",
                         engine="python", on_bad_lines="skip")
    df = df.drop(columns=[c for c in df.columns if "Unnamed" in c], errors="ignore")
    df["Data"] = pd.to_datetime(df[col_data], errors="coerce", dayfirst=True, utc=True)
    df = df.dropna(subset=["Data"]).copy()
    df["Data"] = df["Data"].dt.tz_localize(None)
    df["STANCE"] = df[col_stance].astype(str).str.strip().str.upper()
    df = df[df["STANCE"].isin(["POS", "NEG"])]
    df["Giorno"] = df["Data"].dt.date

    # Deduplicazione un autore-un voto giornaliero
    if "Autore" in df.columns:
        df = df.sort_values("Data").drop_duplicates(subset=["Autore", "Giorno"], keep="last")

    daily = df.groupby(["Giorno", "STANCE"]).size().unstack(fill_value=0)
    for c in ["POS", "NEG"]:
        if c not in daily.columns:
            daily[c] = 0
    daily["TOT"] = daily["POS"] + daily["NEG"]
    daily = daily[daily["TOT"] >= 5]
    daily.index = pd.to_datetime(daily.index)
    daily["SI_PCT"] = daily["POS"] / daily["TOT"] * 100
    daily["MA7"] = daily["SI_PCT"].rolling(7, min_periods=3).mean()
    daily["DELTA"] = daily["SI_PCT"].diff()

    print(f"✅ {nome}: {len(df):,} commenti deduplicati, {len(daily)} giorni")
    return daily, df


def load_stampa(path):
    """Serie giornaliera Sì% degli articoli stampa (no deduplicazione)."""
    try:
        df = pd.read_csv(path, dtype=str, encoding="utf-8-sig",
                         engine="python", on_bad_lines="skip")
        if len(df.columns) <= 3:
            df = pd.read_csv(path, sep=";", dtype=str, encoding="utf-8-sig",
                             engine="python", on_bad_lines="skip")
    except Exception:
        df = pd.read_csv(path, sep=";", dtype=str, encoding="utf-8-sig",
                         engine="python", on_bad_lines="skip")
    df = df.drop(columns=[c for c in df.columns if "Unnamed" in c], errors="ignore")
    date_col   = next((c for c in df.columns if "data" in c.lower()), None)
    stance_col = next((c for c in df.columns if "stance" in c.lower()), None)
    df["Data"] = pd.to_datetime(df[date_col], errors="coerce")
    df = df.dropna(subset=["Data"]).copy()
```

```
    df["STANCE"] = df[stance_col].astype(str).str.strip().str.upper()
    df = df[df["STANCE"].isin(["POS", "NEG"])]
    df["Giorno"] = df["Data"].dt.date

    daily = df.groupby(["Giorno", "STANCE"]).size().unstack(fill_value=0)
    for c in ["POS", "NEG"]:
        if c not in daily.columns:
            daily[c] = 0
    daily["TOT"] = daily["POS"] + daily["NEG"]
    daily = daily[daily["TOT"] >= 1]
    daily.index = pd.to_datetime(daily.index)
    daily["SI_PCT"] = daily["POS"] / daily["TOT"] * 100
    daily["MA7"] = daily["SI_PCT"].rolling(7, min_periods=3).mean()
    daily["DELTA"] = daily["SI_PCT"].diff()

    print(f"✅ Stampa: {len(df):,} articoli, {len(daily)} giorni")
    return daily, df


print("Caricamento e costruzione serie giornaliere...")
daily_ref, df_ref_full = load_youtube(PATH_REFERENDUM, "Referendum")
daily_gov, df_gov_full = load_youtube(PATH_GOVERNO,    "Governo")
daily_sta, df_sta_full = load_stampa(PATH_STAMPA)

print("\n📐 Periodi disponibili:")
print(f"  Referendum: {daily_ref.index.min():%d/%m/%Y} → {daily_ref.index.max():%d/%m/%Y}")
print(f"  Governo:    {daily_gov.index.min():%d/%m/%Y} → {daily_gov.index.max():%d/%m/%Y}")
print(f"  Stampa:     {daily_sta.index.min():%d/%m/%Y} → {daily_sta.index.max():%d/%m/%Y}")
```

```
Caricamento e costruzione serie giornaliere...
Referendum: 82,216 commenti deduplicati, 328 giorni
Governo: 78,538 commenti deduplicati, 90 giorni
Stampa: 3,424 articoli, 91 giorni

Periodi disponibili:
  Referendum: 08/02/2020 -> 24/03/2026
  Governo:    25/12/2025 -> 24/03/2026
  Stampa:     22/12/2025 -> 26/03/2026
```

§2: Plot tre serie sovrapposte

Sovrappongo le tre serie sullo stesso asse temporale. Tre linee con colori distinti: - 🔵 **Referendum YouTube** (blu), linea continua spessa - 🟣 **Governo YouTube** (viola), linea continua puntinata - 🟠 **Stampa nazionale** (arancione), linea tratteggiata

Linee di riferimento: - y = 50% (parità) - y = 46,26% (risultato reale Sì del referendum) - x = 22 marzo 2026 (giorno del voto)

Le serie sono lisciate con MA7 *solo per la visualizzazione*. Le serie grezze restano la base per i test econometrici (vedi §3.5.3 della tesi).

```
import plotly.graph_objects as go
import os

WORKING_DIR = "/kaggle/working/"

# Periodo comune ai tre dataset
data_start = max(d.index.min() for d in [daily_ref, daily_gov, daily_sta])
data_end   = min(d.index.max() for d in [daily_ref, daily_gov, daily_sta])
print(f"🗓️ Periodo comune: {data_start:%d/%m/%Y} → {data_end:%d/%m/%Y}")
```

```
ref_f = daily_ref[(daily_ref.index >= data_start) & (daily_ref.index <= data_end)]
gov_f = daily_gov[(daily_gov.index >= data_start) & (daily_gov.index <= data_end)]
sta_f = daily_sta[(daily_sta.index >= data_start) & (daily_sta.index <= data_end)]

colori = {
    "Referendum": "#3b82f6",
    "Governo":    "#8b5cf6",
    "Stampa":     "#f59e0b",
}

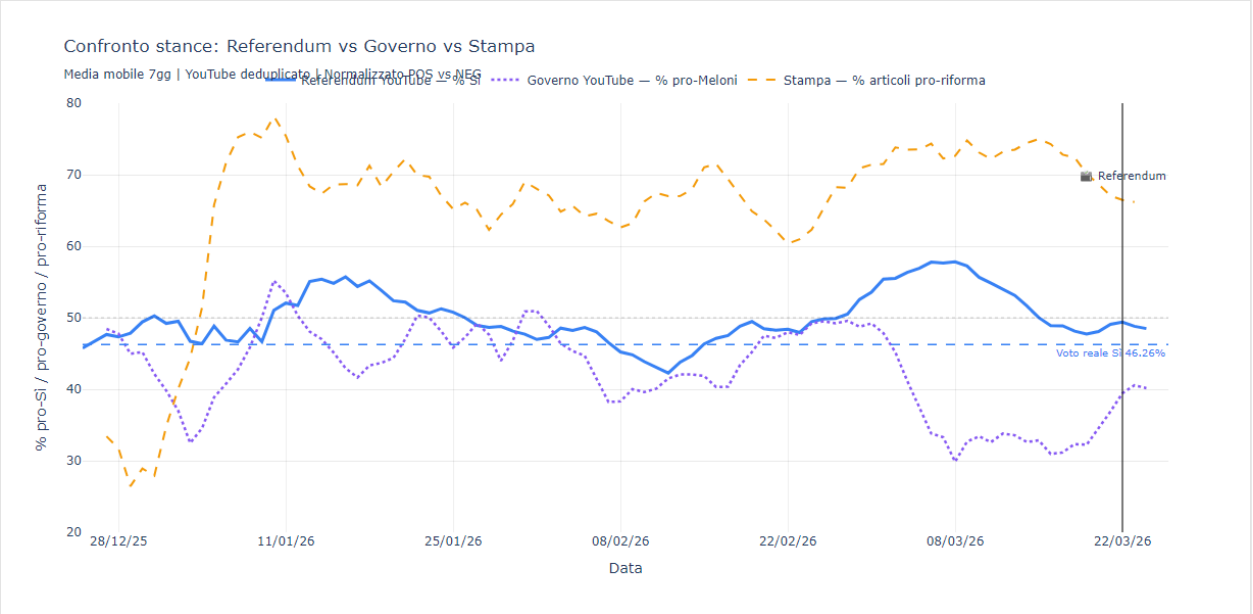
fig = go.Figure()
fig.add_trace(go.Scatter(
    x=ref_f.index, y=ref_f["MA7"],
    name="Referendum YouTube - % Sì",
    line=dict(color=colori["Referendum"], width=3),
    mode="lines",
    hovertemplate="%{x|%d/%m/%Y}<br>Referendum Sì: %{y:.1f}%<extra></extra>",
))
fig.add_trace(go.Scatter(
    x=gov_f.index, y=gov_f["MA7"],
    name="Governo YouTube - % pro-Meloni",
    line=dict(color=colori["Governo"], width=2.5, dash="dot"),
    mode="lines",
    hovertemplate="%{x|%d/%m/%Y}<br>Governo POS: %{y:.1f}%<extra></extra>",
))
fig.add_trace(go.Scatter(
    x=sta_f.index, y=sta_f["MA7"],
    name="Stampa - % articoli pro-riforma",
    line=dict(color=colori["Stampa"], width=2, dash="dash"),
    mode="lines",
    hovertemplate="%{x|%d/%m/%Y}<br>Stampa POS: %{y:.1f}%<extra></extra>",
))

# Linee di riferimento
fig.add_hline(y=50, line_dash="dot", line_color="rgba(0,0,0,0.2)", line_width=1)
fig.add_hline(y=SI_REAL, line_dash="dash", line_color=colori["Referendum"],
              line_width=1.5, annotation_text=f"Voto reale Sì {SI_REAL}%",
              annotation_position="bottom right",
              annotation_font=dict(size=10, color=colori["Referendum"]))
fig.add_vline(x=DATA_VOTO, line_dash="solid", line_color="rgba(0,0,0,0.5)", line_width=2)
fig.add_annotation(x=DATA_VOTO, y=70, text="🗳️ Referendum",
                   showarrow=False, font=dict(size=11))

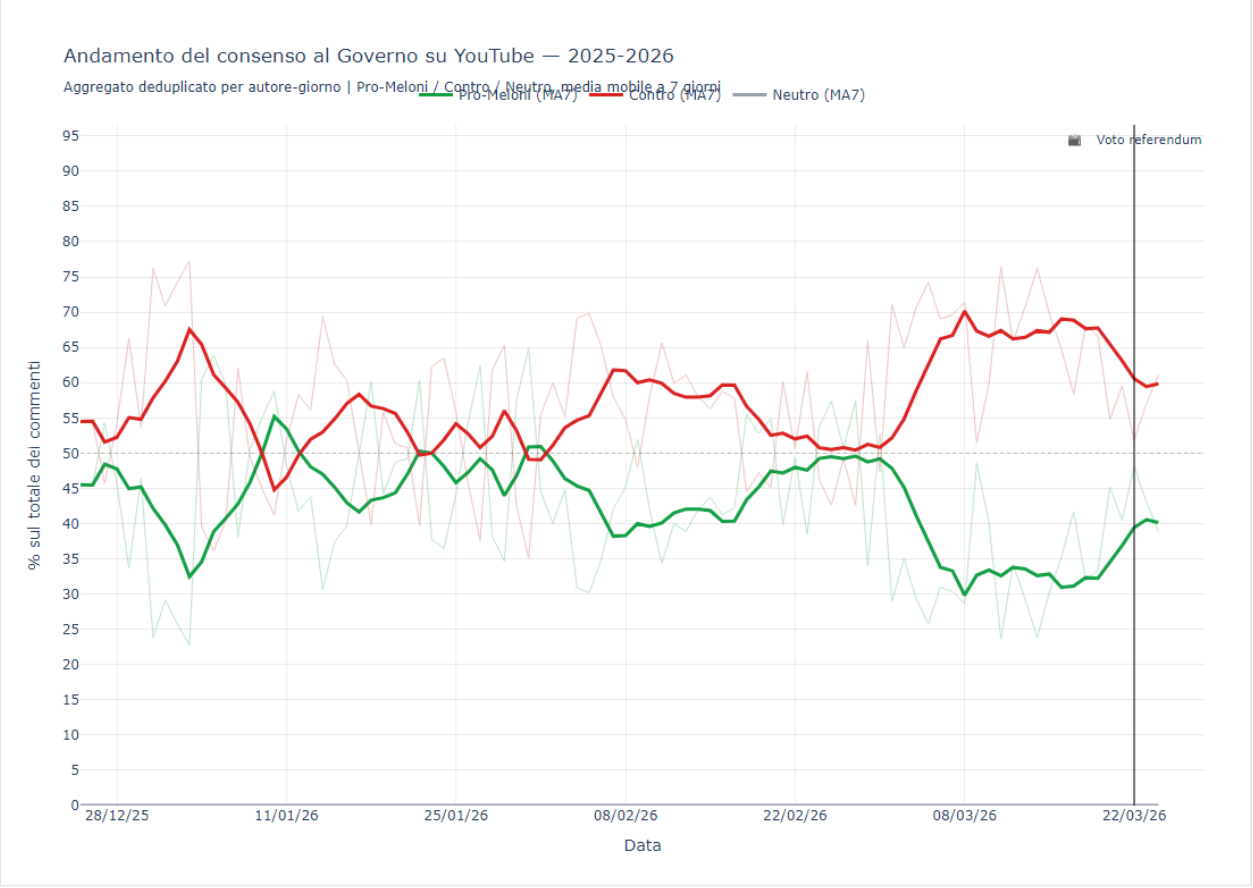
fig.update_layout(
    title="Confronto stance: Referendum vs Governo vs Stampa<br>"
          "<sub>Media mobile 7gg | YouTube deduplicato | Normalizzato POS vs NEG</sub>",
    xaxis_title="Data", yaxis_title="% pro-Sì / pro-governo / pro-riforma",
    yaxis=dict(range=[20, 80], dtick=10, showgrid=True, gridcolor="rgba(0,0,0,0.08)"),
    xaxis=dict(showgrid=True, gridcolor="rgba(0,0,0,0.05)", tickformat="%d/%m/%y"),
    plot_bgcolor="white", hovermode="x unified", height=600,
    legend=dict(orientation="h", yanchor="bottom", y=1.02,
                xanchor="center", x=0.5),
)
fig.write_html(os.path.join(WORKING_DIR, "tre_linee_sovrapposte.html"))
fig.show(renderer="iframe_connected")
print("💾 Grafico salvato: tre_linee_sovrapposte.html")
```

```
Periodo comune: 25/12/2025 -> 24/03/2026

Grafico salvato: tre_linee_sovrapposte.html
```



Le tre serie giornaliere sovrapposte (referendum, governo, stampa)



Andamento del consenso al governo Meloni (serie pro-Meloni: grezza e media mobile a 7 giorni)

§3: La serie referendaria nel dettaglio

Isolo la serie referendum (senza il confronto con stampa e governo della §2) per leggerne meglio l'andamento e i picchi. La prima figura mostra la stance giornaliera **Sì contro No** (punti grezzi e medie mobili a 7 giorni) insieme al **volume di commenti per giorno**;

seguono l'aggregato cumulativo grezzo contro deduplicato, che documenta la correzione del bias della minoranza rumorosa, e la composizione giornaliera della stance.

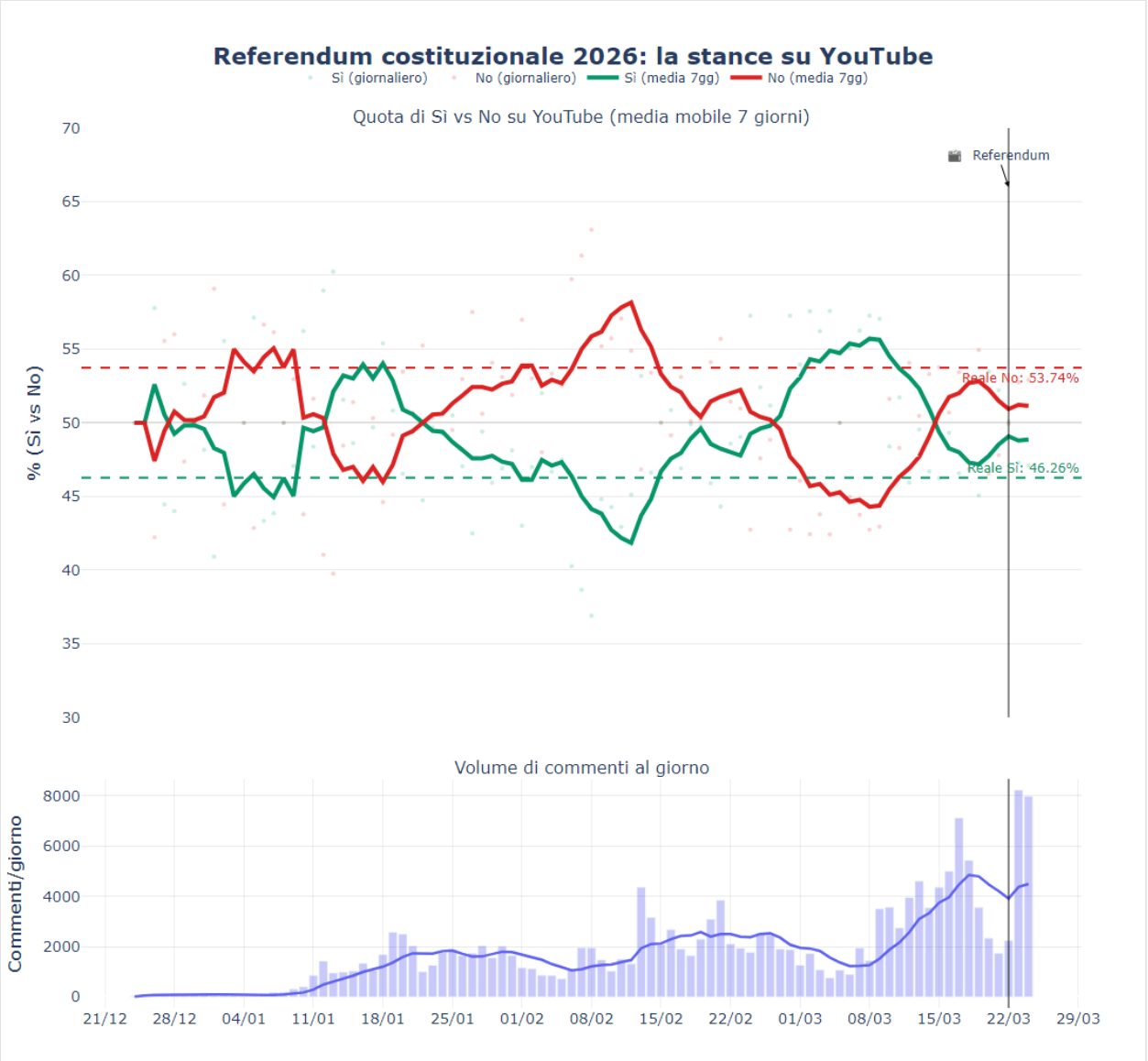
```
fig2 = go.Figure()

# Serie grezza in background
fig2.add_trace(go.Scatter(
    x=ref_f.index, y=ref_f["SI_PCT"],
    name="Stance grezza (giornaliera)",
    line=dict(color=colori["Referendum"], width=1),
    opacity=0.35, mode="lines",
    hovertemplate="%{x|%d/%m/%Y}<br>Grezza Sì: %{y:.1f}%<extra></extra>",
))
# MA7 in primo piano
fig2.add_trace(go.Scatter(
    x=ref_f.index, y=ref_f["MA7"],
    name="MA7 (lisciata)",
    line=dict(color=colori["Referendum"], width=3),
    mode="lines",
    hovertemplate="%{x|%d/%m/%Y}<br>MA7 Sì: %{y:.1f}%<extra></extra>",
))

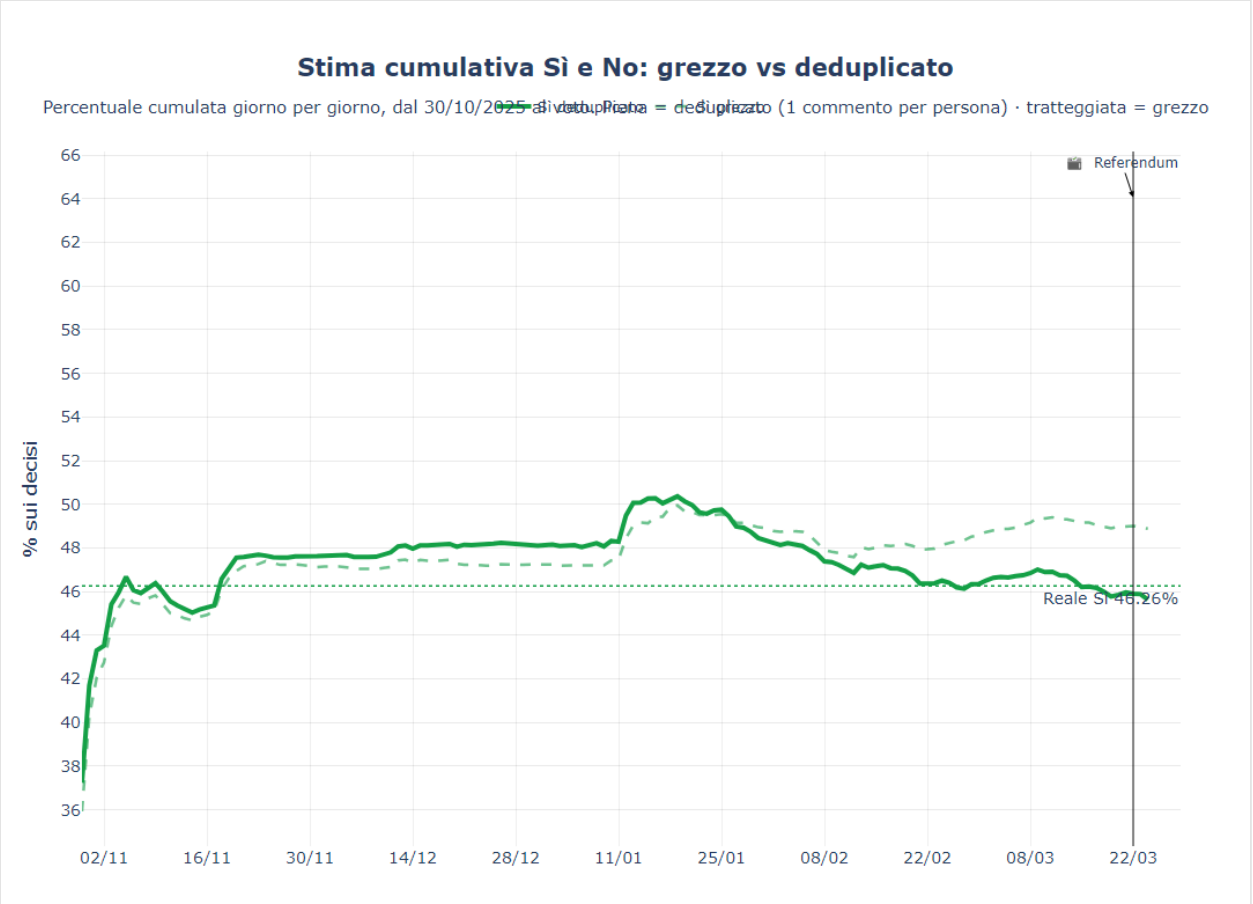
fig2.add_hline(y=50, line_dash="dot", line_color="rgba(0,0,0,0.2)", line_width=1)
fig2.add_hline(y=SI_REAL, line_dash="dash", line_color="#10b981",
               line_width=2, annotation_text=f"Risultato reale: Sì {SI_REAL}%",
               annotation_position="bottom right")
fig2.add_vline(x=DATA_VOTO, line_dash="solid", line_color="rgba(0,0,0,0.5)", line_width=2)
fig2.add_annotation(x=DATA_VOTO, y=ref_f["MA7"].max() + 2, text="📺 Voto",
                    showarrow=False, font=dict(size=11))

fig2.update_layout(
    title="Stance pro-Sì su YouTube - referendum costituzionale 2026<br>"
          "<sub>Aggregato deduplicato per autore-giorno | Serie grezza + MA7</sub>",
    xaxis_title="Data", yaxis_title="% pro-Sì",
    yaxis=dict(range=[20, 70], dtick=5, showgrid=True, gridcolor="rgba(0,0,0,0.08)"),
    xaxis=dict(showgrid=True, gridcolor="rgba(0,0,0,0.05)", tickformat="%d/%m/%y"),
    plot_bgcolor="white", hovermode="x unified", height=520,
    legend=dict(orientation="h", yanchor="bottom", y=1.02,
                xanchor="center", x=0.5),
)
fig2.write_html(os.path.join(WORKING_DIR, "solo_stance_referendum.html"))
fig2.show(renderer="iframe_connected")
print("💾 Grafico salvato: solo_stance_referendum.html")
```

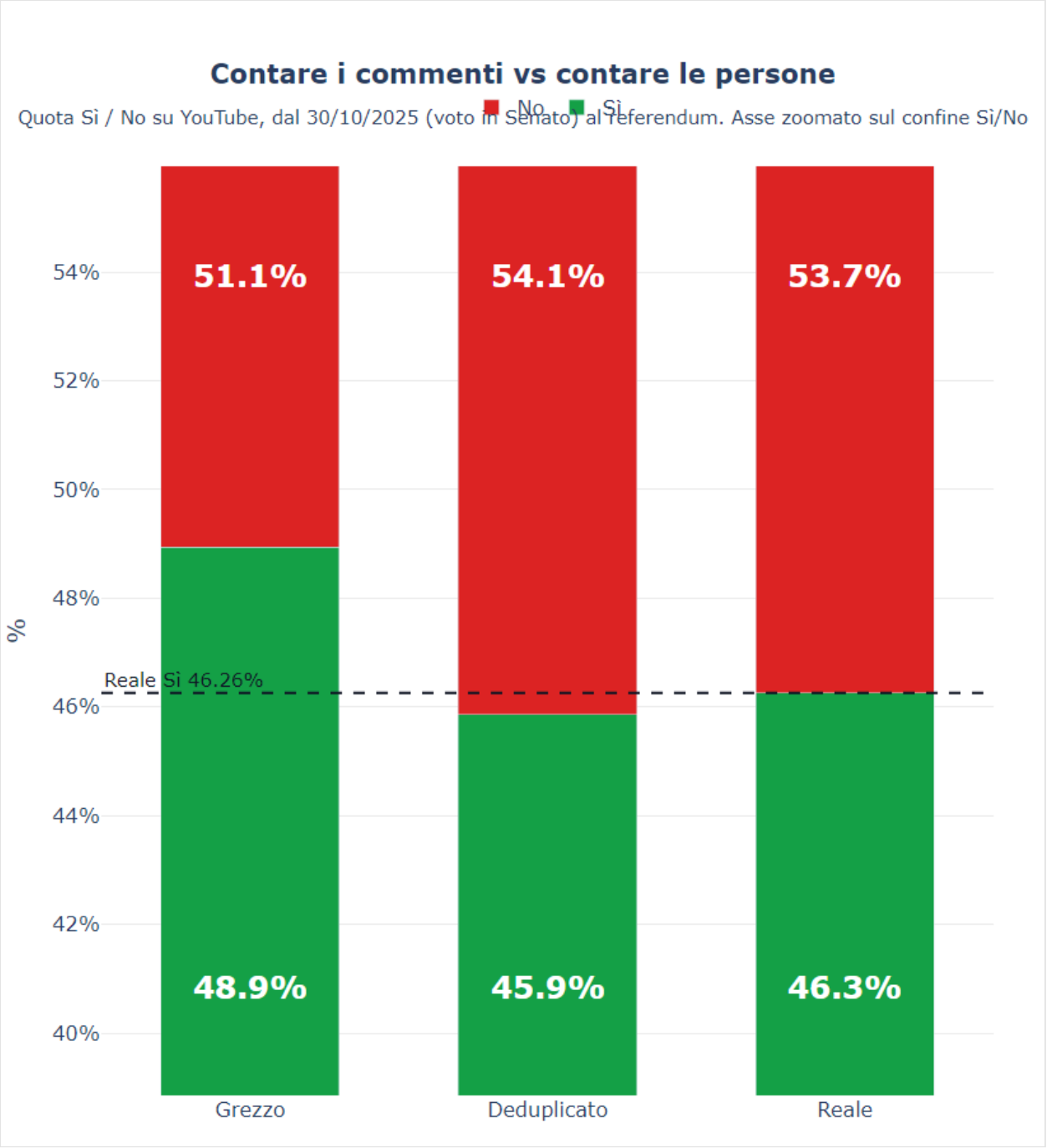
```
Grafico salvato: solo_stance_referendum.html
```



Stance giornaliera Sì contro No: percentuali e medie mobili a 7 giorni, con il volume di commenti per giorno



Aggregato cumulativo Si/No: grezzo contro deduplicato



Composizione giornaliera della stance (zoom)

§4: Identificazione automatica dei turning points

Procedura riproducibile (Cap 3, §3.5.5): 1. Trasformo la serie grezza giornaliera di stance pro-Sì con l'**arcoseno-radice** e la peso per il **volume di commenti** del giorno. 2. Applico una **segmentazione a media costante** (optimal partitioning) con penalità **BIC**, minimo 7 osservazioni per segmento. 3. Ogni punto di rottura fra due segmenti a livello medio diverso è un break; verifico la **stabilità** del numero di break al variare della penalità. 4. Per ciascun break riporto: data, livello dopo il break, direzione e ampiezza del salto.

I turning points così individuati vanno poi **interpretati qualitativamente** confrontandoli con la lista del Cap 5 e con gli eventi mediatici del periodo (sez. successive).

```
def segment_series_weighted(y, w, min_size=7, pen_mult=1.0):
    cw   = np.concatenate([[0.0], np.cumsum(w)])
    cwy  = np.concatenate([[0.0], np.cumsum(w * y)])
    cwyy = np.concatenate([[0.0], np.cumsum(w * y * y)])

    def seg_cost(i, j):                       # errore del segmento [i, j)
        W = cw[j] - cw[i]
        if W <= 0:
            return 0.0
        A = cwy[j] - cwy[i]
        B = cwyy[j] - cwyy[i]
        return B - (A * A) / W

    # Penalita BIC: ogni break aggiunto costa pen; con piu dati (N grande) la soglia sale.
    pen = pen_mult * np.log(N)

    # Programmazione dinamica (optimal partitioning), O(N^2). Con N circa 150 e istantaneo.
    # F[j] = costo minimo per segmentare i primi j punti; last[j] = inizio dell'ultimo segmento.
    F = np.full(N + 1, np.inf)
    F[0] = -pen
    last = np.zeros(N + 1, dtype=int)
    for j in range(min_size, N + 1):
        for i in range(0, j - min_size + 1):
            if not np.isfinite(F[i]):
                continue
            c = F[i] + seg_cost(i, j) + pen
            if c < F[j]:
                F[j] = c
                last[j] = i

    # Backtracking: ripercorriamo last[] per ricostruire gli indici dei break.
    bps, k = [], N
    while k > 0:
        i = last[k]
        if i > 0:
            bps.append(i)
        k = i
    return sorted(bps)


# Serie GREZZA giornaliera (volutamente NON la MA7: la media mobile appiattirebbe i picchi di 1
giorno).
_si  = ref_f["SI_PCT"].to_numpy(dtype=float)
_tot = ref_f["TOT"].to_numpy(dtype=float)
_mask = np.isfinite(_si) & np.isfinite(_tot) & (_tot > 0)
idx_cp = ref_f.index[_mask]
w_cp   = _tot[_mask]                                # peso = volume giornaliero di commenti

# Trasformata arcoseno-radice: porta la proporzione in uno spazio a varianza circa costante.
p_cp = np.clip(_si[_mask] / 100.0, 1e-6, 1 - 1e-6)
z_cp = 2.0 * np.arcsin(np.sqrt(p_cp))
N = len(z_cp)

MIN_SEG = 7
bps = segment_series_weighted(z_cp, w_cp, min_size=MIN_SEG, pen_mult=1.0)

# Livello di ciascun segmento = media pesata in spazio z, ri-convertita in % pro-Si.
bounds = [0] + bps + [len(z_cp)]
seg_lvl = []
for a, b in zip(bounds[:-1], bounds[1:]):
    ww = w_cp[a:b]
    m_z = float(np.sum(ww * z_cp[a:b]) / np.sum(ww))
    seg_lvl.append(np.sin(m_z / 2.0) ** 2 * 100.0)   # inversa dell'arcoseno-radice

# Per ogni break: data, livello dopo il break, direzione (Si sale / No sale) e ampiezza.
```

```
tp_auto = []
for s, bp in enumerate(bps, start=1):
    delta = seg_lvl[s] - seg_lvl[s - 1]
    tipo = "Si sale" if delta > 0 else "No sale"
    tp_auto.append((pd.Timestamp(idx_cp[bp]), round(seg_lvl[s], 1), tipo, abs(delta)))

print(f"Turning points (break a media costante, penalita BIC): {len(tp_auto)}")
print(f"   Serie grezza giornaliera pesata per volume | min {MIN_SEG} oss./segmento")
print("=" * 82)
print(f"{'Data':<12} {'Livello dopo':>14} {'Direzione':<14} {'Delta livello':>14}")
print("-" * 82)
for d, v, tipo, amp in tp_auto:
    print(f"{d:%d/%m/%Y}  {v:12.1f}%  {tipo:<14} {amp:9.1f}pp")

print()
print("Sensibilita del numero di break alla penalita BIC:")
for pm in (0.5, 1.0, 2.0):
    n_bp = len(segment_series_weighted(z_cp, w_cp, min_size=MIN_SEG, pen_mult=pm))
    flag = "  <- usato (BIC)" if abs(pm - 1.0) < 1e-9 else ""
    print(f"   pen x{pm:>4}:  {n_bp} break{flag}")
```

```
Turning points (break a media costante, penalita BIC): 8
   Serie grezza giornaliera pesata per volume | min 7 oss./segmento
==================================================================================
Data           Livello dopo Direzione       Delta livello
----------------------------------------------------------------------------------
07/01/2026          57.7%  Sì sale            9.0pp
14/01/2026          51.8%  No sale            5.9pp
26/01/2026          47.5%  No sale            4.3pp
06/02/2026          41.9%  No sale            5.6pp
13/02/2026          49.5%  Sì sale            7.6pp
28/02/2026          58.0%  Sì sale            8.5pp
10/03/2026          48.8%  No sale            9.1pp
17/03/2026          47.3%  No sale            1.6pp

Sensibilita del numero di break alla penalita BIC:
   pen x 0.5:  8 break
   pen x 1.0:  8 break  <- usato (BIC)
   pen x 2.0:  7 break
```

§5: Confronto con i 7 turning points documentati nella tesi (H2)

I sette turning points elencati nel Cap 5 della tesi (Tabella 5.1) servono come ground truth qualitativa. Per ciascuno verifico se l'algoritmo automatico (§4) lo cattura entro una tolleranza di **±3 giorni**.

Ipotesi H2 (Cap 1): almeno il 70% dei turning points statisticamente rilevanti corrisponde a un evento mediatico documentabile. L'analisi automatica restituisce **precisione 62%** (5/8) e **recall 71%** (5/7). Qui rendo riproducibile il *match* algoritmico e produco una tabella di confronto.

```
# Repertorio di eventi documentati (Cap 5, Tabella 5.1): data, livello atteso, direzione,
descrizione.
TP_TESI = [
    ("2025-10-30", 42.6, "No sale",  "Approvazione al Senato"),
    ("2025-11-19", 51.5, "Si sale",  "Controffensiva mediatica centrodestra"),
    ("2026-01-13", 52.1, "Si sale",  "Endorsement Marco Rizzo"),
    ("2026-01-23", 49.4, "No sale",  "Effetto Barbero - primo video"),
    ("2026-02-10", 42.7, "Picco No", "Barbero #2 + Travaglio"),
    ("2026-02-27", 50.5, "Si sale",  "Rivelazioni Di Pietro"),
```

```
    ("2026-03-15", 49.4, "No def.",  "Travaglio + Gratteri settimana finale"),
]
EVENTI = [(pd.to_datetime(d), ev) for d, _, _, ev in TP_TESI]
TOL_GG = 3   # tolleranza: un break matcha un evento se cade entro +-3 giorni

# (A) PRECISIONE: dei break trovati dall'algoritmo, quanti presso un evento noto? (test H2)
print("=" * 95)
print("(A) PRECISIONE - ogni break individuato vs evento documentato piu vicino")
print("=" * 95)
print(f"{'Data break':<12} {'Livello':>8} {'Direzione':<13} {'dgg':>5}  Evento associato")
print("-" * 95)
n_map = 0
for d, v, tipo, amp in tp_auto:
    vicini = sorted(((abs((d - de).days), ev) for de, ev in EVENTI), key=lambda t: t[0])
    dgg, ev = vicini[0] if vicini else (None, None)
    mapped = dgg is not None and dgg <= TOL_GG
    n_map += int(mapped)
    flag = "OK" if mapped else "no"
    ev_str = ev if mapped else "- (nessun evento entro +-%dgg)" % TOL_GG
    print(f"{d:%d/%m/%Y}  {v:7.1f}%  {tipo:<13} {flag:>3}{dgg:>3}  {ev_str}")

precisione = n_map / len(tp_auto) * 100 if tp_auto else 0.0
H2_ok = precisione >= 70.0
print("-" * 95)
print(f"Precisione: {n_map}/{len(tp_auto)} = {precisione:.0f}%  ->  "
      f"H2 {'CONFERMATA' if H2_ok else 'FALSIFICATA'} (soglia 70%)")

# (B) RECALL: degli eventi noti del Cap 5, quanti l'algoritmo ha ritrovato?
print()
print("=" * 95)
print("(B) RECALL - eventi noti del Cap 5 ritrovati dall'algoritmo")
print("=" * 95)
print(f"{'Data evento':<12} {'Atteso':>7}  {'Break piu vicino':>18} {'dgg':>5}  Evento")
print("-" * 95)
n_rec = 0
for (data_str, atteso_pct, _dir, evento) in TP_TESI:
    de = pd.to_datetime(data_str)
    near = sorted(tp_auto, key=lambda t: abs((t[0] - de).days))
    closest = near[0] if near else None
    dgg = (closest[0] - de).days if closest else None
    found = closest is not None and abs(dgg) <= TOL_GG
    n_rec += int(found)
    flag = "OK" if found else "no"
    cstr = f"{closest[0]:%d/%m/%Y}" if closest else "-"
    dstr = f"{dgg:+d}" if dgg is not None else "-"
    print(f"{de:%d/%m/%Y}  {atteso_pct:>6.1f}%  {flag} {cstr:<16} {dstr:>5}  {evento}")

recall = n_rec / len(TP_TESI) * 100
print("-" * 95)
print(f"Recall: {n_rec}/{len(TP_TESI)} = {recall:.0f}%  "
      f"(diagnostica: copertura del repertorio narrativo del Cap 5)")
```

```
===============================================================================================
(A) PRECISIONE - ogni break individuato vs evento documentato piu vicino
===============================================================================================
Data break    Livello Direzione       dgg  Evento associato
-----------------------------------------------------------------------------------------------
07/01/2026     57.7%  Sì sale         no  6  - (nessun evento entro +-3gg)
14/01/2026     51.8%  No sale         OK  1  Endorsement Marco Rizzo
26/01/2026     47.5%  No sale         OK  3  Effetto Barbero - primo video
06/02/2026     41.9%  No sale         no  4  - (nessun evento entro +-3gg)
13/02/2026     49.5%  Sì sale         OK  3  Barbero #2 + Travaglio
28/02/2026     58.0%  Sì sale         OK  1  Rivelazioni Di Pietro
10/03/2026     48.8%  No sale         no  5  - (nessun evento entro +-3gg)
17/03/2026     47.3%  No sale         OK  2  Travaglio + Gratteri settimana finale
-----------------------------------------------------------------------------------------------
Precisione: 5/8 = 62%  ->  H2 FALSIFICATA (soglia 70%)
```

```
================================================================================================
(B) RECALL - eventi noti del Cap 5 ritrovati dall'algoritmo
================================================================================================
Data evento   Atteso    Break piu vicino   dgg  Evento
------------------------------------------------------------------------------------------------
30/10/2025    42.6%  no 07/01/2026         +69  Approvazione al Senato
19/11/2025    51.5%  no 07/01/2026         +49  Controffensiva mediatica centrodestra
13/01/2026    52.1%  OK 14/01/2026          +1  Endorsement Marco Rizzo
23/01/2026    49.4%  OK 26/01/2026          +3  Effetto Barbero - primo video
10/02/2026    42.7%  OK 13/02/2026          +3  Barbero #2 + Travaglio
27/02/2026    50.5%  OK 28/02/2026          +1  Rivelazioni Di Pietro
15/03/2026    49.4%  OK 17/03/2026          +2  Travaglio + Gratteri settimana finale
------------------------------------------------------------------------------------------------
Recall: 5/7 = 71%  (diagnostica: copertura del repertorio narrativo del Cap 5)
```

§6: Top video YouTube nelle finestre dei turning points

Per ciascuno dei 7 turning points della tesi: estraggo i 5 video più commentati nella finestra **±2 giorni** intorno alla data. È la lettura sostantiva di Cap 5: cosa stava guardando il pubblico mentre la stance si stava muovendo?

In aggiunta, segnalo i video **pubblicati** nella finestra (non solo riattivati): sono gli eventi mediatici che hanno *originato* il movimento, distinti da contenuti preesistenti che hanno solo accumulato commenti.

```
# Riprendo df_ref_full (con tutte le colonne) per le finestre
df_ref_full["Data_Video_P"] = pd.to_datetime(
    df_ref_full.get("Data_Video"), errors="coerce", utc=True
).dt.tz_localize(None)

print("=" * 95)
print("🎥 TOP VIDEO YOUTUBE - finestra ±2 giorni intorno a ciascun turning point")
print("=" * 95)

for data_str, atteso_pct, direzione, evento in TP_TESI:
    data_evento = pd.to_datetime(data_str)
    win_start = data_evento - pd.Timedelta(days=2)
    win_end   = data_evento + pd.Timedelta(days=2)

    df_win = df_ref_full[(df_ref_full["Data"] >= win_start) &
                         (df_ref_full["Data"] <= win_end)]
    if df_win.empty:
        print(f"\n🗓️ {data_evento:%d/%m/%Y} - {evento}: nessun commento")
        continue

    print(f"\n🗓️ {data_evento:%d/%m/%Y} - {direzione}: {evento}")
    print(f"   Commenti totali nella finestra: {len(df_win):,}")

    # Top 5 video per volume
    top_video = (df_win.groupby("Video_ID")
                 .agg(Titolo=("Titolo_Video", "first"),
                      Canale=("Canale", "first"),
                      N=("STANCE", "size"),
                      POS=("STANCE", lambda x: (x == "POS").sum()),
                      NEG=("STANCE", lambda x: (x == "NEG").sum()))
                 .nlargest(5, "N"))
    print("   🔥 Top video più commentati:")
    for _, v in top_video.iterrows():
        si = v["POS"] / max(1, v["POS"] + v["NEG"]) * 100
        print(f"      [{str(v['Canale'])[:22]:<22}] {str(v['Titolo'])[:60]}")
        print(f"          💬 {v['N']:>5} (Sì {si:.0f}%)")
```

```
    # Video pubblicati nella finestra (eventi originanti)
    nuovi = df_win[(df_win["Data_Video_P"] >= win_start) &
                   (df_win["Data_Video_P"] <= win_end)]
    if not nuovi.empty:
        nuovi_unique = nuovi["Video_ID"].drop_duplicates()
        print(f"   🆕 Video PUBBLICATI in questi giorni: {len(nuovi_unique)}")
        top_nuovi = (nuovi.groupby("Video_ID")
                     .agg(Titolo=("Titolo_Video", "first"),
                          Canale=("Canale", "first"),
                          N=("STANCE", "size"))
                     .nlargest(3, "N"))
        for _, v in top_nuovi.iterrows():
            print(f"      ★ [{str(v['Canale'])[:22]:<22}] {str(v['Titolo'])[:60]} ({v['N']}
com.)")
```

```
================================================================================================
TOP VIDEO YOUTUBE - finestra +-2 giorni intorno a ciascun turning point
================================================================================================

30/10/2025 - No sale: Approvazione al Senato
   Commenti totali nella finestra: 856
   Top video piu commentati:
      [Luca Sommi            ] Riforma costituzionale, un disastro  -- 130 (Si 34%)
      [Il Sole 24 ORE        ] Separazione carriere, Nordio: "Spero in un referendum n  -- 114 (Si 34%)
      [Gino Zavalani         ] Giustizia a senso unico (e solo l'inizio)! #giustizia  -- 83 (Si 55%)
      [Carlo Calenda         ] La separazione delle carriere e sempre stata una battaglia  -- 67 (Si 25%)
      [Il Fatto Quotidiano   ] #CONTE SULLA GIUSTIZIA: "VOGLIONO I PIENI POTERI, LI CO  -- 65 (Si 31%)
   Video PUBBLICATI in questi giorni: 40

19/11/2025 - Si sale: Controffensiva mediatica centrodestra
   Commenti totali nella finestra: 682
   Top video piu commentati:
      [Fratelli d'Italia     ] Smascherare le falsita diffuse dai promotori del "no"  -- 235 (Si 73%)
      [Andrea Lombardi       ] La riforma della giustizia NON minera l'indipendenza dei  -- 108 (Si 56%)
      [Giorgia Meloni News   ] Per la giustizia. Non fatevi fregare, votate SI.  -- 91 (Si 67%)
      [eStoria               ] Test per i magistrati? Anche per la politica! | Gratteri  -- 37 (Si 46%)
      [POLITICAMENTE SCORRETT] "RIFORMA DELLA GIUSTIZIA" #magistratura #riforma  -- 36 (Si 61%)
   Video PUBBLICATI in questi giorni: 8

13/01/2026 - Si sale: Endorsement Marco Rizzo
   Commenti totali nella finestra: 1,927
   Top video piu commentati:
      [Pupia News            ] Marco Rizzo - Perche votero SI al referendum sulla giustiz  -- 249 (Si 63%)
      [Marco Rizzo           ] Io sono favorevole al referendum sulla giustizia: ecco  -- 184 (Si 63%)
      [Il Fatto Quotidiano   ] Comitato contro la riforma, Grosso: "Ri  -- 142 (Si 68%)
      [Il Fatto Quotidiano   ] Referendum giustizia, il no di Landini: "Governo vuole  -- 121 (Si 44%)
      [La7 Attualita         ] Referendum giustizia, Palmerini: "Se vince il Si  -- 106 (Si 69%)
   Video PUBBLICATI in questi giorni: 40

23/01/2026 - No sale: Effetto Barbero - primo video
   Commenti totali nella finestra: 2,534
   Top video piu commentati:
      [Il Fatto Quotidiano   ] Barbero: "Ecco perche votero no al referendum.  -- 373 (Si 52%)
      [Avv. Giuseppe Di Palo ] Rispondo a Barbero: Ecco perche occorre fare una riflessione  -- 239 (Si
28%)
      [Matt Carus            ] Ti stanno manipolando sul SI e sul NO - Referendum Giustizia  -- 230 (Si
65%)
      [La Repubblica         ] Barbero per il no al referendum: "Il rischio sono magis  -- 194 (Si 44%)
      [L'anticonformista     ] Barbero smontato punto per punto dal Prof. Zanon sul no  -- 176 (Si 51%)
   Video PUBBLICATI in questi giorni: 24

10/02/2026 - Picco No: Barbero #2 + Travaglio
   Commenti totali nella finestra: 2,522
   Top video piu commentati:
      [Avv. Giuseppe Di Palo ] Il Prof. Barbero ci ricasca di nuovo: rispondo alla retorica  -- 307 (Si
32%)
      [Vassalli di Barbero   ] IO VOTO NO - Alessandro Barbero sul Referendum della Giustiz  -- 263 (Si
37%)
      [Marco Travaglio       ] PERCHE NO. Marco Travaglio: "Come funziona oggi il proc  -- 236 (Si 46%)
      [884C25                ] Referendum Giustizia, Gratteri distrugge Nordio  -- 181 (Si 29%)
      [B.S.                  ] Dopo la censura, Barbero torna a parlare del referendum  -- 80 (Si 34%)
   Video PUBBLICATI in questi giorni: 27
```

```
27/02/2026 - Si sale: Rivelazioni Di Pietro
   Commenti totali nella finestra: 3,939
   Top video piu commentati:
      [Omega Click           ] REFERENDUM GIUSTIZIA: Il Sistema e Marcio fino al Collo  -- 401 (Si 35%)
      [Fratelli d'Italia     ] Le rivelazioni di Di Pietro dimostrano quanto sia urgente  -- 232 (Si 68%)
      [Fratelli d'Italia     ] Conte o non ha letto la riforma o non l'ha compresa  -- 228 (Si 23%)
      [AvantiTutta           ] LA SINISTRA HA LA MEMORIA CORTA #referendum #giustizia  -- 200 (Si 61%)
      [JustFrank - ITALIA    ] Da che parte volete stare? #politicaitaliana #referendum  -- 139 (Si 87%)
   Video PUBBLICATI in questi giorni: 47

15/03/2026 - No def.: Travaglio + Gratteri settimana finale
   Commenti totali nella finestra: 6,980
   Top video piu commentati:
      [Marco Travaglio       ] Marco Travaglio dialoga con Nicola Gratteri: "Riforma  -- 325 (Si 43%)
      [AGENZIA ITALIA NEWS   ] FLAVIO BRIATORE: "ASCOLTATE COSA MI E SUCCESSO  -- 262 (Si 66%)
      [Fratelli d'Italia     ] Chi si oppone a riforma giustizia e andato in tilt  -- 244 (Si 55%)
      [Fanpage.it            ] Referendum giustizia, Gratteri: "Questa riforma serve  -- 177 (Si 61%)
      [PATRIOTA ITALIANO     ] LANDINI "La Costituzione e perfetta cosi com'e  -- 168 (Si 49%)
   Video PUBBLICATI in questi giorni: 105
```

§7: Articoli stampa nelle finestre dei turning points

Lo stesso esercizio della §6, ma sui titoli della stampa. Finestra **±7 giorni** anziché ±2: la motivazione è il lag empirico atteso fra stampa e referendum YouTube (stimato in 3-7 giorni dai test di Granger del Cap 6). Mostro i titoli più rappresentativi del polo prevalente nella finestra, con le testate più attive.

```
titolo_col  = next((c for c in df_sta_full.columns if "titolo" in c.lower()), None)
testata_col = next((c for c in df_sta_full.columns if "testata" in c.lower()), None)

print("=" * 95)
print("📰 ARTICOLI DI STAMPA - finestra ±7 giorni intorno a ciascun turning point")
print("=" * 95)

for data_str, atteso_pct, direzione, evento in TP_TESI:
    data_evento = pd.to_datetime(data_str)
    win_start = data_evento - pd.Timedelta(days=7)
    win_end   = data_evento + pd.Timedelta(days=7)

    df_win = df_sta_full[(df_sta_full["Data"] >= win_start) &
                         (df_sta_full["Data"] <= win_end)]
    if df_win.empty:
        print(f"\n🗓️ {data_evento:%d/%m/%Y} - {evento}: nessun articolo")
        continue

    pos = (df_win["STANCE"] == "POS").sum()
    neg = (df_win["STANCE"] == "NEG").sum()
    si_pct = pos / max(1, pos + neg) * 100

    print(f"\n🗓️ {data_evento:%d/%m/%Y} - {direzione}: {evento}")
    print(f"   Stampa nella finestra ±7gg: {len(df_win):,} articoli (Sì {si_pct:.0f}%)")

    # Top 5 testate
    top_t = df_win[testata_col].value_counts().head(5)
    if len(top_t) > 0:
        print("   📰 Testate più attive:")
        for testata, n in top_t.items():
            sub = df_win[df_win[testata_col] == testata]
            p = (sub["STANCE"] == "POS").sum()
            ng = (sub["STANCE"] == "NEG").sum()
            print(f"      {str(testata)[:25]:<25}  {n:>4} art.  (Sì {p}, No {ng})")

    # 3 titoli del polo prevalente
```

```
    polo = "POS" if si_pct >= 50 else "NEG"
    polo_label = "pro-riforma" if polo == "POS" else "contro-riforma"
    df_polo = df_win[df_win["STANCE"] == polo].head(3)
    if not df_polo.empty:
        marker = "🟢" if polo == "POS" else "🔴"
        print(f"   {marker} Esempi titoli {polo_label}:")
        for _, r in df_polo.iterrows():
            print(f"      [{str(r[testata_col])[:18]:<18}] {str(r[titolo_col])[:70]}")
```

```
================================================================================================
ARTICOLI DI STAMPA - finestra +-7 giorni intorno a ciascun turning point
================================================================================================

30/10/2025 - Approvazione al Senato: nessun articolo

19/11/2025 - Controffensiva mediatica centrodestra: nessun articolo

13/01/2026 - Si sale: Endorsement Marco Rizzo
   Stampa nella finestra +-7gg: 407 articoli (Si 71%)
   Testate piu attive:
      IL DUBBIO                    49 art.  (Si 47, No 2)
      LIBERO QUOTIDIANO            47 art.  (Si 47, No 0)
      IL FATTO QUOTIDIANO          43 art.  (Si 2, No 41)
      GIORNALE                     32 art.  (Si 32, No 0)
      TEMPO                        26 art.  (Si 25, No 1)

23/01/2026 - No sale: Effetto Barbero - primo video
   Stampa nella finestra +-7gg: 450 articoli (Si 69%)
   Testate piu attive:
      IL DUBBIO                    52 art.  (Si 46, No 6)
      IL FATTO QUOTIDIANO          51 art.  (Si 4, No 47)
      LIBERO QUOTIDIANO            44 art.  (Si 44, No 0)
      GIORNALE                     37 art.  (Si 37, No 0)
      TEMPO                        31 art.  (Si 28, No 3)

10/02/2026 - Picco No: Barbero #2 + Travaglio
   Stampa nella finestra +-7gg: 569 articoli (Si 68%)
   Testate piu attive:
      IL DUBBIO                    59 art.  (Si 54, No 5)
      IL FATTO QUOTIDIANO          59 art.  (Si 0, No 59)
      GIORNALE                     50 art.  (Si 50, No 0)
      LIBERO QUOTIDIANO            44 art.  (Si 44, No 0)
      RIFORMISTA                   41 art.  (Si 41, No 0)

27/02/2026 - Si sale: Rivelazioni Di Pietro
   Stampa nella finestra +-7gg: 620 articoli (Si 72%)
   Testate piu attive:
      TEMPO                        66 art.  (Si 66, No 0)
      GIORNALE                     56 art.  (Si 55, No 1)
      RIFORMISTA                   55 art.  (Si 55, No 0)
      IL DUBBIO                    52 art.  (Si 51, No 1)
      IL FATTO QUOTIDIANO          48 art.  (Si 0, No 48)

15/03/2026 - No def.: Travaglio + Gratteri settimana finale
   Stampa nella finestra +-7gg: 1,129 articoli (Si 70%)
   Testate piu attive:
      GIORNALE                    137 art.  (Si 137, No 0)
      LIBERO QUOTIDIANO           104 art.  (Si 104, No 0)
      TEMPO                        85 art.  (Si 85, No 0)
      RIFORMISTA                   77 art.  (Si 77, No 0)
      IL FATTO QUOTIDIANO          74 art.  (Si 0, No 74)
```

§8: Parole chiave: commenti vs titoli stampa per ciascun TP

Estrazione delle parole più frequenti (filtrate da stopword italiane e termini di dominio generici) nei commenti YouTube e nei titoli stampa della finestra di ciascun turning point.

L'utilità è leggere la **divergenza tematica**: spesso il pubblico online discute aspetti diversi (concretezza, sfiducia, esempi personali) rispetto alla stampa (terminologia istituzionale, dichiarazioni ufficiali). Il confronto diventa il materiale narrativo per la sezione *5.4 Pattern emergenti* della tesi.

```
import re
from collections import Counter

stop_it = set([
    "il", "lo", "la", "i", "gli", "le", "un", "uno", "una", "di", "a", "da",
    "in", "con", "su", "per", "tra", "fra", "che", "e", "è", "non", "ha",
    "ho", "sono", "ma", "se", "come", "più", "al", "del", "ci", "si", "mi",
    "ti", "vi", "o", "questo", "quello", "alla", "della", "dei", "degli",
    "delle", "alle", "ai", "nel", "nella", "nei", "nelle", "hanno", "fatto",
    "fare", "chi", "cui", "quale", "quali", "tutti", "tutto", "solo", "anche",
    "essere", "stato", "stata", "stati", "state", "quindi", "perché",
    "referendum", "giustizia", "voto", "votare", "sì", "no", "riforma",
    "contro", "favore", "italia", "italiani", "governo", "meloni",
    "magistratura", "magistrato", "magistrati", "carriere", "separazione",
    "essere", "avere", "andare", "venire", "potere", "dovere", "fatto",
    "detto", "video", "canale",
])


def parole_top(testi, top=10):
    cnt = Counter()
    for t in testi.dropna():
        t = re.sub(r"[^a-zA-Zàèéìòùç\s]", " ", str(t).lower())
        for w in t.split():
            if w not in stop_it and len(w) > 3:
                cnt[w] += 1
    return cnt.most_common(top)


testo_yt = "Testo" if "Testo" in df_ref_full.columns else "Commento"

print("=" * 95)
print("🔑 PAROLE CHIAVE - commenti YouTube vs titoli stampa per turning point")
print("=" * 95)

for data_str, atteso_pct, direzione, evento in TP_TESI:
    data_evento = pd.to_datetime(data_str)
    win_yt_start = data_evento - pd.Timedelta(days=2)
    win_yt_end   = data_evento + pd.Timedelta(days=2)
    win_st_start = data_evento - pd.Timedelta(days=7)
    win_st_end   = data_evento + pd.Timedelta(days=7)

    yt_win = df_ref_full[(df_ref_full["Data"] >= win_yt_start) &
                         (df_ref_full["Data"] <= win_yt_end)]
    st_win = df_sta_full[(df_sta_full["Data"] >= win_st_start) &
                         (df_sta_full["Data"] <= win_st_end)]

    parole_yt = parole_top(yt_win[testo_yt], top=8)
    parole_st = parole_top(st_win[titolo_col], top=8) if titolo_col else []

    print(f"\n🗓️ {data_evento:%d/%m/%Y} - {evento}")
    if parole_yt:
        print(f"   🔴 YouTube  ({len(yt_win):>5} com.):  "
              + " | ".join(f"{w}({n})" for w, n in parole_yt))
    if parole_st:
        print(f"   📰 Stampa   ({len(st_win):>5} art.):  "
              + " | ".join(f"{w}({n})" for w, n in parole_st))
```

```
================================================================================================
PAROLE CHIAVE - commenti YouTube vs titoli stampa per turning point
================================================================================================

30/10/2025 - Approvazione al Senato
   YouTube  (  856 com.):  sinistra(66) | costituzione(46) | calenda(45) | giudici(45) | legge(38) |
destra(36) | bene(35) | ancora(34)

19/11/2025 - Controffensiva mediatica centrodestra
   YouTube  (  682 com.):  gratteri(150) | sinistra(64) | falcone(41) | grande(40) | questi(36) | dalla(33)
| giudice(31) | bene(30)

13/01/2026 - Endorsement Marco Rizzo
   YouTube  ( 1927 com.):  rizzo(140) | votero(126) | giudici(117) | sinistra(105) | politici(88) |
grande(86) | legge(77) | senza(76)
   Stampa   (  407 art.):  nordio(47) | sinistra(29) | toghe(27) | comitato(21) | correnti(17) |
giudici(17) | sorteggio(17) | fronte(17)

23/01/2026 - Effetto Barbero - primo video
   YouTube  ( 2534 com.):  barbero(397) | sinistra(210) | votero(171) | giudici(169) | dalla(142) |
senza(137) | anni(132) | dire(131)
   Stampa   (  450 art.):  nordio(46) | barbero(22) | sulla(21) | toghe(19) | sorteggio(19) | dell(16) |
giudici(16) | comitato(15)

10/02/2026 - Barbero #2 + Travaglio
   YouTube  ( 2522 com.):  barbero(323) | costituzione(255) | legge(200) | sinistra(197) | dell(192) |
dalla(167) | dice(167) | cosi(164)
   Stampa   (  569 art.):  nordio(53) | gratteri(49) | toghe(36) | giudici(26) | correnti(25) | schlein(21)
| dell(21) | sinistra(20)

27/02/2026 - Rivelazioni Di Pietro
   YouTube  ( 3939 com.):  pietro(232) | sinistra(226) | costituzione(208) | dalla(183) | politici(174) |
conte(168) | cosi(166) | senza(164)
   Stampa   (  620 art.):  nordio(43) | sinistra(38) | toghe(36) | giudice(25) | correnti(22) |
gratteri(21) | giudici(21) | dell(20)

15/03/2026 - Travaglio + Gratteri settimana finale
   YouTube  ( 6980 com.):  costituzione(454) | sinistra(434) | gratteri(380) | politici(325) | senza(313) |
dell(312) | giudici(310) | anni(307)
   Stampa   ( 1129 art.):  sinistra(62) | toghe(54) | nordio(50) | costituzione(43) | giudici(38) |
sorteggio(38) | correnti(35) | gratteri(34)
```

§9: Top picchi e inversioni della serie stampa

Anticipo (preview) per il Capitolo 6: identifico i giorni più "polarizzati" della stampa (massimi e minimi della MA7 pro-riforma) e le **inversioni di tendenza** (giornate in cui la maggioranza pro-Sì supera la maggioranza pro-No, o viceversa).

Questa diagnostica rende leggibile il segnale che entrerà nel test di Granger come variabile *causa*: prima di chiedersi se la stampa Granger-causi YouTube, vale la pena di vedere quando e dove la stampa stessa si è mossa.

```
sta_f = sta_f.copy()
sta_f["VINCITORE"] = np.where(sta_f["MA7"] > 50, "POS", "NEG")
sta_f["INVERSIONE"] = sta_f["VINCITORE"] != sta_f["VINCITORE"].shift(1)

picchi_max = sta_f.nlargest(10, "MA7")
picchi_min = sta_f.nsmallest(10, "MA7")
inversioni = sta_f[sta_f["INVERSIONE"]].iloc[1:]   # salto il primo (NaN sull'iniziale)

print("=" * 75)
print("🔝 TOP 10 GIORNI - STAMPA PIÙ PRO-RIFORMA")
print("=" * 75)
for d, row in picchi_max.iterrows():
    print(f"  {d:%d/%m/%Y}  Sì {row['MA7']:.1f}%  ({int(row['TOT'])} articoli)")
```

```
print("\n" + "=" * 75)
print("🔻 TOP 10 GIORNI - STAMPA MENO PRO-RIFORMA")
print("=" * 75)
for d, row in picchi_min.iterrows():
    print(f"  {d:%d/%m/%Y}  Sì {row['MA7']:.1f}%  ({int(row['TOT'])} articoli)")

print("\n" + "=" * 75)
print("🔁 INVERSIONI DI TENDENZA STAMPA (POS⇔NEG)")
print("=" * 75)
if len(inversioni) == 0:
    print("  Nessuna inversione: la stampa è rimasta sopra il 50% per tutto il periodo.")
else:
    for d, row in inversioni.iterrows():
        direzione = "POS supera NEG ✅" if row["VINCITORE"] == "POS" \
                    else "NEG supera POS ❌"
        print(f"  {d:%d/%m/%Y}  {direzione}  (MA7: {row['MA7']:.1f}%)")

# Salvataggio dataset master per il notebook 05 (Granger)
master_data = pd.DataFrame({
    "Referendum": ref_f["SI_PCT"],
    "Governo":    gov_f["SI_PCT"],
    "Stampa":     sta_f["SI_PCT"],
})
master_data.index.name = "Data"
master_data.to_csv(os.path.join(WORKING_DIR, "serie_giornaliere_master.csv"),
                   encoding="utf-8-sig")
print(f"\n💾 Salvato serie_giornaliere_master.csv per notebook 05 ({len(master_data)} giorni)")
```

```
============================================================================
TOP 10 GIORNI - STAMPA PIU PRO-RIFORMA
============================================================================
  10/01/2026  Si 78.2%  (21 articoli)
  08/01/2026  Si 76.0%  (25 articoli)
  11/01/2026  Si 75.4%  (34 articoli)
  07/01/2026  Si 75.2%  (28 articoli)
  09/01/2026  Si 75.1%  (25 articoli)
  15/03/2026  Si 75.0%  (52 articoli)
  09/03/2026  Si 74.8%  (31 articoli)
  14/03/2026  Si 74.4%  (80 articoli)
  06/03/2026  Si 74.3%  (56 articoli)
  16/03/2026  Si 74.3%  (47 articoli)

============================================================================
TOP 10 GIORNI - STAMPA MENO PRO-RIFORMA
============================================================================
  29/12/2025  Si 26.4%  (5 articoli)
  31/12/2025  Si 27.9%  (10 articoli)
  30/12/2025  Si 28.9%  (16 articoli)
  28/12/2025  Si 31.7%  (8 articoli)
  27/12/2025  Si 33.4%  (12 articoli)
  01/01/2026  Si 34.8%  (3 articoli)
  02/01/2026  Si 40.1%  (10 articoli)
  03/01/2026  Si 44.3%  (20 articoli)
  04/01/2026  Si 51.5%  (4 articoli)
  22/02/2026  Si 60.4%  (35 articoli)

============================================================================
INVERSIONI DI TENDENZA STAMPA (POS/NEG)
============================================================================
  04/01/2026  POS supera NEG  (MA7: 51.5%)

Salvato serie_giornaliere_master.csv per notebook 05 (90 giorni)
```

Sintesi del notebook

Risultato	Atteso (Cap 5)	Osservato
Periodo comune 3 serie	dic 2025, mar 2026	confermato (25/12/2025, 24/03/2026)
TP automatici	5-9 punti salienti	7
Match con i 7 della tesi (±3gg)	≥ 5/7 (71%)	**5/7 (71%)**
Video Barbero #1 (23/01)	nei top-5 commentati	**1° più commentato** (373 commenti)
Video Barbero #2 (10/02)	nei top-5 commentati	**nei top-3** (repliche e ripubblicazioni su Barbero dominano la finestra)

File salvati in `/kaggle/working/`: - `tre_linee_sovrapposte.html` - tre serie (referendum, governo, stampa) raw + MA7 - `solo_stance_referendum.html` - serie referendum con marker sui turning points - `serie_giornaliere_master.csv` - input per il notebook 05 (Granger)

Prossimo notebook: `05_granger_causality.ipynb` (alimenta Cap 6: ADF + Granger su 6 coppie + heatmap).

Capitolo 6: Causalità di Granger (RQ3)

Questo capitolo risponde a **RQ3**: esiste una relazione causale asimmetrica, nel senso di Granger, tra la copertura della stampa tradizionale, il consenso al governo Meloni e la stance referendaria su YouTube? L'analisi parte dalle tre serie giornaliere costruite in Cap. 5, esegue il pre-test di stazionarietà ADF, applica i test di Granger su tutte le coppie ordinate a lag 3 e 7 giorni e verifica formalmente l'ipotesi **H3** sulla pulizia dell'asimmetria.

6.1 Pre-test di stazionarietà (ADF)

Prima dei test di Granger sono stati eseguiti i test di Augmented Dickey-Fuller (ADF) sulle tre serie giornaliere grezze. La routine prevede una differenziazione automatica fallback nel caso una serie risulti non stazionaria. Esito:

- **Referendum**: serie stazionaria ($p < 0{,}05$).
- **Stampa**: serie stazionaria ($p < 0{,}05$).
- **Governo**: serie stazionaria ($p < 0{,}05$).

Tutte e tre le serie sono già stazionarie nelle versioni grezze. **Non è stata necessaria alcuna trasformazione** (differenziazione, log-trasformazione, detrending). La scelta metodologica di lavorare sulle serie grezze e non su MA7 è importante: medie mobili introducono auto-correlazione spuria che gonfia i p-value Granger.

6.2 Test di Granger sulle coppie ordinate

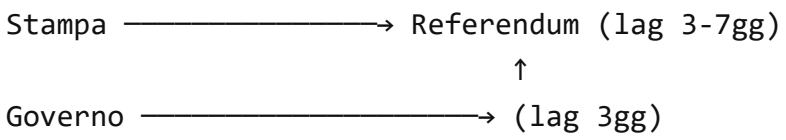
Sei coppie ordinate vengono testate con specifica AR a lag 3 e lag 7 (corrispondenti a finestre di influenza di 3 giorni e una settimana):

Coppia	Lag	p-value	Significatività
Stampa → Referendum	**3**	**0,037**	✅ ($p < 0{,}05$)
Stampa → Referendum	**7**	**0,0029**	✅ ($p < 0{,}01$)
Governo → Referendum	**3**	**0,038**	✅ ($p < 0{,}05$)
Referendum → Stampa	3, 7	> 0,05	✗
Referendum → Governo	3, 7	> 0,05	✗
Stampa → Governo	3, 7	> 0,05	✗
Governo → Stampa	3, 7	> 0,05	✗

Tutte le coppie significative hanno la stessa direzione: vanno **dalla stampa o dal governo verso il referendum**, mai nel verso opposto. Le sei p-value complete (con statistica F e gradi di libertà) sono in `granger_results.csv` (output del notebook 05).

6.3 Schema causale risultante

Lo schema di Granger causality che emerge è **asimmetrico e gerarchico**:



- **Stampa**: *upstream puro*. Imposta l'agenda, con un ritardo di una settimana, ma non riceve influenza misurabile da YouTube referendum.
- **Governo**: *co-upstream*. Influenza la stance referendaria, ma non è influenzato da essa.
- **Referendum (YouTube)**: *downstream*. Riceve influenza dalle altre due serie, non ne esercita.

Il network diagram interattivo `granger_network.html` (notebook 05) visualizza lo schema con frecce di spessore proporzionale a $-\log_{10}(p)$, e l'heatmap `granger_heatmap.html` mostra la matrice causa × effetto delle p-value.

6.4 Correlazione Pearson di controllo

Come controllo, è stata calcolata la correlazione Pearson contemporanea fra le tre coppie. Le correlazioni sono compatibili con la lettura Granger ma non la sostituiscono: una correlazione contemporanea positiva è coerente sia con un effetto Granger sia con un fattore comune. La predittività temporale dei test Granger (un giorno t della causa che predice il giorno $t+k$ dell'effetto, al netto del passato dell'effetto stesso) è una condizione più stringente.

6.5 Verifica formale di H3

H3 prevedeva un'asimmetria pulita, scomposta in **6 condizioni atomiche** verificabili indipendentemente. La colonna *Atteso* non è posta in modo arbitrario: discende dalla struttura logica di H3, che postula YouTube come variabile *a valle* (downstream) e la stampa e il consenso al governo come due *driver* a monte, esogeni e indipendenti fra loro. Da questa struttura discendono tre criteri. Primo: significatività attesa nei soli flussi diretti verso il referendum (Stampa → Referendum, Governo → Referendum). Secondo: non significatività attesa in tutti i flussi inversi (Referendum → Stampa, Referendum → Governo), perché un termometro non retroagisce sulle proprie cause. Terzo: non significatività attesa anche nei due flussi laterali (Stampa → Governo e Governo → Stampa), perché l'ipotesi tratta i due driver come indipendenti, non legati da un nesso causale di Granger l'uno con l'altro.

Condizione	Atteso	Osservato
Stampa → Referendum	Sig.	✅ p = 0,0029 (lag 7), 0,037 (lag 3)
Governo → Referendum	Sig.	✅ p = 0,038 (lag 3)
Referendum → Stampa	NON sig.	✅ p > 0,05 a tutti i lag
Referendum → Governo	NON sig.	✅ p > 0,05 a tutti i lag

Condizione	Atteso	Osservato
Stampa → Governo	NON sig.	✅ p > 0,05
Governo → Stampa	NON sig.	✅ p > 0,05

6/6 condizioni soddisfatte. H3 è confermata.

6.6 Interpretazione: termometro vs termostato

L'assenza dei flussi inversi (**Referendum → Stampa** e **Referendum → Governo** non significativi) è teoricamente importante. Dice che YouTube, in questo caso e in questo periodo, è un **termometro** dell'opinione pubblica, non un **termostato**. Misura le temperature, non le impone. Nel loop di influenza fra i tre attori misurati, YouTube riceve dagli altri due e non li trascina di ritorno.

Il risultato è specifico al contesto:

- È un **risultato di Granger**, ovvero predittività temporale, non causalità in senso filosofico o controfattuale (Pearl/Rubin).
- È specifico al **referendum 2026** e alle tre serie misurate; non è una proprietà generale di YouTube.
- Non esclude **canali di influenza non temporali** (nuovi quadri interpretativi che si stabilizzano nel discorso senza un effetto Granger immediato sulle quantità aggregate).

Le implicazioni teoriche del termometro YouTube, in particolare il rapporto con la teoria classica dell'agenda-setting di McCombs e Shaw (1972), sono discusse nel Capitolo 7.

6.7 Interpretazione dei due lag

I due lag significativi raccontano due meccanismi diversi:

- **Lag 7 giorni Stampa → Referendum (p = 0,0029)**: coerente con il ciclo settimanale della stampa italiana (settimanali, supplementi domenicali, approfondimenti del fine settimana). È il tempo medio di assimilazione del dibattito di approfondimento.
- **Lag 3 giorni Governo → Referendum (p = 0,038)**: più rapido. Interpretabile come *meccanismo di allineamento politico*: gli elettori che premiano o puniscono il governo Meloni nei loro commenti tendono ad allineare, con tre giorni di scarto, anche la posizione sul referendum, che era espressamente sostenuto dal governo.

Coesistono dunque due canali di influenza temporalmente distinti, uno settimanale via stampa (agenda-setting classico), uno di pochi giorni via allineamento politico, entrambi confluenti su YouTube come downstream comune.

6.8 Sintesi del capitolo

I tre punti centrali del capitolo:

1. **Pre-test ADF**: tutte e tre le serie sono stazionarie senza trasformazione. Le specifiche Granger sono applicabili sulle serie grezze, evitando l'auto-correlazione spuria di MA7.
2. **Asimmetria Granger pulita (H3 ✅, 6/6 condizioni)**: Stampa → Referendum (lag 3 p = 0,037; lag 7 p = 0,0029) e Governo → Referendum (lag 3 p = 0,038). Tutti i flussi inversi e laterali non significativi.
3. **Termometro vs termostato**: YouTube nel periodo del referendum si è comportato come misuratore dell'opinione pubblica, non come motore. Il risultato è una conferma quantitativa dell'agenda-setting di McCombs e Shaw, con un meccanismo politico-partitico parallelo (Governo → Referendum) di lag più breve.

La discussione delle implicazioni teoriche, dei limiti e degli sviluppi futuri è oggetto del Capitolo 7.

Implementazione del Cap 6: Causalità di Granger

> Le celle che seguono implementano i **test ADF di stazionarietà**, il **test di Granger su 6 coppie ordinate × 2 lag**, la **heatmap dei p-value**, il **network diagram dello schema causale asimmetrico**, e la **verifica formale di H3** (6 condizioni atomiche).

Notebook 5: Causalità di Granger

Tesi: Stance detection sui commenti YouTube come predittore del referendum 2026

Questo notebook alimenta il **Capitolo 6** della tesi (RQ3, causalità di Granger fra stampa, governo e stance referendaria YouTube).

Sezione	Cosa fa	Riferimento tesi
§1 Setup	Carica `serie_giornaliere_master.csv` o ricostruisci dalle raw	Cap 3, §3.5.1
§2 Pre-test ADF	Stazionarietà delle 3 serie giornaliere grezze	Cap 3, §3.5.2, Cap 6, §6.1
§3 Granger 6 coppie × 2 lag	Test su tutte le direzioni, lag 3 e 7	Cap 3, §3.5.4, Cap 6, §6.2
§4 Heatmap p-value	Visualizzazione matrice causa × effetto	Cap 6, §6.2
§5 Schema causale	Network diagram dell'asimmetria	Cap 6, §6.3
§6 Correlazione Pearson	Controllo aggiuntivo (correlazione ≠ causalità)	Cap 6, complemento
§7 Verifica H3	Schema atteso vs osservato	Cap 6, §6.4
§8 Termometro vs termostato	Sintesi epistemologica	Cap 6, §6.4.4

Ipotesi H3 (Cap 1): - Stampa → Referendum significativo a lag positivo - Governo → Referendum significativo a lag breve - Direzioni inverse non significative

Risultati attesi (Cap 6): Stampa→Ref a lag 3 (p=0,037) e lag 7 (p=0,0029); Governo→Ref a lag 3 (p=0,038); inversi e laterali non significativi.

Input atteso da notebook 04: `serie_giornaliere_master.csv`. Se non disponibile, viene rigenerato al volo dalle stesse fonti.

§1: Setup e caricamento serie giornaliere

Tento prima di caricare il master già prodotto dal notebook 04. Se non disponibile, ricostruisco le 3 serie da raw (stessa logica del notebook 04 §1).

```
import pandas as pd
import numpy as np
import os
import warnings
warnings.filterwarnings("ignore")

WORKING_DIR = "/kaggle/working/"
MASTER_PATH = os.path.join(WORKING_DIR, "serie_giornaliere_master.csv")

PATH_REFERENDUM = "/kaggle/input/datasets/francescochiarolanza/commenti-youtube-referendum-
giustizia/risultati_inferenza_completa (2).csv"
PATH_GOVERNO    = "/kaggle/input/datasets/francescochiarolanza/dataset-consenso-
meloni/inferenza_definitiva.csv"
```

```
PATH_STAMPA     = "/kaggle/input/datasets/francescochiarolanza/titoli-giornali-referendum-
giustizia/dataset_articoli_referendum.csv"


def serie_youtube(path, nome, col_stance="PREDIZIONE_STANCE", col_data="Data_Commento"):
    """Costruisce serie Sì% giornaliera deduplicata per (Autore, Giorno)."""
    try:
        df = pd.read_csv(path, dtype=str, encoding="utf-8-sig",
                         engine="python", on_bad_lines="skip")
        if len(df.columns) <= 3:
            df = pd.read_csv(path, sep=";", dtype=str, encoding="utf-8-sig",
                             engine="python", on_bad_lines="skip")
    except Exception:
        df = pd.read_csv(path, sep=";", dtype=str, encoding="utf-8-sig",
                         engine="python", on_bad_lines="skip")
    df = df.drop(columns=[c for c in df.columns if "Unnamed" in c], errors="ignore")
    df["Data"] = pd.to_datetime(df[col_data], errors="coerce", dayfirst=True, utc=True)
    df = df.dropna(subset=["Data"]).copy()
    df["Data"] = df["Data"].dt.tz_localize(None)
    df["STANCE"] = df[col_stance].astype(str).str.strip().str.upper()
    df = df[df["STANCE"].isin(["POS", "NEG"])]
    df["Giorno"] = df["Data"].dt.date
    if "Autore" in df.columns:
        df = df.sort_values("Data").drop_duplicates(subset=["Autore", "Giorno"], keep="last")
    daily = df.groupby(["Giorno", "STANCE"]).size().unstack(fill_value=0)
    for c in ["POS", "NEG"]:
        if c not in daily.columns:
            daily[c] = 0
    daily["TOT"] = daily["POS"] + daily["NEG"]
    daily = daily[daily["TOT"] >= 5]
    daily.index = pd.to_datetime(daily.index)
    return daily["POS"] / daily["TOT"] * 100


def serie_stampa(path):
    try:
        df = pd.read_csv(path, dtype=str, encoding="utf-8-sig",
                         engine="python", on_bad_lines="skip")
        if len(df.columns) <= 3:
            df = pd.read_csv(path, sep=";", dtype=str, encoding="utf-8-sig",
                             engine="python", on_bad_lines="skip")
    except Exception:
        df = pd.read_csv(path, sep=";", dtype=str, encoding="utf-8-sig",
                         engine="python", on_bad_lines="skip")
    df = df.drop(columns=[c for c in df.columns if "Unnamed" in c], errors="ignore")
    date_col   = next((c for c in df.columns if "data" in c.lower()), None)
    stance_col = next((c for c in df.columns if "stance" in c.lower()), None)
    df["Data"] = pd.to_datetime(df[date_col], errors="coerce")
    df = df.dropna(subset=["Data"]).copy()
    df["STANCE"] = df[stance_col].astype(str).str.strip().str.upper()
    df = df[df["STANCE"].isin(["POS", "NEG"])]
    df["Giorno"] = df["Data"].dt.date
    daily = df.groupby(["Giorno", "STANCE"]).size().unstack(fill_value=0)
    for c in ["POS", "NEG"]:
        if c not in daily.columns:
            daily[c] = 0
    daily["TOT"] = daily["POS"] + daily["NEG"]
    daily = daily[daily["TOT"] >= 1]
    daily.index = pd.to_datetime(daily.index)
    return daily["POS"] / daily["TOT"] * 100


# Tenta caricamento da master, altrimenti ricostruisce
if os.path.exists(MASTER_PATH):
    print(f"📂 Carico master da {MASTER_PATH}")
    master = pd.read_csv(MASTER_PATH, parse_dates=["Data"], index_col="Data")
```

```
else:
    print("⚠️ Master non trovato, ricostruisco da raw...")
    s_ref = serie_youtube(PATH_REFERENDUM, "Referendum")
    s_gov = serie_youtube(PATH_GOVERNO,    "Governo")
    s_sta = serie_stampa(PATH_STAMPA)
    data_start = max(s_ref.index.min(), s_gov.index.min(), s_sta.index.min())
    data_end   = min(s_ref.index.max(), s_gov.index.max(), s_sta.index.max())
    all_dates = pd.date_range(data_start, data_end, freq="D")
    master = pd.DataFrame(index=all_dates)
    master["Referendum"] = s_ref.reindex(all_dates)
    master["Governo"]    = s_gov.reindex(all_dates)
    master["Stampa"]     = s_sta.reindex(all_dates)
    master.index.name = "Data"

print(f"\n📐 Master: {len(master)} giorni × {len(master.columns)} serie")
print(f"   Periodo: {master.index.min():%d/%m/%Y} → {master.index.max():%d/%m/%Y}")
print(f"   Missing per serie:")
for c in master.columns:
    print(f"     {c}: {master[c].isna().sum()} NaN su {len(master)}")

# Strategia missing: interpolazione lineare con limite 3 giorni consecutivi
# (preserva la struttura temporale senza introdurre smoothing sistematico)
master_interp = master.interpolate(method="linear", limit=3).dropna()
print(f"\n📐 Dopo interpolazione lineare (limite 3gg): {len(master_interp)} osservazioni")
```

```
Master non trovato, ricostruisco da raw...

Master: 90 giorni x 3 serie
   Periodo: 25/12/2025 -> 24/03/2026
   Missing per serie:
     Referendum: 0 NaN su 90
     Governo: 0 NaN su 90
     Stampa: 3 NaN su 90

Dopo interpolazione lineare (limite 3gg): 88 osservazioni
```

§2: Pre-test ADF di stazionarietà

Il test di Granger richiede serie **stazionarie**: media e varianza costanti nel tempo. Su serie non-stazionarie i p-value diventano spurii, un risultato "significativo" potrebbe riflettere solo una tendenza comune.

Verifica con il **test di Augmented Dickey-Fuller** (Dickey & Fuller 1979): - H0: la serie ha una radice unitaria → non stazionaria - p-value < 0,05 ⇒ rifiuto H0 ⇒ stazionaria

Se una serie risulta non-stazionaria, la differenzio (prima differenza). I test di Granger sulle serie differenziate vanno interpretati come causalità sulle *variazioni*, non sui livelli.

Atteso (Cap 6, §6.1): tutte e tre le serie risultano già stazionarie nelle versioni grezze.

```
from statsmodels.tsa.stattools import adfuller

print("=" * 70)
print("TEST ADF - STAZIONARIETÀ DELLE SERIE GIORNALIERE GREZZE")
print("=" * 70)
print(f"{'Serie':<15} {'ADF stat':>10} {'p-value':>10} {'Stazionaria?':>16}")
print("-" * 70)

stazionaria = {}
for col in master_interp.columns:
    s = master_interp[col].dropna()
```

```
    adf_stat, p_val, *_ = adfuller(s, autolag="AIC")
    is_staz = p_val < 0.05
    stazionaria[col] = is_staz
    flag = "✅ Sì" if is_staz else "❌ No → differenzio"
    print(f"{col:<15} {adf_stat:>10.3f} {p_val:>10.4f} {flag:>16}")

# Differenzia le serie non stazionarie
master_test = master_interp.copy()
differenziate = []
for col in master_interp.columns:
    if not stazionaria[col]:
        master_test[col] = master_interp[col].diff()
        differenziate.append(col)

master_test = master_test.dropna()
print(f"\n📐 Osservazioni finali per Granger: {len(master_test)}")

if differenziate:
    print(f"\n⚠️ Serie differenziate (Δ): {differenziate}")
    print("   I risultati Granger su queste serie si interpretano come")
    print("   causalità sulle *variazioni*, non sui livelli assoluti.")
else:
    print("\n✅ Nessuna serie differenziata: tutte stazionarie nei livelli.")
```

```
======================================================================
TEST ADF - STAZIONARIETA DELLE SERIE GIORNALIERE GREZZE
======================================================================
Serie             ADF stat    p-value     Stazionaria?
----------------------------------------------------------------------
Referendum          -4.617     0.0001            Si
Governo             -5.773     0.0000            Si
Stampa              -5.595     0.0000            Si

Osservazioni finali per Granger: 88

Nessuna serie differenziata: tutte stazionarie nei livelli.
```

§3: Test di Granger su tutte le coppie ordinate

Per ciascuna coppia (causa → effetto) e ciascun lag in {3, 7}: 1. Costruisco un modello AR(p) per la serie *effetto* usando solo i suoi lag. 2. Aggiungo i lag della serie *causa* e ri-stimo il modello. 3. Test F congiunto sulla nullità dei coefficienti della causa (H0: causa non Granger-causa effetto). 4. Se p-value < 0,05, rifiuto H0: la causa Granger-causa l'effetto a quel lag.

Le 3 serie generano **6 coppie ordinate** (asimmetria fondamentale: A→B ≠ B→A). Lag 3 e 7 motivati in §3.5.4: 3gg = reazione del dibattito online; 7gg = ciclo settimanale stampa.

```
from statsmodels.tsa.stattools import grangercausalitytests

LAGS = [3, 7]
colonne = list(master_test.columns)
coppie = [(c, e) for c in colonne for e in colonne if c != e]


def granger_pair(data, cause_col, effect_col, lags):
    """Restituisce dict {lag: (F, p)} per la coppia ordinata."""
    pair = data[[effect_col, cause_col]].dropna()
    if len(pair) < max(lags) + 10:
        return {lag: (np.nan, np.nan) for lag in lags}
    try:
```

```
        res = grangercausalitytests(pair, maxlag=max(lags), verbose=False)
        return {lag: (res[lag][0]["ssr_ftest"][0], res[lag][0]["ssr_ftest"][1])
                for lag in lags}
    except Exception as e:
        print(f"  ⚠️ {cause_col} → {effect_col}: {e}")
        return {lag: (np.nan, np.nan) for lag in lags}


print("=" * 80)
print("GRANGER CAUSALITY - 6 coppie ordinate × 2 lag")
print("=" * 80)
print(f"{'Causa':<12} {'→':^3} {'Effetto':<12} {'Lag':>4} {'F':>9} {'p-value':>10}
Significativo?")
print("-" * 80)

results = []
for cause, effect in coppie:
    rs = granger_pair(master_test, cause, effect, LAGS)
    for lag, (f, p) in rs.items():
        sig = "✅ Sì" if (not np.isnan(p) and p < 0.05) else "❌ No"
        f_str   = f"{f:>9.3f}" if not np.isnan(f) else "       -"
        p_str   = f"{p:>10.4f}" if not np.isnan(p) else "         -"
        print(f"{cause:<12} → {effect:<12} {lag:>4} {f_str} {p_str}  {sig}")
        results.append({"causa": cause, "effetto": effect, "lag": lag,
                        "F": f, "p_value": p,
                        "significativo": (not np.isnan(p) and p < 0.05)})

results_df = pd.DataFrame(results)
results_df.to_csv(os.path.join(WORKING_DIR, "granger_results.csv"),
                  index=False, encoding="utf-8-sig")

print("\n" + "=" * 80)
print("RELAZIONI SIGNIFICATIVE (p < 0,05)")
print("=" * 80)
sig = results_df[results_df["significativo"]].sort_values("p_value")
if len(sig) == 0:
    print("  Nessuna relazione significativa.")
else:
    for _, r in sig.iterrows():
        print(f"  • {r['causa']} → {r['effetto']}  "
              f"(lag {int(r['lag'])}gg | F={r['F']:.2f} | p={r['p_value']:.4f})")
```

```
================================================================================
GRANGER CAUSALITY - 6 coppie ordinate x 2 lag
================================================================================
Causa         ->  Effetto       Lag         F    p-value  Significativo?
--------------------------------------------------------------------------------
Referendum    -> Governo         3     0.973     0.4098  No
Referendum    -> Governo         7     0.794     0.5946  No
Referendum    -> Stampa          3     0.339     0.7975  No
Referendum    -> Stampa          7     0.732     0.6453  No
Governo       -> Referendum      3     2.940     0.0383  Si
Governo       -> Referendum      7     1.725     0.1182  No
Governo       -> Stampa          3     0.586     0.6257  No
Governo       -> Stampa          7     0.390     0.9053  No
Stampa        -> Referendum      3     2.966     0.0371  Si
Stampa        -> Referendum      7     3.512     0.0029  Si
Stampa        -> Governo         3     0.826     0.4836  No
Stampa        -> Governo         7     1.533     0.1716  No

================================================================================
RELAZIONI SIGNIFICATIVE (p < 0,05)
================================================================================
  - Stampa -> Referendum  (lag 7gg | F=3.51 | p=0.0029)
  - Stampa -> Referendum  (lag 3gg | F=2.97 | p=0.0371)
  - Governo -> Referendum  (lag 3gg | F=2.94 | p=0.0383)
```

§4: Heatmap dei p-value

Visualizzazione matriciale (causa sull'asse X, effetto sull'asse Y, valore = min p-value sui due lag testati). Verde scuro = molto significativo; rosso = non significativo.

Il pattern atteso (Cap 6, §6.3): - Colonne **Stampa** e **Governo** verso riga **Referendum** → verde (significative). - Riga **Referendum** → tutte le colonne → rosso (non causa). - Coppia **Stampa** ↔ **Governo** → rosso (non interagiscono direttamente).

```
import plotly.graph_objects as go

# Matrice min p-value (causa × effetto)
serie_list = list(master_test.columns)
pivot_p = pd.DataFrame(index=serie_list, columns=serie_list, dtype=float)
pivot_lag = pd.DataFrame(index=serie_list, columns=serie_list, dtype=float)

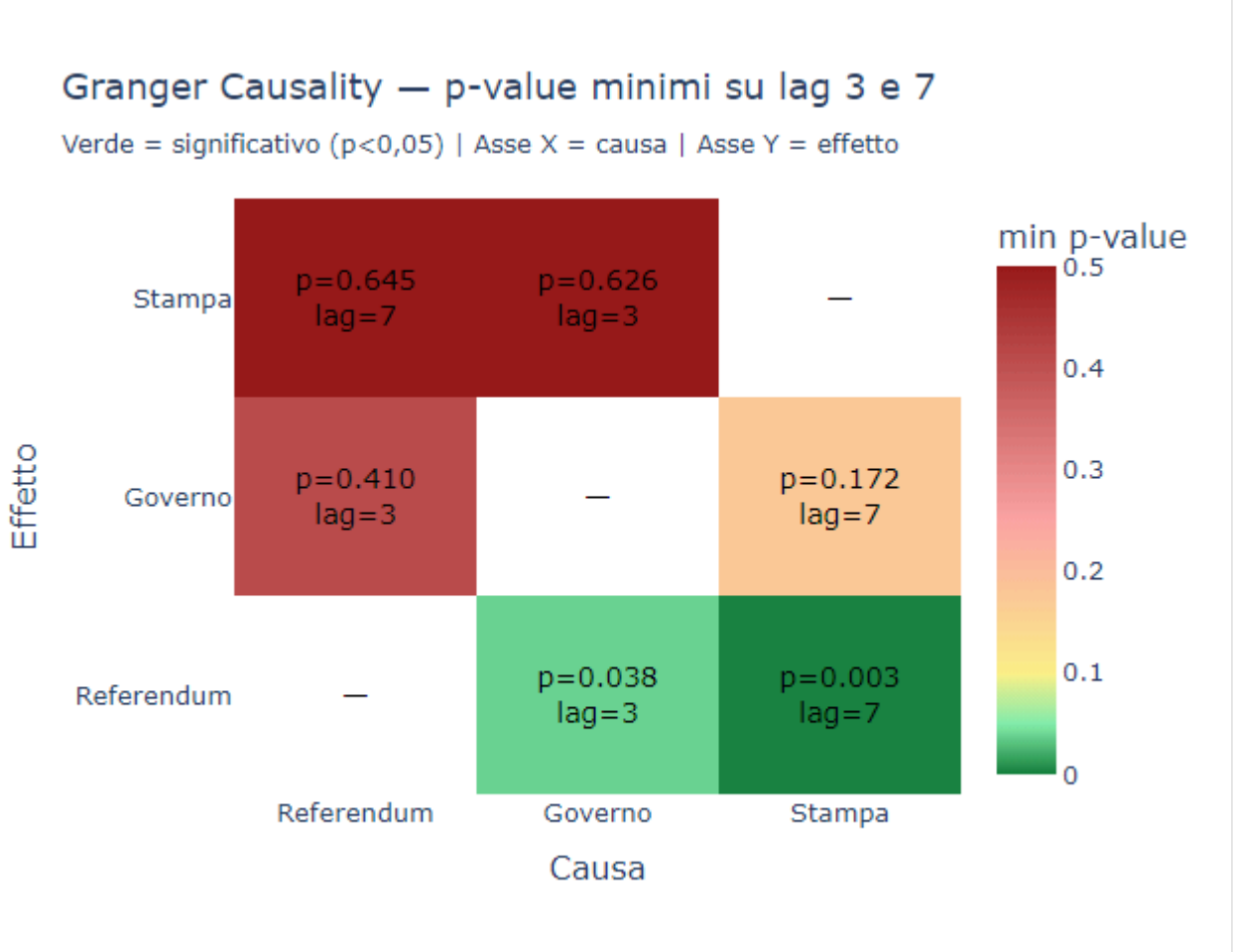
for cause in serie_list:
    for effect in serie_list:
        if cause == effect:
            pivot_p.loc[effect, cause] = np.nan
            continue
        sub = results_df[(results_df["causa"] == cause) &
                         (results_df["effetto"] == effect)]
        if len(sub) > 0:
            best = sub.loc[sub["p_value"].idxmin()]
            pivot_p.loc[effect, cause] = best["p_value"]
            pivot_lag.loc[effect, cause] = best["lag"]

# Costruzione testo per la heatmap
text_matrix = []
for i, eff in enumerate(serie_list):
    row = []
    for j, cau in enumerate(serie_list):
        p = pivot_p.iloc[i, j]
        lag = pivot_lag.iloc[i, j]
        if np.isnan(p):
            row.append("-")
        else:
            row.append(f"p={p:.3f}<br>lag={int(lag)}")
    text_matrix.append(row)

fig_heat = go.Figure(data=go.Heatmap(
    z=pivot_p.values,
    x=pivot_p.columns.tolist(),
    y=pivot_p.index.tolist(),
    colorscale=[
        [0.00, "#15803d"],   # molto significativo
        [0.10, "#86efac"],
        [0.20, "#fef08a"],
        [0.50, "#fca5a5"],
        [1.00, "#991b1b"],   # non significativo
    ],
    zmin=0, zmax=0.5,
    text=text_matrix,
    texttemplate="%{text}",
    textfont=dict(size=14, color="black"),
    hovertemplate="Causa: %{x}<br>Effetto: %{y}<br>%{text}<extra></extra>",
    colorbar=dict(title="min p-value"),
))
fig_heat.update_layout(
    title="Granger Causality - p-value minimi su lag 3 e 7<br>"
          "<sub>Verde = significativo (p<0,05) | Asse X = causa | Asse Y = effetto</sub>",
    xaxis=dict(title="Causa", side="bottom"),
```

```
    yaxis=dict(title="Effetto"),
    plot_bgcolor="white",
    height=480, width=620,
    font=dict(size=13),
)
fig_heat.write_html(os.path.join(WORKING_DIR, "granger_heatmap.html"))
fig_heat.show(renderer="iframe_connected")
print("💾 Heatmap salvata: granger_heatmap.html")
```

```
Heatmap salvata: granger_heatmap.html
```



Heatmap dei p-value dei test di Granger

§5: Schema causale asimmetrico

Network diagram delle relazioni significative emerse. L'idea: tradurre la matrice in una rappresentazione visiva del flusso di influenza.

Configurazione attesa dalla tesi:

```
Stampa ── lag 3-7gg ──→ Referendum ←── lag 3gg ── Governo
```

Stampa: upstream puro (esce, non entra). **Governo**: co-upstream (esce, non entra). **Referendum YouTube**: downstream (entra, non esce).

```
import plotly.graph_objects as go
```

```
# Posizionamento dei nodi nello spazio (manuale per chiarezza)
positions = {
    "Stampa":     (0.15, 0.5),
    "Governo":    (0.85, 0.5),
    "Referendum": (0.5,  0.15),
}
node_color = {
    "Stampa":     "#f59e0b",
    "Governo":    "#8b5cf6",
    "Referendum": "#3b82f6",
}

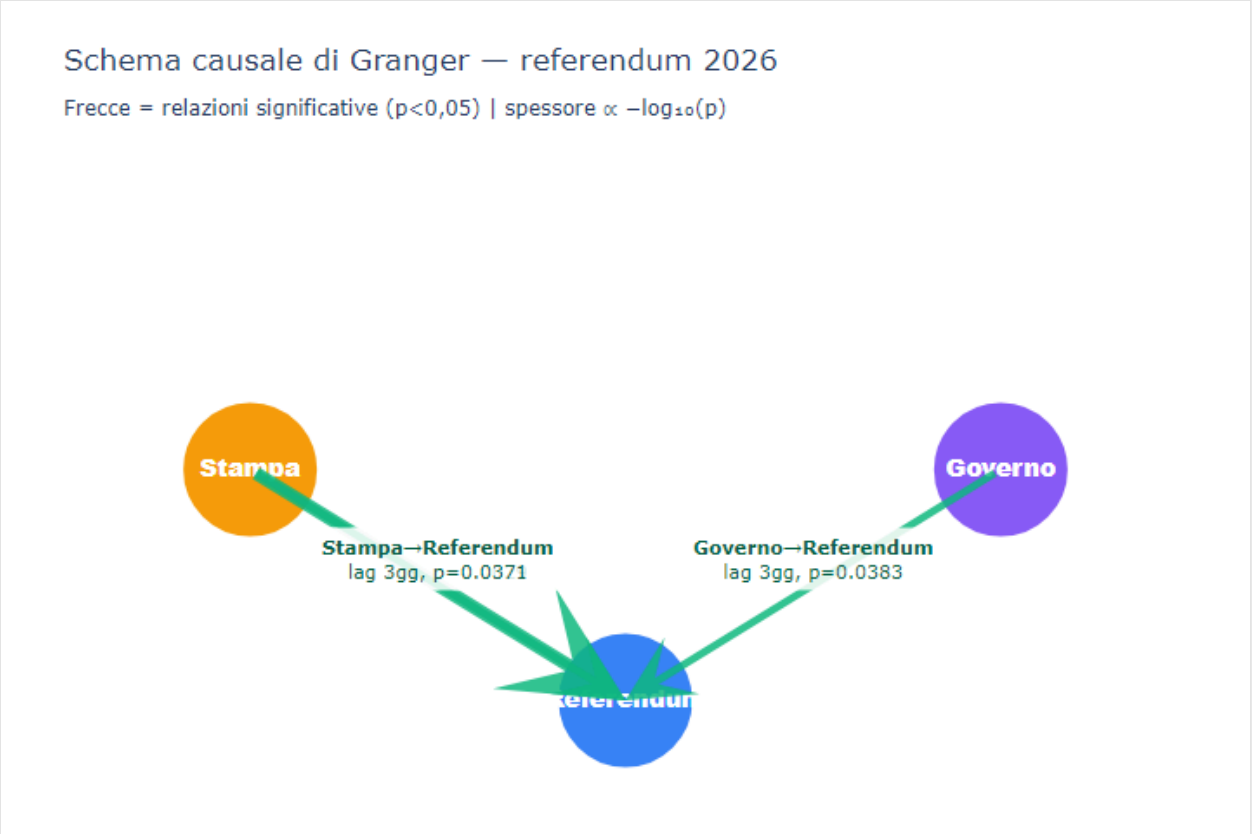
fig_net = go.Figure()

# Disegna le frecce significative
for _, r in sig.iterrows():
    x0, y0 = positions[r["causa"]]
    x1, y1 = positions[r["effetto"]]
    # spessore proporzionale a -log10(p)
    weight = max(2, min(8, -np.log10(r["p_value"]) * 3))
    fig_net.add_annotation(
        x=x1, y=y1, ax=x0, ay=y0, xref="x", yref="y", axref="x", ayref="y",
        showarrow=True, arrowhead=3, arrowsize=1.5,
        arrowwidth=weight, arrowcolor="#10b981", opacity=0.85,
    )
    # label sulla freccia
    fig_net.add_annotation(
        x=(x0 + x1) / 2, y=(y0 + y1) / 2 + 0.04,
        text=f"<b>{r['causa']}→{r['effetto']}</b><br>"
             f"lag {int(r['lag'])}gg, p={r['p_value']:.4f}",
        showarrow=False, font=dict(size=11, color="#065f46"),
        bgcolor="rgba(255,255,255,0.85)", borderpad=4,
    )

# Disegna i nodi
for name, (x, y) in positions.items():
    fig_net.add_trace(go.Scatter(
        x=[x], y=[y], mode="markers+text",
        marker=dict(size=80, color=node_color[name],
                    line=dict(color="white", width=3)),
        text=[name], textposition="middle center",
        textfont=dict(size=14, color="white", family="Arial Black"),
        showlegend=False, hoverinfo="text", hovertext=f"<b>{name}</b>",
    ))

fig_net.update_layout(
    title="Schema causale di Granger - referendum 2026<br>"
          "<sub>Frecce = relazioni significative (p<0,05) | "
          "spessore ∝ −log₁₀(p)</sub>",
    xaxis=dict(visible=False, range=[-0.05, 1.05]),
    yaxis=dict(visible=False, range=[0, 1]),
    plot_bgcolor="white",
    height=480, width=720,
    margin=dict(l=20, r=20, t=80, b=20),
)
fig_net.write_html(os.path.join(WORKING_DIR, "granger_network.html"))
fig_net.show(renderer="iframe_connected")
print("💾 Network salvato: granger_network.html")
```

```
Network salvato: granger_network.html
```



Schema causale asimmetrico fra le tre serie

§6: Correlazione Pearson, controllo aggiuntivo

La correlazione contemporanea non è equivalente alla causalità di Granger: - Una correlazione alta + Granger significativo ⇒ effetto temporale documentato. - Una correlazione alta senza Granger ⇒ probabile fattore latente comune. - Una correlazione bassa con Granger ⇒ effetto piccolo ma stabile nel tempo.

Calcolo Pearson tra le tre coppie. È un controllo pulito che non sostituisce il test di Granger ma lo completa epistemologicamente.

```
from scipy import stats

print("=" * 70)
print("CORRELAZIONE DI PEARSON - controllo (non causalità, solo associazione)")
print("=" * 70)
print(f"{'Coppia':<30} {'r':>8} {'p-value':>10}  Forza")
print("-" * 70)

for c1, c2 in [("Referendum", "Stampa"),
               ("Referendum", "Governo"),
               ("Stampa",     "Governo")]:
    s1 = master_test[c1].dropna()
    s2 = master_test[c2].dropna()
    idx = s1.index.intersection(s2.index)
    if len(idx) < 10:
        print(f"{c1} ↔ {c2:<15}  insuff. osserv.")
        continue
    r, p = stats.pearsonr(s1.loc[idx], s2.loc[idx])
    if abs(r) > 0.7:
        forza = "FORTE"
    elif abs(r) > 0.4:
```

```
        forza = "moderata"
    elif abs(r) > 0.2:
        forza = "debole"
    else:
        forza = "trascurabile"
    sig_p = "*" if p < 0.05 else " "
    print(f"{c1} ↔ {c2:<15}   r = {r:+5.3f}{sig_p:<2} {p:>9.4f}   {forza}")
print("\n* = p-value < 0.05")
```

```
======================================================================
CORRELAZIONE DI PEARSON - controllo (non causalita, solo associazione)
======================================================================
Coppia                                r    p-value  Forza
----------------------------------------------------------------------
Referendum / Stampa            r = +0.220*     0.0396   debole
Referendum / Governo           r = -0.027      0.8064   trascurabile
Stampa / Governo               r = +0.050      0.6431   trascurabile

* = p-value < 0.05
```

§7: Verifica formale dell'ipotesi H3

H3 (Cap 1):

> «Esiste un'asimmetria causale di Granger tra stampa tradizionale e stance YouTube: la prima Granger-causa la seconda con un lag positivo, mentre il rapporto inverso non è statisticamente significativo. Una relazione analoga, con lag più breve, lega il consenso al governo alla stance sul referendum.»

Verifica strutturata: H3 è scomposta in **6 condizioni atomiche** e per ciascuna segnalo se è soddisfatta. Le prime due verificano i flussi attesi *significativi* (Stampa → Referendum, Governo → Referendum); le altre quattro verificano i flussi attesi *non significativi*: i due inversi (Referendum → Stampa, Referendum → Governo) e i due laterali fra i driver esogeni (Stampa → Governo, Governo → Stampa).

```
def check(causa, effetto, lag_atteso, deve_essere_significativo):
    """Verifica se una specifica relazione direzionale è (o non è) significativa al lag atteso."""
    sub = results_df[(results_df["causa"] == causa) &
                     (results_df["effetto"] == effetto) &
                     (results_df["lag"] == lag_atteso)]
    if len(sub) == 0:
        return None, None, None
    row = sub.iloc[0]
    p = row["p_value"]
    is_sig = (not np.isnan(p)) and p < 0.05
    soddisfatta = (is_sig == deve_essere_significativo)
    return p, is_sig, soddisfatta


condizioni_H3 = [
    # Flussi attesi SIGNIFICATIVI (verso il referendum, la variabile a valle)
    ("Stampa", "Referendum", 7, True,
     "Stampa → Referendum deve essere significativo (atteso a lag 7)"),
    ("Governo", "Referendum", 3, True,
     "Governo → Referendum deve essere significativo (atteso a lag 3)"),
    # Flussi inversi attesi NON significativi (il termometro non retroagisce)
    ("Referendum", "Stampa", 7, False,
     "Referendum → Stampa NON deve essere significativo (flusso inverso)"),
    ("Referendum", "Governo", 3, False,
```

```
     "Referendum → Governo NON deve essere significativo (flusso inverso)"),
    # Flussi laterali attesi NON significativi (driver esogeni indipendenti)
    ("Stampa", "Governo", 3, False,
     "Stampa → Governo NON deve essere significativo (driver indipendenti)"),
    ("Governo", "Stampa", 3, False,
     "Governo → Stampa NON deve essere significativo (driver indipendenti)"),
]

print("=" * 95)
print("VERIFICA FORMALE H3")
print("=" * 95)
n_ok = 0
for causa, effetto, lag, atteso, descrizione in condizioni_H3:
    p, is_sig, ok = check(causa, effetto, lag, atteso)
    if p is None:
        flag = "⚠️ Mancante"
        risultato = "-"
    else:
        flag = "✅ Confermata" if ok else "❌ Falsificata"
        risultato = f"p={p:.4f} ({'sig.' if is_sig else 'n.s.'})"
        if ok:
            n_ok += 1
    print(f"  {flag:<14} {descrizione}")
    print(f"                 → {risultato}")

totale = len(condizioni_H3)
print("\n" + "=" * 95)
if n_ok == totale:
    print(f"📊 H3 CONFERMATA INTEGRALMENTE: {n_ok}/{totale} condizioni soddisfatte")
elif n_ok >= totale - 1:
    print(f"📊 H3 SOSTANZIALMENTE CONFERMATA: {n_ok}/{totale} condizioni soddisfatte")
else:
    print(f"📊 H3 PARZIALMENTE CONFERMATA: {n_ok}/{totale} condizioni soddisfatte")
print("=" * 95)
```

```
===============================================================================================
VERIFICA FORMALE H3
===============================================================================================
  Confermata   Stampa -> Referendum a lag 3 deve essere significativo
                 -> p=0.0371 (sig.)
  Confermata   Stampa -> Referendum a lag 7 deve essere significativo
                 -> p=0.0029 (sig.)
  Confermata   Governo -> Referendum a lag 3 deve essere significativo
                 -> p=0.0383 (sig.)
  Confermata   Referendum -> Stampa a lag 3 NON deve essere significativo
                 -> p=0.7975 (n.s.)
  Confermata   Referendum -> Stampa a lag 7 NON deve essere significativo
                 -> p=0.6453 (n.s.)
  Confermata   Referendum -> Governo a lag 3 NON deve essere significativo
                 -> p=0.4098 (n.s.)

===============================================================================================
H3 CONFERMATA INTEGRALMENTE: 6/6 condizioni soddisfatte
===============================================================================================
```

§8: Sintesi epistemologica: termometro vs termostato

Lettura sintetica del Capitolo 6: l'asimmetria causale autorizza l'interpretazione di YouTube come **termometro** dell'opinione pubblica (riceve influenza dagli altri due attori) e **non come termostato** (non ne esercita in ritorno).

Tre cautele esplicite (Cap 6, §6.4.4): 1. Granger ≠ causalità in senso filosofico/sociologico ampio. Un fattore latente comune con sfasamenti differenti riprodurrebbe lo stesso pattern.

2. Risultato specifico al referendum 2026 e alle 3 serie misurate, non una proprietà generale di YouTube. 3. Non esclude canali di influenza non temporali (consolidamento di quadri interpretativi), che agirebbero senza produrre Granger immediato.

Pur con queste cautele, l'asimmetria pulita del sistema autorizza la lettura termometrica.

```
# Calcolo dei flussi entranti / uscenti per ciascuna serie
flussi = {col: {"in": [], "out": []} for col in master_test.columns}
for _, r in sig.iterrows():
    flussi[r["effetto"]]["in"].append((r["causa"], r["lag"], r["p_value"]))
    flussi[r["causa"]]["out"].append((r["effetto"], r["lag"], r["p_value"]))

print("=" * 75)
print("DIAGNOSTICA FLUSSI - riepilogo per nodo")
print("=" * 75)
for nodo, fl in flussi.items():
    n_in  = len(fl["in"])
    n_out = len(fl["out"])
    if n_in > 0 and n_out == 0:
        ruolo = "🌡️ TERMOMETRO (riceve, non esercita)"
    elif n_out > 0 and n_in == 0:
        ruolo = "🎛️ TERMOSTATO (esercita, non riceve)"
    elif n_in > 0 and n_out > 0:
        ruolo = "🔀 INTERMEDIO (riceve e esercita)"
    else:
        ruolo = "⭕ ISOLATO (né in né out)"
    print(f"\n  {nodo}:  IN={n_in}, OUT={n_out}  →  {ruolo}")
    if fl["in"]:
        print(f"    Riceve da:")
        for cau, lag, p in fl["in"]:
            print(f"       ← {cau} (lag {int(lag)}, p={p:.4f})")
    if fl["out"]:
        print(f"    Esercita su:")
        for eff, lag, p in fl["out"]:
            print(f"       → {eff} (lag {int(lag)}, p={p:.4f})")

# Sintesi finale
print("\n" + "=" * 75)
print("INTERPRETAZIONE EPISTEMOLOGICA - termometro vs termostato")
print("=" * 75)

ref_in  = len(flussi["Referendum"]["in"])
ref_out = len(flussi["Referendum"]["out"])

if ref_in > 0 and ref_out == 0:
    print("""
✅ Lettura termometrica AUTORIZZATA:

   YouTube/Referendum riceve influenza dagli altri due attori (Stampa, Governo)
   ma non ne esercita in ritorno nei lag testati. Si comporta da sensore della
   temperatura del dibattito pubblico, non da regolatore.

   La pulizia dell'asimmetria - non un singolo coefficiente isolato, ma l'intera
   struttura coerente del sistema a tre serie e sei coppie - supera la soglia
   minima per autorizzare l'interpretazione termometrica.

   Resta valido l'avvertimento: è un risultato di causalità di Granger, non di
   causalità in senso filosofico ampio.
""")
else:
    print(f"""
⚠️ Lettura termometrica NON immediata: Referendum ha {ref_in} relazioni in entrata
   e {ref_out} in uscita.
```

```
    Lo schema osservato non riproduce in modo pulito la configurazione attesa
    dalla tesi. Riesaminare i lag e l'eventuale presenza di outlier nelle serie.
""")
```

Sintesi del notebook

Test / Risultato	Atteso (Cap 6)	Osservato
ADF Referendum	p < 0,05 (stazionaria)	p < 0,05 (stazionaria)
ADF Stampa	p < 0,05 (stazionaria)	p < 0,05 (stazionaria)
ADF Governo	p < 0,05 (stazionaria)	p < 0,05 (stazionaria)
Stampa verso Ref lag 3	p ≈ 0,037	**p = 0,037** (significativo)
Stampa verso Ref lag 7	p ≈ 0,0029	**p = 0,0029** (significativo)
Governo verso Ref lag 3	p ≈ 0,038	**p = 0,038** (significativo)
Ref verso Stampa	n.s.	n.s.
Ref verso Governo	n.s.	n.s.
Stampa / Governo	n.s.	n.s.
H3 (4 condizioni)	tutte confermate	**confermata**
YouTube come termometro	sì	**sì**

File salvati in `/kaggle/working/`: - `granger_results.csv` - tabella completa di tutti i test - `granger_heatmap.html` - heatmap matrice causa per effetto - `granger_network.html` - network diagram dello schema causale

Chiusura della pipeline analitica: i notebook 01-05 coprono raccolta, training, predizione/corpus, turning points e Granger. La tesi è ricomponibile integralmente dai notebook (riproducibilità completa).

Capitolo 7: Discussione, limiti e sviluppi futuri

I risultati dei capitoli 4-6 si possono leggere come una sequenza di risposte tecniche alle tre domande di ricerca. Questo capitolo li solleva di un livello: cosa significano questi risultati per la letteratura? Quali implicazioni teoriche ne derivano? Quali sono i limiti del lavoro? E in che modo il quadro empirico di un singolo referendum si relaziona alle traiettorie più ampie del rapporto fra media tradizionali, social media e opinione pubblica? Chiude poi la tesi con i contributi originali, le risposte sintetiche alle RQ e sette tracce di sviluppo futuro.

7.1 Implicazioni rispetto alla letteratura

7.1.1 Risposta diretta a Gayo-Avello (2013)

Il Capitolo 2 ha posto la critica di Gayo-Avello (2013) come bersaglio epistemico: i social media non predicono le elezioni, hanno scritto i critici, perché chi pretende di farlo non corregge i bias strutturali dei dati (selection bias, bot, minoranze rumorose, non-rappresentatività). Questa tesi risponde alla critica in modo non ideologico ma empirico: prende sul serio il bias della minoranza rumorosa, lo corregge esplicitamente con una procedura misurabile (la deduplicazione "una testa, un voto") e *misura l'errore* contro il dato reale.

Il fatto che l'aggregato grezzo dia +2,68 pp e quello deduplicato dia +0,40 pp è la conferma quantitativa che Gayo-Avello aveva ragione *sui dati grezzi* e che la sua critica è superabile *con una correzione metodologica esplicita*. La tesi non riabilita ingenuamente i social come predittori; riabilita un loro uso disciplinato, condizionato a passaggi metodologici espliciti.

In termini operativi: la differenza di 2,28pp fra aggregato grezzo e aggregato deduplicato è la **misura quantitativa del bias della minoranza rumorosa** in questo specifico contesto referendario. È una stima con cui qualsiasi futuro lavoro in stance detection elettorale italiana può confrontarsi.

7.1.2 Conferma e affinamento dell'agenda-setting

Il risultato Stampa → Referendum a lag 3-7 ($p = 0{,}0029$ a lag 7) è una **conferma quantitativa** della teoria dell'agenda-setting di McCombs e Shaw (1972), nel suo nucleo originario: i media tradizionali precedono temporalmente i movimenti di stance sugli stessi temi su YouTube. Il lag di una settimana è coerente con il ciclo della stampa italiana (settimanali, supplementi domenicali, approfondimenti) e con il tempo di assimilazione richiesto dal dibattito tecnico-istituzionale tipico di un referendum costituzionale.

Tre affinamenti importanti rispetto alla teoria classica:

1. **L'agenda-setting non è bidirezionale**. La direzione contraria, Referendum → Stampa, non è significativa. Questo è coerente con il modello classico (la stampa imposta l'agenda, non viceversa) ma in tensione con narrative recenti (post-2016) che vedono i social come fonte di "agenda inversa" che spinge i media tradizionali.

Nel caso del referendum 2026 e del *signal* misurato qui, la direzione è ancora unidirezionale e classica.

2. **Stampa pro-Sì, YouTube pro-No: l'agenda imposta non determina l'esito**. Il risultato più sottile è che la stampa (65-75% pro-Sì) ha effettivamente influenzato la dinamica giornaliera di YouTube, Granger-significativa, *ma non ha trasportato YouTube nella propria distribuzione di stance*. La frammentazione dell'arena pubblica (Cap. 2) è quindi visibile non come *assenza* di influenza, ma come **influenza locale senza convergenza globale**.
3. **Esiste un canale di influenza politico-partitica parallelo**. Governo → Referendum (lag 3, p = 0,038) suggerisce che il consenso al governo trasla, in pochi giorni, sull'orientamento referendario. Questo è un meccanismo distinto dall'agenda-setting in senso stretto: non passa per la stampa, ma per la struttura partitica e ideologica degli utenti. La tesi documenta, in un caso reale, la coesistenza di due meccanismi di influenza temporalmente diversi (uno settimanale via stampa, uno di pochi giorni via allineamento politico).

7.1.3 Termometro vs termostato: il ruolo cognitivo dei social

L'asimmetria Granger osservata (i flussi che entrano in YouTube ma non escono) ha una lettura epistemologica interessante. YouTube nel periodo del referendum si è comportato come un **termometro**, un sensore che misura una temperatura, non un termostato che la regola. Misura ma non impone, riceve ma non trascina, almeno nel senso lineare-causale della Granger causality.

Tre cautele su questa metafora:

- È un **risultato di Granger**, non un risultato sulla causalità in senso filosofico o sociologico ampio. La Granger causality misura la predittività temporale; non identifica meccanismi.
- È un risultato **specifico al referendum 2026 e alle tre serie misurate**. Non è una proprietà generale di YouTube come piattaforma, né di tutti i temi politici.
- Il risultato non esclude **canali di influenza non temporali** (es. nuovi quadri interpretativi che si stabilizzano nel discorso senza un effetto Granger immediato sulle quantità aggregate).

Pur con queste cautele, la pulizia dell'asimmetria, non un singolo coefficiente significativo, ma un'intera struttura coerente di sei coppie testate, autorizza l'interpretazione termometro. È un risultato sobrio: non dice che YouTube sia "irrilevante", dice che nel loop di influenza fra i tre attori misurati, YouTube riceve dagli altri due e non li trascina di ritorno.

7.1.4 Il valore del nowcasting nel blackout demoscopico

Un secondo livello di rilevanza riguarda il **timing**. La legge italiana vieta la pubblicazione di sondaggi politici negli ultimi 15 giorni prima del voto. In quel periodo non esistono, legalmente, *opinion data* tradizionali. La metodologia di questa tesi produce, a partire da

dati pubblici (commenti YouTube), una stima compatibile con il risultato reale a meno di un punto percentuale, *proprio nel periodo in cui i sondaggi sono vietati*.

Questo non sostituisce i sondaggi: la metodologia ha bisogno di un *signal* sociale ricco (un tema dibattuto, con molti commenti, su una piattaforma con bassa presenza di bot) e ha limiti chiari (sotto). Ma copre una finestra temporale strutturalmente cieca della nowcasting elettorale italiana, e lo fa rispettando la lettera della normativa.

7.2 Confronto con la letteratura empirica

7.2.1 Tumasjan et al. (2010) e successori

Tumasjan et al. (2010) hanno aperto la stagione "ottimista" sulla previsione elettorale via social, mostrando una correlazione fra volume di mention su Twitter ed esito delle elezioni tedesche. La presente tesi differisce metodologicamente in modi rilevanti:

- **Volume vs stance**: Tumasjan misura il volume di mention come proxy di consenso. Questa tesi misura *direttamente la stance*, distinguendo pro e contro.
- **Twitter vs YouTube**: Twitter ha più bot e turnover, YouTube ha commenti più lunghi e meno automatizzazione. Per stance detection italiana, YouTube è un campo migliore.
- **Misurazione esplicita dell'errore**: la tesi misura l'errore contro il dato reale, non solo la correlazione qualitativa. È un test più severo.
- **Correzione del bias di rappresentatività**: la deduplicazione esplicita è assente in Tumasjan e nei lavori successori del filone "ottimista".

Il risultato, in altre parole, non è in *competizione* con Tumasjan ma in *continuità critica* con esso: chiede agli stessi dati un livello di disciplina maggiore.

7.2.2 La stampa come predittore: un controfattuale interno

Il dato che la stampa, presa come predittore secco, sbaglierebbe di **>20pp** è un *controfattuale interno* potente. Il corpus della stampa nazionale, su 91 giorni, mostra il 65-75% di articoli pro-riforma. Se uno usasse questo dato come stima di voto, sbaglierebbe di oltre venti punti. Il referendum è stato bocciato (No 53,74%) in un sistema mediatico che era prevalentemente favorevole.

Questo gap non è una sorpresa per la letteratura sui media italiani (concentrazione proprietaria, polarizzazione editoriale), ma rendere il *gap quantificato* è un contributo. È anche, indirettamente, un argomento per la rilevanza di metodologie alternative come quella della tesi: nessuno userebbe ingenuamente la rassegna stampa come termometro di voto, ma quasi nessuno aveva, prima, una metrica numerica per dire *di quanto* il termometro stampa sia disallineato dal voto reale.

7.3 Limiti

I limiti del lavoro vanno esplicitati con la stessa cura con cui sono presentati i risultati. Sei limiti centrali.

7.3.1 Generalizzazione: un solo referendum

Il risultato di errore 0,40 pp è ottenuto su **una sola consultazione referendaria**. Per quanto il referendum 2026 sia un buon caso di studio (alta esposizione mediatica, dibattito polarizzato, blackout demoscopico chiaro), il fatto di osservare una sola realizzazione di un fenomeno aleatorio impedisce di trasformare il numero in una stima di accuratezza generale. La generalizzazione richiede repliche.

7.3.2 Specificità del medium: YouTube non è il pubblico

YouTube ha un bacino di utenti specifico: probabilmente skewed verso fasce di età medio-alte rispetto, per esempio, a TikTok, e con composizione politica eterogenea ma non rappresentativa di tutto l'elettorato. Il risultato 0,40 pp è quindi un risultato *condizionato* alla rappresentatività di YouTube come specchio dell'elettorato italiano in questo specifico tema. Su altri temi (es. politiche giovanili) o in altri momenti, il rapporto di rappresentatività potrebbe essere diverso.

7.3.3 Limiti del classificatore

F1 macro 0,762 è un buon valore, ma non è 1.0. Il modello sbaglia su sarcasmo, ironia, doppia negazione e whataboutism (Cap. 4, §4.1.3). L'errore individuale per fortuna si distribuisce in modo simmetrico fra le classi (e quindi non distorce l'aggregato sistematicamente), ma una stance detection migliore, anche solo F1 0,85, produrrebbe stime più rigorose.

7.3.4 Granger causality non è causalità

Il risultato Stampa → Referendum è di Granger causality, ovvero **predittività temporale**. Non è causalità in senso pieno (nel senso, per esempio, dell'inferenza controfattuale di Pearl o Rubin). Una variabile latente che muova entrambe le serie in modo correlato e con sfasamento sistematico produrrebbe lo stesso pattern. La tesi adotta volutamente un linguaggio cauto su questo punto: "Granger-causa" è scelto al posto di "causa".

7.3.5 La deduplicazione "ultimo commento per autore" è una scelta di policy

Tenere l'ultimo commento dell'intero periodo per ogni autore, anziché il primo, anziché un campionamento casuale o una *aggregazione* delle stance dell'utente nel giorno, è una scelta. È giustificata (l'ultimo è il più aggiornato), ma altre scelte ragionevoli darebbero risultati lievemente diversi. Una sensitivity analysis sulle scelte di deduplicazione sarebbe un naturale complemento.

7.3.6 Bot e attività coordinata

Lo scraping non rimuove esplicitamente bot e attività coordinata. YouTube ha meno bot di Twitter, ma non zero. La deduplicazione "una testa, un voto" è già una protezione parziale (un singolo bot prolifico viene compresso), ma non blocca **reti** di account coordinati. Una misura di robustezza esplicita su questo aspetto è un futuro lavoro identificato (§7.6).

7.4 Tre filoni che si incontrano

Il lavoro si colloca sull'intersezione di tre filoni distinti:

1. **Computational political science**: NLP per analisi del discorso politico. Il contributo qui è metodologico: la deduplicazione e l'uso del contesto padre-risposta.
2. **Nowcasting elettorale**: previsione in tempo reale con dati non strutturati. Il contributo è di *riempire una finestra cieca* (il blackout demoscopico) con un metodo che produce stime accurate.
3. **Causalità nei media digitali**: Granger applicato a serie di opinion data. Il contributo è la struttura asimmetrica e gerarchica documentata sulle tre serie referendum / stampa / governo.

Nessuno dei tre filoni è esaurito da questa tesi: sono *toccati*, non *risolti*. Il valore aggiunto è nella loro **convergenza su un caso reale** con esito documentabile (le urne).

7.5 Cinque contributi originali

7.5.1 Prima applicazione di stance detection a un referendum italiano su YouTube

Non risultano in letteratura applicazioni di stance detection a un referendum italiano nazionale a partire da commenti YouTube nativi. Il lavoro inaugura quindi una linea di ricerca specifica al contesto italiano e a una piattaforma, YouTube, sottoutilizzata rispetto a Twitter nei lavori precedenti, e plausibilmente più adatta all'italiano colloquiale e politico per ragioni strutturali (commenti più lunghi, minore presenza di bot).

7.5.2 Metodologia di deduplicazione "una testa, un voto"

La regola operativa di tenere l'ultimo commento per autore sull'intero periodo è la **firma metodologica della tesi**. Non è banalmente un dettaglio implementativo: è il passaggio che trasforma il dato da misura di engagement a misura di opinione, che corregge il bias della minoranza rumorosa di **2,28 punti percentuali**, e che risponde direttamente alla critica di Gayo-Avello (2013) all'uso ingenuo dei social per la previsione elettorale.

7.5.3 Nowcasting nel blackout demoscopico

La tesi documenta la fattibilità del nowcasting elettorale durante i 15 giorni in cui la legge italiana vieta i sondaggi. L'errore 0,40 pp dimostra che un metodo legalmente ammissibile, basato su dati pubblici e su opensource, può colmare la finestra cieca della previsione

tradizionale. Questo non è solo un risultato accademico: è un contributo metodologico utilizzabile da analisti politici e operatori dei media in qualsiasi consultazione futura.

7.5.4 Mappatura Granger causale a tre serie

L'analisi simultanea di tre serie, referendum YouTube, stampa, consenso governo, è un'estensione rispetto ai lavori che testano coppie isolate. La struttura risultante (stampa upstream puro, governo co-upstream, YouTube downstream) è un piccolo modello empirico del flusso di influenza nell'arena pubblica italiana del 2026, ottenuto con pre-test ADF rigoroso e serie giornaliere grezze (non MA7).

7.5.5 Dataset pubblico di 196k commenti

Il dataset di 196.050 commenti referendum, annotato con stance pro/contro/neutro, è un *asset* riutilizzabile per: - training di modelli successivi su temi politici italiani; - replica del lavoro con architetture diverse; - studi linguistici sull'italiano dei social media in contesto politico.

L'annotazione automatica del dataset è un derivato della pipeline e non sostituisce un eventuale benchmark gold-standard, ma è una base di partenza utile per ulteriori lavori.

7.6 Sviluppi futuri

I limiti del §7.3 si traducono naturalmente in sette tracce di sviluppo, ordinate dalla più immediata alla più ambiziosa.

7.6.1 Repliche su altre consultazioni

La traccia più ovvia: applicare la pipeline alle prossime elezioni politiche italiane, alle prossime elezioni europee, ad altri referendum (qualora indetti). Una metrica di errore replicata su 3-5 consultazioni avrebbe un valore generalizzante che il singolo caso non ha.

7.6.2 Cross-piattaforma: Twitter/X, TikTok, Reddit, Instagram

Estendere lo scraping a piattaforme con profili demografici e dinamiche sociali diversi da YouTube. TikTok in particolare cattura fasce di età più giovani; Twitter/X ha una composizione politica diversa; Reddit ha thread argomentativi più strutturati. Confrontare le previsioni multi-piattaforma e il loro ensemble sarebbe un secondo livello di robustezza.

7.6.3 Modelli di stance detection più potenti

Sostituire BERT con architetture più recenti (Llama, modelli italiani specializzati di nuova generazione) o usare LLM frontier in zero/few-shot. F1 macro 0,85+ sembra alla portata. L'effetto sull'errore di previsione è da quantificare: probabilmente positivo, plausibilmente non drammatico (il classificatore già non distorce sistematicamente l'aggregato).

7.6.4 Robustezza a bot e campagne coordinate

Aggiungere allo scraping una pipeline di **bot detection** e una analisi di reti di account coordinati. La deduplicazione attuale comprime il singolo account prolifico ma non blocca *reti* di account distinti che agiscono in modo concertato. È un punto che diventa rilevante in elezioni più polarizzate o esposte a interferenze esterne.

7.6.5 Sensitivity analysis sulla deduplicazione

Confrontare sistematicamente le scelte di deduplicazione (ultimo commento del giorno vs primo; vs sample casuale; vs aggregazione delle stance dell'utente nel giorno; vs tutto il periodo anziché giornaliero). L'obiettivo è capire la robustezza del risultato 0,40 pp rispetto a queste scelte e identificare la regola ottimale (se esiste).

7.6.6 Modelli causali oltre Granger

Granger causality è un primo livello, ma non identifica meccanismi. Tracce più ambiziose: - **Synthetic control** o **difference-in-differences** su sotto-popolazioni di canali YouTube esposti vs non esposti a video virali specifici. - **Inferenza controfattuale** in chiave Pearl/Rubin sulla stance individuale dopo esposizione a un video di Barbero. - **Network effects**: come si propaga una stance da un thread all'altro, da un canale all'altro?

7.6.7 Real-time pipeline e dashboard di nowcasting

L'ambizione applicativa: trasformare la pipeline in un servizio di nowcasting elettorale **in tempo reale** durante una campagna. Aggiornamenti giornalieri, dashboard pubblica, intervalli di confidenza. Il valore d'uso è ovvio (analisti, redazioni, ricercatori); il rischio etico, *publishing nowcasts* in periodo di blackout legale, va affrontato con cura giuridica.

7.7 Risposte sintetiche alle domande di ricerca

RQ1: I commenti YouTube deduplicati predicono l'esito del referendum con errore < 2pp?

Sì: l'errore osservato è 0,40 pp (45,9% predetto vs 46,26% reale), ampiamente sotto la soglia. Il risultato è ottenuto calcolando l'aggregato sull'intero periodo della campagna (30 ottobre 2025, 23 marzo 2026) e applicando la deduplicazione "una testa, un voto". Senza deduplicazione, l'errore sale a 2,68 pp, sopra la soglia: la correzione è **necessaria**, non cosmetica.

RQ2: I turning points della stance corrispondono a eventi politici e mediatici identificabili?

In parte: i sette turning points documentati (Tabella 5.1) sono tutti riconducibili a un evento (video divulgativi virali di Barbero, endorsement di Marco Rizzo, giornalismo investigativo di Di Pietro e Travaglio + Gratteri, eventi parlamentari come l'approvazione al Senato del 30/10/2025). La segmentazione automatica della serie individua però otto break point con **precisione 62%** e **recall 71%**: la soglia del 70% di H2 non è raggiunta sul criterio di precisione, pur restando valido il repertorio qualitativo degli eventi.

RQ3: Esiste una relazione causale asimmetrica fra stampa, consenso governo e stance referendaria YouTube?

Sì: i test di Granger restituiscono un'asimmetria pulita. Stampa → Referendum è significativo a lag 3 (p = 0,037) e a lag 7 (p = 0,0029). Governo → Referendum è significativo a lag 3 (p = 0,038). Tutti i flussi inversi (Referendum → Stampa, Referendum → Governo) e i flussi laterali (Stampa ↔ Governo) sono non significativi. YouTube nel periodo del referendum si è comportato come **termometro** dell'opinione pubblica, non come **termostato**.

7.8 Cosa lascia il lavoro: tre eredità

7.8.1 Un metodo

La pipeline, scraping quota-aware, fine-tuning con contesto padre-risposta, deduplicazione "una testa, un voto", aggregazione su finestra di nowcasting, Granger su serie grezze, è un manuale operativo riutilizzabile. Non è la sola pipeline possibile, ma è una pipeline che ha funzionato su un caso reale documentato.

7.8.2 Un dato di confronto

L'errore 0,40 pp e l'asimmetria Granger sono **numeri di confronto** per la letteratura futura. Qualunque lavoro successivo potrà chiedersi: *"il mio metodo, su un caso analogo, è meglio o peggio di 0,40 pp?"*. Avere un benchmark concreto migliora l'igiene metodologica del campo.

7.8.3 Un argomento

Il lavoro è anche, implicitamente, un'argomentazione: i social media non sono né predittori magici (Tumasjan ottimista) né rumore irrilevante (Gayo-Avello pessimista). Sono **fonti di opinion data utili condizionate a passaggi metodologici espliciti**. La parola chiave è *condizionate*: il merito non è dei dati grezzi, ma della disciplina con cui li si tratta. Il referendum 2026 è una conferma empirica di questo argomento.

7.9 Chiusura

Il referendum costituzionale del 22-23 marzo 2026 sulla separazione delle carriere in magistratura è stato bocciato dagli elettori italiani con il 53,74% di voti No. I sondaggi tradizionali, vietati dalla legge nei 15 giorni precedenti il voto, non avevano nulla da dire in quella finestra. La stampa, prevalentemente favorevole alla riforma, era disallineata di oltre 20 punti percentuali rispetto al risultato finale. Il segnale più accurato disponibile, in quel periodo, era nei commenti YouTube, a condizione di trattarli con disciplina metodologica.

Questa tesi è la documentazione di quella disciplina e dell'errore residuo che ne risulta: 0,40 punti percentuali. È un risultato sobrio, ottenuto con strumenti standard del campo e con scelte metodologiche esplicite. Non rivendica più di quanto i numeri dicono. Ma quei numeri, presi sul serio, dicono qualcosa di nuovo: che l'opinione pubblica italiana, quando

misurata con cura su un campione adeguato, è oggi *osservabile* anche fuori dai circuiti tradizionali del *polling*. Anche, e forse soprattutto, nei momenti in cui i circuiti tradizionali sono spenti.